



รายงาน Mini Project

Pick and Pay

จัดทำโดย

นายพชรพล	เกตุแก้ว	รหัสนักศึกษา 6610110190
นายภูริ	โททอง	รหัสนักศึกษา 6610110598
นายจักรชัย	ปล้องไหม	รหัสนักศึกษา 6610110428

เสนอ

....

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา

240-319 Embedded System Developer Module

ภาคเรียนที่ 1 ปีการศึกษา 2568

คณะวิศวกรรมศาสตร์ มหาวิทยาลัยสงขลานครินทร์ วิทยาเขตหาดใหญ่

คำนำ

การจัดทำรายงานฉบับนี้มีจุดมุ่งหมายเพื่อศึกษาและนำเสนอแนวคิด “ร้านค้าไร้คนขาย” หรือระบบ Pick and Pay ซึ่งเป็นนวัตกรรมที่ผสมผสานเทคโนโลยีกับธุรกิจค้าปลีก เพื่อให้เกิดประโยชน์ทั้งต่อผู้ประกอบการและผู้บริโภค ผู้จัดทำได้ศึกษา รวบรวม และเรียบเรียงข้อมูลเพื่อสร้างองค์ความรู้ที่สามารถนำไปใช้ประโยชน์ในเชิงวิชาการและการประยุกต์ใช้จริง

ผู้จัดทำขอขอบพระคุณอาจารย์ผู้สอนที่ได้มอบความรู้และแนวทางในการดำเนินการรายงานครั้งนี้ และหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจศึกษาเรื่องร้านค้าไร้คนขาย เพื่อใช้เป็นแนวทางในการเรียนรู้และต่อยอดความรู้ต่อไป

คณะผู้จัดทำ

สารบัญ

คำนำ.....	ข
สารบัญ	ค
สารบัญรูปภาพ.....	ช
บทที่ 1 บทนำ	1
1.1 ที่มาของโครงการ.....	1
1.2 ความสำคัญของโครงการ.....	2
1.3 วัตถุประสงค์ของโครงการ	2
1.4 ขอบเขตของโครงการ	3
1.5 อุปกรณ์ที่ใช้.....	3
1.5.1 Arduino UNO R3.....	3
1.5.2 ESP32	6
1.5.3 ODROID C4.....	9
1.5.4 HX711 and Load cell (5 kg).....	12
1.5.5 Ultrasonic Module HC-SR04.....	15
1.5.6 Micro Servo SG90 and MG90S	17
1.5.7 4x4 Matrix Membrane Keypad	19
1.5.8 PCF8574 8Bit Parallel Expander I2C-Bus	21
1.5.9 RC522 RFID	23
1.5.10 DHT22	25
1.5.11 1602 LCD (Blue Screen) 16x2 LCD with I2C Adapter	27
1.5.12 Tact Switch Push Button.....	29
1.5.13 LED (Light-Emitting Diode)	31
1.5.14 Webcam NWC-560	33

บทที่ 2 การออกแบบและพัฒนา	35
2.1 แผนดำเนินการที่ของทีม	35
2.2 แผนงบประมาณ	36
2.2.1 การจัดสรรงบประมาณ	37
2.3 Schematic	39
2.4 State Diagram ของระบบ	41
2.5 Flowchart ของระบบ	42
2.6 Database Schema	43
2.7 การออกแบบ UI/UX	46
บทที่ 3 การเขียนโปรแกรมและการทำงาน	50
3.1 การจัดการระบบ	50
3.1.1 Embedded System	50
3.1.2 Database System	50
3.2 โปรแกรมและการทำงานของ Front UNO	50
3.3 โปรแกรมและการทำงานของ Back UNO	65
3.4 โปรแกรมและการทำงานของ ESP32	81
บทที่ 4 ชิ้นงานและการทำงาน	97
4.1 ชิ้นงาน	97
4.1.1 Gate Entry	97
4.1.2 Product Shelf	98
4.1.3 Display Screen	100
4.1.4 Combination Board Box	101
4.1.5 Pusher	102
4.1.6 Slider	104

4.1.7	Card Scanner and Keypad.....	105
4.1.8	Product Receiving Box / Disc.....	106
4.1.9	Gate Exit.....	108
4.2	ประสิทธิภาพ.....	109
4.2.1	Gate Entry.....	109
4.2.2	Product Shelf.....	109
4.2.3	Display Screen.....	109
4.2.4	Combination Board Box.....	109
4.2.5	Pusher.....	109
4.2.6	Slider.....	110
4.2.7	Card Scanner and Keypad.....	110
4.2.8	Product Receiving Box / Disc.....	110
4.2.9	Gate Exit.....	110
4.3	การพัฒนา.....	111
4.3.1	Pusher Remodel.....	111
4.3.2	Card Scanner & Keypad Remodel.....	113
4.3.3	Slider Frame.....	114
4.4	ภาพรวมชิ้นงานทั้งหมด.....	115
บทที่ 5 สรุปและข้อเสนอแนะ		116
5.1	สรุปโครงการ.....	116
5.2	ประเมินผลโครงการในมุมมองผู้จัดทำ	116
5.2.1	ด้านการทำงานของระบบ.....	116
5.2.2	ด้านประสิทธิภาพและความถูกต้อง.....	117
5.2.3	ด้านผู้ใช้งาน.....	117

5.2.4	ด้านการเรียนรู้และการทำงานเป็นทีม.....	117
5.3	ข้อเสนอแนะ	118
5.3.1	ข้อเสนอแนะด้านฮาร์ดแวร์.....	118
5.3.2	ข้อเสนอแนะด้านซอฟต์แวร์	118
5.3.3	ข้อเสนอแนะด้านการใช้งานจริง.....	118
5.3.4	ข้อเสนอแนะด้านการพัฒนาในอนาคต	119
ภาคผนวก		120

สารบัญรูปภาพ

ภาพที่ 1 บอร์ด Arduino ENO R3	3
ภาพที่ 2 ขาและการทำงานของ Arduino UNO หรือ ATmega328p	4
ภาพที่ 3 บอร์ด ESP-WROOM32	6
ภาพที่ 4 ขาและการทำงานของ ESP32.....	7
ภาพที่ 5 บอร์ด ODROID-C4.....	9
ภาพที่ 6 ขาและการทำงานของ ODRIOD-C4.....	10
ภาพที่ 7 HX711.....	12
ภาพที่ 8 Load Cell 5 Kg.....	13
ภาพที่ 9 HC-SR04.....	15
ภาพที่ 10 Micro Servo SG90 และ MG90S	17
ภาพที่ 11 4x4 Matrix Membrane Keypad	19
ภาพที่ 12 PCF8574	21
ภาพที่ 13 RC522 RFID	23
ภาพที่ 14 DHT22	25
ภาพที่ 15 LCD (Blue Screen) 16x2 LCD with I2C Adapter.....	27
ภาพที่ 16 Tact Switch Push Button.....	29
ภาพที่ 17 LED.....	31
ภาพที่ 18 NWC-560	33
ภาพที่ 19 Schematic ระบบทั้งหมด	38
ภาพที่ 20 State Diagram	40
ภาพที่ 21 Flow Chart.....	41
ภาพที่ 22 Database Schema	42
ภาพที่ 23 หน้าแรกของเว็บไซต์ร้านค้า.....	45
ภาพที่ 24 การแสดงผลเมื่อเซนเซอร์ตรวจจับเจอลูกค้า	46
ภาพที่ 25 การแสดงผลเมื่อสินค้าที่เลือกตรง และไม่ตรงกับน้ำหนักของชั้นวาง	47
ภาพที่ 26 การแสดงผล Dashboard	47
ภาพที่ 27 Gate Design	93
ภาพที่ 28 Gate Entry ขึ้นงานจริง	93
ภาพที่ 29 Shelf Design	94

ภาพที่ 30 Shelf ชิ้นงานจริง.....	95
ภาพที่ 31 Display Set up	96
ภาพที่ 32 การจัดวางหน้าจอจริง.....	96
ภาพที่ 33 Combination Board Box Design.....	97
ภาพที่ 34 Combination Board Box ชิ้นงานจริง.....	97
ภาพที่ 35 Pusher and Platform Design.....	98
ภาพที่ 36 Pusher and Platform ชิ้นงานจริง.....	98
ภาพที่ 37 Slider and Frame Design	99
ภาพที่ 38 รูปภาพ Slider จริง.....	100
ภาพที่ 39 Card Scanner (RFID) and Keypad Design	100
ภาพที่ 40 Card Scanner (RFID) and Keypad ชิ้นงานจริง	101
ภาพที่ 41 Product Receiving Box Design	102
ภาพที่ 42 Product Receiving Box ชิ้นงานจริง.....	102
ภาพที่ 43 Gate Exit Design	103
ภาพที่ 44 Gate Exit ชิ้นงานจริง	103
ภาพที่ 45 Pusher Remodel Design	106
ภาพที่ 46 Pusher Arm Design.....	107
ภาพที่ 47 Scanner and Keypad Remodel Design.....	108
ภาพที่ 48 Slider Remodel Design	109
ภาพที่ 49 Pick and Pay.....	110

บทที่ 1

บทนำ

1.1 ที่มาของโครงการ

ในยุคปัจจุบันที่เทคโนโลยีเข้ามามีบทบาทในทุกด้านของชีวิตประจำวัน ผู้คนต่างมองหาวิธีการดำเนินชีวิตที่มีความสะดวก รวดเร็ว และยืดหยุ่นมากยิ่งขึ้น แนวคิดของ "ร้านค้าไร้คนขาย" หรือระบบ Pick and Pay จึงได้ถือกำเนิดขึ้นเพื่อรองรับการเปลี่ยนแปลงของพฤติกรรมผู้บริโภคและความต้องการในการดำเนินธุรกิจยุคใหม่

แนวคิดนี้มีที่มาจากวัตถุประสงค์หลักสองประการ ได้แก่ การอำนวยความสะดวกแก่ลูกค้า และการเพิ่มความสะดวกในการซื้อสินค้าให้กับลูกค้า สำหรับผู้ค้าแล้ว ระบบร้านค้าไร้คนขายช่วยให้สามารถเปิดร้านค้าได้โดยไม่ต้องมีพนักงานประจำร้านตลอดเวลา ลดภาระเรื่องต้นทุนแรงงาน และยังสามารถบริหารจัดการร้านได้จากระยะไกล ซึ่งเป็นประโยชน์อย่างยิ่งสำหรับผู้ประกอบการรายย่อยหรือเจ้าของร้านที่มีเวลาจำกัด ในขณะเดียวกัน ลูกค้าเองก็ได้รับประโยชน์จากความสะดวกสบายของระบบนี้ โดยสามารถเข้ามาเลือกซื้อสินค้าได้ตามต้องการ ในเวลาที่สะดวก โดยไม่จำเป็นต้องรอการบริการจากพนักงาน หรือเผชิญกับเวลาทำการที่จำกัด ลูกค้าสามารถชำระเงินได้เองผ่านระบบอัตโนมัติ ไม่ว่าจะเป็นเครื่องรูดบัตร หรือระบบ QR Code และ Mobile Banking ซึ่งตอบโจทย์ไลฟ์สไตล์ของคนในยุคดิจิทัลที่มุ่งเน้นความรวดเร็วและประสิทธิภาพในการใช้เวลา

นอกจากนี้ ระบบ Pick and Pay ยังสามารถช่วยส่งเสริมความซื่อสัตย์ในสังคมผ่านรูปแบบธุรกิจที่เชื่อมั่นในความรับผิดชอบของลูกค้าในการจ่ายเงินอย่างถูกต้อง และยังสามารถพัฒนาไปสู่รูปแบบร้านค้าที่มีระบบตรวจสอบอัจฉริยะ เช่น กล้องวงจรปิด ปัญญาประดิษฐ์ และเซ็นเซอร์ต่าง ๆ เพื่อเพิ่มความปลอดภัยและความน่าเชื่อถือ

ด้วยเหตุนี้เอง โครงการร้านค้าไร้คนขาย Pick and Pay จึงไม่ได้เป็นเพียงนวัตกรรมทางธุรกิจเท่านั้น แต่ยังเป็นส่วนหนึ่งของการพัฒนาโครงสร้างเศรษฐกิจและสังคมในระดับชุมชน ซึ่งมุ่งเน้นการใช้ทรัพยากรอย่างคุ้มค่า เสริมสร้างความยั่งยืน และตอบสนองต่อวิถีชีวิตของคนรุ่นใหม่อย่างมีประสิทธิภาพ

1.2 ความสำคัญของโครงการ

โครงการร้านค้าไร้คนขาย (Pick and Pay) มีความสำคัญอย่างมากในยุคที่เทคโนโลยีเข้ามามีบทบาทในชีวิตประจำวัน โดยเฉพาะในภาคการค้าปลีกที่ต้องการความยืดหยุ่น ประหยัดเวลา และลดต้นทุนในการดำเนินงาน สำหรับผู้ประกอบการ การเปิดร้านค้าแบบไร้คนขายช่วยลดภาระในการจ้างพนักงาน ลดค่าใช้จ่ายด้านแรงงาน และสามารถบริหารจัดการร้านได้โดยไม่ต้องอยู่เฝ้าตลอดเวลา อีกทั้งยังสามารถเปิดบริการได้ตลอด 24 ชั่วโมง เพิ่มโอกาสในการสร้างรายได้อย่างต่อเนื่องและมีประสิทธิภาพมากยิ่งขึ้น

ในด้านของผู้บริโภค ระบบนี้ช่วยอำนวยความสะดวกในการเลือกซื้อสินค้าได้อย่างอิสระ ไม่ต้องเสียเวลาในการรอคิวหรือพนักงาน ลูกค้าสามารถชำระเงินได้ด้วยตนเองผ่านระบบอัตโนมัติ เช่น การสแกน QR Code หรือการใช้ Mobile Banking ซึ่งตอบโจทย์วิถีชีวิตของผู้บริโภคในยุคดิจิทัลที่เน้นความรวดเร็ว คล่องตัว และไม่จำกัดเวลาในการจับจ่าย

นอกจากนี้ โครงการนี้ยังเป็นการส่งเสริมการประยุกต์ใช้เทคโนโลยีในภาคธุรกิจให้เกิดประโยชน์ในระดับชุมชน ช่วยให้ผู้ประกอบการรายย่อยสามารถเรียนรู้และพัฒนาอาชีพให้สอดคล้องกับยุคสมัย อีกทั้งยังสร้างวัฒนธรรมแห่งความซื่อสัตย์และความรับผิดชอบในสังคม ผ่านการซื้อขายที่ลูกค้าดำเนินการด้วยตนเอง ดังนั้นร้านค้าไร้คนขายจึงเป็นทั้งนวัตกรรมทางธุรกิจและแนวทางใหม่ของการพัฒนาเศรษฐกิจชุมชนอย่างยั่งยืน

1.3 วัตถุประสงค์ของโครงการ

- 1.3.1 เพื่ออำนวยความสะดวกแก่ผู้ประกอบการหรือเจ้าของร้าน ให้สามารถดำเนินการได้โดยไม่ต้องดูแลร้านตลอดเวลา
- 1.3.2 เพื่อเพิ่มความสะดวกให้แก่ลูกค้าในการเลือกซื้อสินค้าได้อย่างอิสระ รวดเร็ว และไม่จำกัดช่วงเวลา

1.4 ขอบเขตของโครงการ

โครงการร้านค้าไร้คนขาย (Pick and Pay) ได้กำหนดขอบเขตในการดำเนินงานไว้เพื่อให้สอดคล้องกับรูปแบบการใช้งานจริงในเบื้องต้น ดังนี้:

- 1.4.1 ร้านค้าจะให้บริการแก่ลูกค้า ครั้งละ 1 คนเท่านั้น เพื่อความเป็นส่วนตัว ความปลอดภัย และเพื่อควบคุมระบบการซื้อขายภายในร้านให้เป็นระเบียบ
- 1.4.2 สินค้าที่จัดจำหน่ายในร้านจะมีจำนวนจำกัดในช่วงเริ่มต้น ได้แก่
 - ขนมห่อสำเร็จรูป
 - น้ำดื่มขวดเล็ก

ซึ่งเป็นสินค้าที่บริโภคได้ง่าย มีความต้องการสูง และเหมาะสมกับระบบการชำระเงินแบบอัตโนมัติ การกำหนดขอบเขตในลักษณะนี้มีวัตถุประสงค์เพื่อทดสอบความเป็นไปได้ของระบบร้านค้าไร้คนขายในขนาดย่อม ก่อนที่จะมีการขยายรูปแบบหรือเพิ่มเติมสินค้าและฟังก์ชันในอนาคต

1.5 อุปกรณ์ที่ใช้

1.5.1 Arduino UNO R3



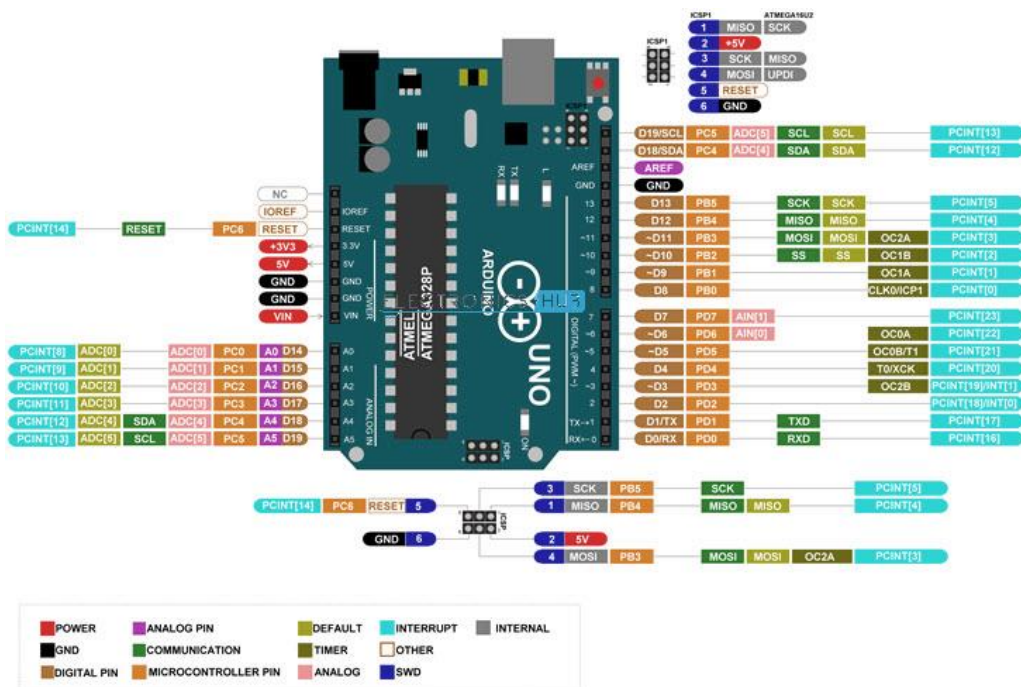
ภาพที่ 1 บอร์ด Arduino UNO R3

Arduino Uno R3 เป็นบอร์ดไมโครคอนโทรลเลอร์ที่ได้รับความนิยมอย่างมากในวงการอิเล็กทรอนิกส์และการพัฒนาอุปกรณ์อัจฉริยะ เพราะใช้งานง่าย เรียนรู้ได้รวดเร็ว และมีความสามารถที่เหมาะสมกับงานตั้งแต่ระดับพื้นฐานไปจนถึงระดับกลาง หัวใจหลักของ Arduino Uno R3 คือชิป ATmega328P ซึ่งเป็นไมโครคอนโทรลเลอร์แบบ 8 บิต ทำงานด้วยความเร็ว 16 MHz มีหน่วยความจำ Flash ขนาด 32KB ใช้สำหรับ

เก็บโปรแกรมที่ผู้ใช้อัปโหลดลงไป ตัวบอร์ดสามารถรับข้อมูลจากเซนเซอร์ และควบคุมอุปกรณ์ต่างๆ เช่น มอเตอร์ไฟ LED หรือจอแสดงผล ได้ผ่านขาคิจิทัลและแอนะล็อก

การเขียนโปรแกรม Arduino ทำผ่านซอฟต์แวร์ที่เรียกว่า Arduino IDE ซึ่งใช้ภาษาคำสั่งคล้ายภาษา C/C++ และมีฟังก์ชันหลักสองส่วนคือ setup() ซึ่งทำงานตอนเริ่มต้น และ loop() ที่ทำงานซ้ำๆ ตลอดเวลาระหว่างที่บอร์ดเปิดอยู่ ผู้ใช้สามารถอัปโหลดโปรแกรมผ่านสาย USB ได้โดยตรง

บอร์ดนี้มีขาคิจิทัล 14 ขา (6 ขาเป็น PWM : Pulse Width Modulation) และขาแอนะล็อก 6 ขา ที่ใช้เชื่อมต่อกับอุปกรณ์ภายนอก นอกจากนี้ยังมีพอร์ตจ่ายไฟและแรงดันไฟฟ้าควบคุมในตัว ทำให้สามารถใช้งานได้ทั้งจากคอมพิวเตอร์หรือแหล่งจ่ายไฟภายนอก



ภาพที่ 2 ขาและการใช้งานของ Arduino UNO หรือ ATmega328p

โครงสร้างของบอร์ด

โครงสร้างของ Arduino Uno R3 ประกอบด้วยส่วนประกอบที่สำคัญดังนี้

1. ไมโครคอนโทรลเลอร์หลัก (ATmega328P): เป็นหน่วยประมวลผลหลัก มีหน่วยความจำ Flash 32KB สำหรับเก็บโปรแกรม, SRAM 2KB สำหรับจัดการข้อมูลชั่วคราว และ EEPROM 1KB สำหรับเก็บข้อมูลถาวร
2. วงจรควบคุม USB-to-Serial (ATmega16U2): ทำหน้าที่แปลงสัญญาณ USB จากคอมพิวเตอร์เป็นสัญญาณอนุกรม (Serial) เพื่อใช้ในการอัปโหลดโปรแกรมและสื่อสารกับบอร์ด
3. พอร์ตอินพุต/เอาต์พุต:
 - ขาดิจิทัล 14 ขา (Digital I/O pins) โดยมี 6 ขารองรับ PWM (Pulse Width Modulation) สำหรับควบคุมความสว่างของไฟ LED หรือความเร็วของมอเตอร์
 - ขาแอนะล็อก 6 ขา (Analog Input pins) สำหรับอ่านค่าจากเซนเซอร์ที่ส่งสัญญาณเป็นแรงดันไฟฟ้า
4. พอร์ตจ่ายไฟ: รองรับแรงดันไฟฟ้าผ่านสาย USB หรือแหล่งจ่ายไฟภายนอก (7–12V โดยทั่วไป) และมีขา 3.3V, 5V และ GND สำหรับต่อวงจรภายนอก
5. คริสตัลออสซิลเลเตอร์ (16 MHz): ทำหน้าที่กำหนดจังหวะการทำงานของไมโครคอนโทรลเลอร์
6. พอร์ต USB Type-B: สำหรับเชื่อมต่อเข้ากับคอมพิวเตอร์เพื่ออัปโหลดโปรแกรมและจ่ายไฟ
7. ขา ICSP (In-Circuit Serial Programming): สำหรับโปรแกรมไมโครคอนโทรลเลอร์โดยตรง

การทำงานของอุปกรณ์

การทำงานของ Arduino Uno R3 เริ่มต้นจากการเขียนโปรแกรมในซอฟต์แวร์ Arduino IDE ซึ่งเป็นโปรแกรมฟรีและใช้งานง่าย ผู้ใช้เขียนโค้ดด้วยภาษาที่มีพื้นฐานคล้าย C/C++ และโครงสร้างโปรแกรมแบ่งออกเป็นสองส่วนหลักคือ

setup(): ใช้กำหนดค่าเริ่มต้น เช่น ตั้งค่าขา I/O หรือเริ่มต้นการสื่อสาร

loop(): ส่วนที่ทำงานซ้ำ ๆ อย่างต่อเนื่อง ตราบใดที่บอร์ดยังเปิดอยู่

เมื่อเขียนโปรแกรมเสร็จ ผู้ใช้สามารถอัปโหลดไปยังบอร์ดผ่านสาย USB ซึ่ง Arduino IDE จะคอมไพล์โค้ดแล้วส่งไปยัง ATmega328P เพื่อเก็บไว้ใน Flash memory หลังจากนั้นไมโครคอนโทรลเลอร์จะเริ่มทำงานตามคำสั่ง เช่น อ่านค่าจากเซนเซอร์ ประมวลผล และส่งสัญญาณควบคุมอุปกรณ์เอาต์พุต เช่น LED มอเตอร์ หรือจอแสดงผล

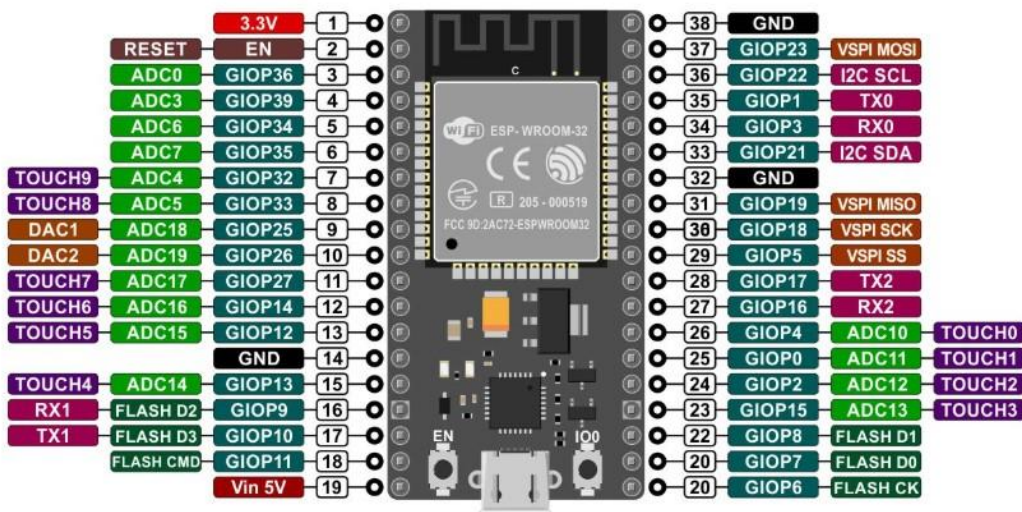
1.5.2 ESP32



ภาพที่ 3 บอร์ด ESP-WROOM32

ESP32 เป็นไมโครคอนโทรลเลอร์แบบ 32 บิต ที่ถูกพัฒนาโดยบริษัท Espressif Systems ซึ่งมีจุดเด่นที่สำคัญคือ รองรับการเชื่อมต่อไร้สายทั้ง Wi-Fi และ Bluetooth ในตัว จึงเหมาะสำหรับงานด้าน Internet of Things (IoT) เป็นอย่างยิ่ง ตัวบอร์ดใช้ชิพแบบ Dual-core ที่มีความเร็วสูงถึง 240 MHz พร้อมด้วยหน่วยความจำขนาดใหญ่ (RAM ~520KB และ Flash ~4MB ในรุ่น DevKit ทั่วไป) ซึ่งมากกว่า Arduino Uno หลายเท่า อีกทั้งยังมี I/O มากมายที่รองรับการเชื่อมต่อแบบดิจิทัล แอนะล็อก PWM สื่อสารแบบ SPI, I2C, UART รวมถึงสามารถตรวจจับการสัมผัส (Touch Sensor) และแปลงสัญญาณดิจิทัลเป็นแอนะล็อก (DAC) ได้ด้วย

ด้วยความสามารถที่ครบเครื่อง ESP32 จึงถูกนำไปประยุกต์ใช้ในงานหลากหลาย เช่น ระบบบ้านอัจฉริยะ เครื่องวัดอุณหภูมิและความชื้นที่ส่งข้อมูลผ่าน Wi-Fi อุปกรณ์ควบคุมจากระยะไกล หรือแม้แต่หุ่นยนต์ที่สามารถเชื่อมต่อกับแอปพลิเคชันบนมือถือ ข้อควรระวังคือ ESP32 ใช้แรงดันไฟ 3.3V ซึ่งต่างจาก Arduino Uno ที่ใช้ 5V ผู้ใช้จึงต้องระมัดระวังไม่ให้จ่ายไฟเกินที่กำหนด แม้จะมีความสามารถสูง แต่ราคาของ ESP32 กลับใกล้เคียงกับ Arduino Uno ทำให้มันกลายเป็นทางเลือกยอดนิยมสำหรับผู้ที่ต้องการพัฒนาอุปกรณ์อัจฉริยะหรือระบบเชื่อมต่ออินเทอร์เน็ตในราคาประหยัด



ภาพที่ 4 ขาและการใช้งานของ ESP32

โครงสร้างของบอร์ด

โครงสร้างหลักของ ESP32 ประกอบด้วยส่วนสำคัญดังนี้

1. หน่วยประมวลผล (CPU): Dual-core Tensilica Xtensa LX6 ความเร็วสูงสุด 240 MHz
2. หน่วยความจำ (RAM): ประมาณ 520 KB และ Flash memory มาตรฐาน ~4 MB (บางรุ่นอาจขยายได้ถึง 16 MB)
3. โมดูลสื่อสารไร้สาย: Wi-Fi (802.11 b/g/n) และ Bluetooth (ทั้ง Classic และ BLE)
4. พอร์ต I/O: มากกว่า 30 ขา รองรับดิจิทัล อินพุต/เอาต์พุต, PWM, ADC (Analog to Digital Converter), DAC (Digital to Analog Converter), Touch Sensor, SPI, I2C, UAR
5. แรงดันไฟฟ้า: ทำงานที่ 3.3V (ต้องระวังห้ามจ่ายไฟ 5V ตรงเข้าขา I/O)
6. วงจรเสริม: มีตัวควบคุมพลังงาน (Power Management Unit) ที่ช่วยให้บอร์ดทำงานในโหมดประหยัดพลังงานได้

การทำงานของบอร์ด

ESP32 ทำงานโดยการโหลดโปรแกรมลงไปในหน่วยความจำ Flash ผ่านสาย USB หรือโปรแกรมเมอร์ (เช่น UART หรือ JTAG) การเขียนโค้ดสามารถใช้ Arduino IDE, PlatformIO, หรือ ESP-IDF (Espressif IoT Development Framework) เมื่อเริ่มทำงาน ซีพียูจะประมวลผลคำสั่งตามโปรแกรม เช่น อ่านค่าจากเซนเซอร์ แปลงเป็นข้อมูลดิจิทัล และส่งผ่าน Wi-Fi ไปยังเซิร์ฟเวอร์ หรือควบคุมอุปกรณ์เอาต์พุต เช่น รีเลย์ มอเตอร์ หรือ จอแสดงผล นอกจากนี้ ESP32 ยังมีโหมด Deep Sleep ที่ช่วยประหยัดพลังงานอย่างมาก เหมาะกับอุปกรณ์ที่ใช้แบตเตอรี่เป็นแหล่งจ่ายไฟ เป็นลำดับดังนี้

1. การจ่ายพลังงาน (Power On) เมื่อจ่ายไฟเลี้ยง (3.3V ผ่าน USB หรือแหล่งจ่ายภายนอก) ESP32 จะเริ่มต้นระบบ (Booting)
2. การบูต (Boot Process) Bootloader ที่อยู่ใน Flash จะทำงานก่อน โดยตรวจสอบว่ามีโปรแกรมใหม่ที่ต้องแฟลชเข้ามาหรือไม่ ถ้าไม่มี ก็จะเริ่มรันโปรแกรมที่ถูกเก็บไว้ในหน่วยความจำ Flash
3. การโหลดและเริ่มต้นโปรแกรม (Initialization) ESP32 โหลดโค้ดที่ผู้ใช้เขียน (ผ่าน Arduino IDE หรือ ESP-IDF) ฟังก์ชัน setup() จะถูกเรียกเพื่อกำหนดค่าเริ่มต้น เช่น ตั้งค่า Wi-Fi, Bluetooth, ขา I/O หรือเริ่มการสื่อสารกับเซนเซอร์
4. การทำงานหลัก (Main Execution Loop) หลังจาก setup() เสร็จแล้ว จะเข้าสู่ฟังก์ชัน loop() (หรือ task ใน FreeRTOS) ที่ทำงานซ้ำ ๆ
5. การสื่อสาร (Communication) ESP32
 - ใช้ Wi-Fi เพื่อเชื่อมต่อกับเครือข่ายอินเทอร์เน็ตหรือ LAN
 - ใช้ Bluetooth เพื่อสื่อสารกับสมาร์ทโฟนหรืออุปกรณ์ใกล้เคียง
 - ใช้พอร์ตสื่อสาร SPI, I2C, UART สำหรับเชื่อมกับชิปหรืออุปกรณ์เสริม
6. การจัดการพลังงาน (Power Management) ถ้าไม่ได้ใช้งานต่อเนื่อง ESP32 สามารถเข้าสู่โหมด Sleep/Deep Sleep เพื่อลดการใช้พลังงาน ตื่นขึ้นมา (Wake-up) ได้จากตัวจับเวลา (Timer), สัญญาณจาก GPIO, หรือสัญญาณไร้สาย
7. การทำงานต่อเนื่อง (Continuous Operation) ขึ้นตอน loop() จะวนซ้ำไปเรื่อย ๆ ตรวจจับที่บอร์ดยังได้รับพลังงาน ทำให้ ESP32 สามารถตรวจจับสภาพแวดล้อมและควบคุมอุปกรณ์แบบ Real-time ได้

1.5.3 ODROID C4



ภาพที่ 5 บอร์ด ODROID-C4

ODROID-C4 คือคอมพิวเตอร์ขนาดเล็กแบบบอร์ดเดียว (Single Board Computer: SBC) ที่ถูกออกแบบและผลิตโดยบริษัท Hardkernel จากประเทศเกาหลีใต้ จุดมุ่งหมายหลักของบอร์ดรุ่นนี้คือการเป็นคอมพิวเตอร์ต้นทุนต่ำแต่มีสมรรถนะสูง เหมาะสำหรับงานที่หลากหลาย ตั้งแต่การใช้งานทั่วไปเหมือนคอมพิวเตอร์ตั้งโต๊ะขนาดเล็ก การเป็นศูนย์กลางความบันเทิงภายในบ้าน (Home Theater PC) การทำงานเป็นเซิร์ฟเวอร์ส่วนตัว ไปจนถึงงานด้านวิศวกรรมและวิจัยที่ต้องใช้แพลตฟอร์มสำหรับทดลองพัฒนา ODROID-C4 จัดอยู่ในกลุ่มผลิตภัณฑ์ที่มีบทบาทคล้ายกับ Raspberry Pi ซึ่งเป็น SBC ที่ได้รับความนิยมสูง แต่ ODROID-C4 โดดเด่นที่การใช้ชิปเซ็ต Amlogic S905X3 แบบ Quad-core ARM Cortex-A55 พร้อม GPU Mali-G31 MP2 และหน่วยความจำ DDR4 ขนาด 4GB ส่งผลให้มีประสิทธิภาพสูงกว่าคู่แข่งหลายรุ่น ทั้งในด้านความเร็ว ความเสถียร และความสามารถในการรองรับมัลติมีเดียความละเอียดสูง

โครงสร้างบอร์ด

1. SoC / CPU: Amlogic S905X3 — Quad-core ARM Cortex-A55 ประมวลผลที่ระดับประมาณ 2.0 GHz (ข้อมูลโรงงานระบุความถี่ ~2.0–2.016 GHz) ซึ่งออกแบบบนกระบวนการผลิต 12nm ให้ประสิทธิภาพต่อวัตต์ดีขึ้นและ IPC สูงกว่ารุ่นก่อน
2. GPU: ARM Mali-G31 MP2 — รองรับการเรนเดอร์กราฟิกและการถอดรหัสวิดีโอฮาร์ดแวร์ (รองรับ 4K@60Hz, HDR, CEC ขึ้นกับระบบปฏิบัติการและไดรเวอร์)
3. หน่วยความจำ (RAM): DDR4 ขนาด 4GB (บอร์ดรุ่นมาตรฐานเป็น 4GB) — DDR4 ช่วยเพิ่มแบนด์วิดท์และความเร็วเมื่อเทียบกับ DDR3/DDR2

4. หน่วยเก็บข้อมูล: ช่องใส่ microSD สำหรับบูทและเก็บ OS และ eMMC connector (สามารถต่อ eMMC module เพื่อให้ได้การอ่าน/เขียนที่เร็วและทนทานกว่า microSD)
5. พอร์ต I/O และการเชื่อมต่อ: HDMI 2.0 (รองรับ 4K@60Hz), 4× USB 3.0 (host), 1× USB 2.0 OTG (micro-USB), 1× Gigabit Ethernet (RJ-45), 40-pin GPIO header (2.54 mm), audio header (บางบอร์ด/อุปกรณ์เสริม), debug UART
6. แหล่งจ่ายไฟ & พลังงาน: แนะนำใช้ DC 12V/2A (ข้อมูลแนะนำจากผู้ผลิต/ผู้จัดจำหน่าย) เพื่อรองรับอุปกรณ์ต่อพ่วงและ USB
7. การระบายความร้อน: บอร์ดมาพร้อมฮีตซิงค์ขนาดเล็ก และออกแบบให้มีช่องติดตั้งพัดลมเสริมสำหรับงานที่ต้องรันหนักเป็นเวลานาน (เพื่อป้องกัน thermal throttling)
8. ภายนอก: ขนาดบอร์ดประมาณ 85 × 56 mm (เล็ก พกพาง่าย) — รูปแบบและตำแหน่งพอร์ตออกแบบเพื่อการใช้งานบนโน้ตบุ๊กหรือใส่เคสพร้อมช่องระบาย

3.3V Power	1		2	5.0V Power
I2C_EE_M2_SDA/GPIOX_17(#)	3		4	5.0V Power
I2C_EE_M2_SCL/GPIOX_18(#)	5		6	Ground
PWM_C/GPIOX_5(#)	7		8	GPIOX_12(#)/UART_EE_A_TX
Ground	9		10	GPIOX_13(#)/UART_EE_A_RX
PWM_D/GPIOX_3(#)	11		12	GPIOX_16(#)/PWM_E
GPIOX_4(#)	13		14	Ground
GPIOX_7(#)	15		16	GPIOX_0(#)
3.3V Power	17		18	GPIOX_1(#)
SPI_A_MOSI/GPIOX_8(#)	19		20	Ground
SPI_A_MISO/GPIOX_9(#)	21		22	GPIOX_2(#)
SPI_A_SCLK/GPIOX_11(#)	23		24	GPIOX_10(#)/SPI_A_SS0
Ground	25		26	GPIOH_6(#)
I2C_EE_M3_SDA/GPIOA_14(#)	27		28	GPIOA_15(#)/I2C_EE_M3_SCL
UART_EE_A_CTS/GPIOX_14(#)	29		30	Ground
UART_EE_A_RTS/GPIOX_15(#)	31		32	GPIOH_7(#)
PWM_A/GPIOX_6(#)	33		34	Ground
PWM_B/GPIOX_19(#)	35		36	GPIOH_5(#)/PWM_F
ADC.AIN2	37		38	VDDIO_A01V8
Ground	39		40	ADC.AIN0

ภาพที่ 6 ขาและการทำงานของ ODRIOD-C4

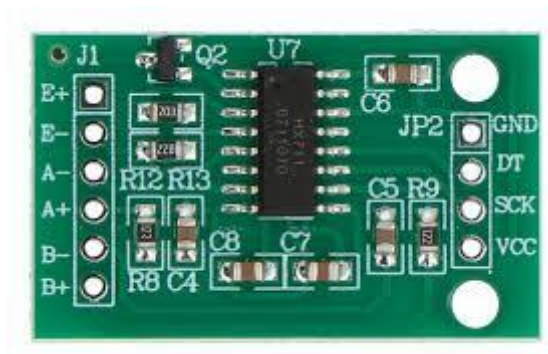
การทำงานของอุปกรณ์

การทำงานของ ODROID-C4 สามารถอธิบายเป็นขั้น ๆ ดังนี้:

1. ขั้นตอนบูท (Boot flow): บอร์ดจะอ่านบูตโหลดเดอร์/แฟ้มระบบปฏิบัติการจาก eMMC (ถ้ามีและตั้งค่าให้บูทจาก eMMC) หรือจาก microSD ตามลำดับที่ตั้งไว้ — Boot loader (เช่น U-Boot ที่พอร์ตโดยผู้พัฒนา) จะเริ่ม kernel และ init system ของ Linux/Android. การตั้งค่า boot และ image (Ubuntu/Android/CoreELEC) มักถูกเตรียมเป็น image ที่ผู้ใช้ก็อปใส่ microSD หรือแฟลชลง eMMC
2. การประมวลผลและกราฟิก: CPU (Cortex-A55) ทำงานกับ kernel ในการจัดสรรงานทั่วไป (process scheduling, I/O, network) ขณะที่ GPU (Mali-G31) จัดการเรนเดอร์ภาพและเร่งการถอดรหัสวิดีโอ (hardware accelerated decoding) เมื่อไดรเวอร์และซอฟต์แวร์ (เช่น Kodi หรือ Android video player) รองรับการใช้อาร์ดแวร์. เมื่อรันวิดีโอ 4K@60Hz บอร์ดจะใช้พอร์ต HDMI 2.0 ส่งสัญญาณไปยังจอ
3. การสื่อสาร I/O และเครือข่าย: พอร์ต USB 3.0 ให้แบนด์วิดท์สูงสำหรับอุปกรณ์จัดเก็บข้อมูลภายนอก (เช่น SSD/USB-HDD) และอุปกรณ์เสริมอื่น ๆ; Gigabit Ethernet ใช้สำหรับเชื่อมต่อเครือข่ายความเร็วสูง เช่น เป็น NAS หรือ media server. GPIO header ให้สัญญาณดิจิทัลและบัสอย่าง UART, I2C, SPI (แล้วแต่เฟิร์มแวร์/ด็อกคิวเมนต์) เพื่อเชื่อมเซนเซอร์ โมดูลสื่อสาร หรือบอร์ดควบคุมอื่น ๆ
4. การจัดการพลังงานและความร้อน: เมื่อระบบทำงานหนัก เช่น รัน container หลายตัวหรือถอดรหัสวิดีโอ 4K เครื่องจะสร้างความร้อน ฮีตซิงค์และ/หรือพัดลมช่วยลดความร้อนเพื่อรักษาความเร็วคงที่ (ป้องกัน throttling). การจ่ายไฟที่มีประสิทธิภาพและเสถียร (เช่น 12V/2A ตามคำแนะนำ) ช่วยให้ USB และอุปกรณ์ต่อพ่วงทำงานได้อย่างมั่นคง

1.5.4 HX711 and Load cell (5 kg)

HX711



ภาพที่ 7 HX711

HX711 เป็นชิปแบบ ADC (Analog to Digital Converter) ที่ถูกออกแบบมาเฉพาะสำหรับอ่านสัญญาณจาก Load Cell โดยเฉพาะ มาพร้อมวงจรขยายสัญญาณภายใน (PGA – Programmable Gain Amplifier) ทำให้สามารถขยายสัญญาณที่เล็กมากให้ใหญ่มากพอที่ ADC จะอ่านค่าได้

โครงสร้างภายใน ประกอบด้วย:

- PGA (Programmable Gain Amplifier) – เพิ่มสัญญาณให้ใหญ่ขึ้น ปรับค่าได้ 32x, 64x, หรือ 128x
- ADC 24-bit – แปลงสัญญาณแอนะล็อกจาก Load Cell เป็นดิจิทัล
- Serial Interface – ใช้เพียง 2 สายในการสื่อสาร (DT และ SCK)
- Noise Filter – กรองสัญญาณรบกวนเพื่อความแม่นยำ

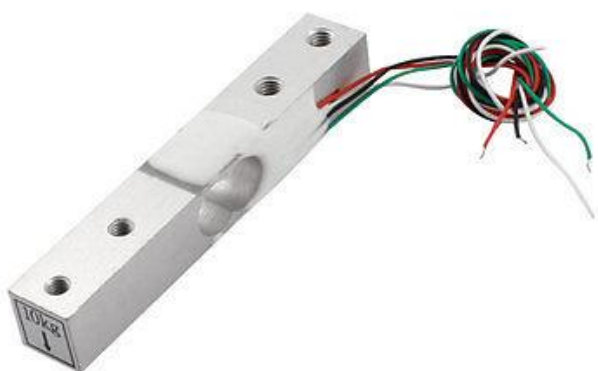
คุณสมบัติของ HX711

รายการ	รายละเอียด
แรงดันไฟเลี้ยง	2.6V – 5.5V
ความละเอียด ADC	24 บิต
อัตราขยาย	32x, 64x, 128x
อินเทอร์เฟซข้อมูล	2 สาย (Data, Clock)
แพ็คเกจ	SOP-16 (นิยมใช้บนโมดูล PCB)

การทำงานภายใน HX711

1. รับสัญญาณแรงดันจาก Load Cell ผ่านขา A+ และ A-
2. ขยายสัญญาณ ผ่าน PGA โดยค่าขยายส่วนใหญ่จะอยู่ที่ 128 เท่า (ค่าดีฟอลต์)
3. แปลงสัญญาณแอนะล็อกเป็นดิจิทัล ด้วย ADC แบบ 24 บิต – ให้ความละเอียดสูงมาก
4. ส่งข้อมูลแบบดิจิทัลออกทางพอร์ต Data (DT) ไปยังไมโครคอนโทรลเลอร์ โดยใช้สัญญาณนาฬิกาจาก SCK (Serial Clock)
5. ข้อมูลที่ส่งออกมาจะเป็นค่า Digital Output ที่สามารถนำไปคำนวณหรือแสดงผลได้

Load Cell



ภาพที่ 8 Load Cell 5 Kg

Load Cell คือเซนเซอร์ที่ใช้วัด “แรง” หรือ “น้ำหนัก” โดยเปลี่ยนแรงกดหรือแรงดึงให้เป็นสัญญาณไฟฟ้าแบบแอนะล็อก โดยปกติแล้วจะใช้ Strain Gauge ซึ่งเป็นตัวต้านทานไฟฟ้าชนิดพิเศษ ที่เมื่อมีแรงมากกระทำบนวัสดุที่ยึดติดกับ Strain Gauge ค่า “ความต้านทาน” จะเปลี่ยนแปลง ซึ่งสามารถตรวจจับและแปลงเป็นแรงดันไฟฟ้าได้

ประเภทของ Load Cell

1. Single Point Load Cell – ใช้ในเครื่องชั่งน้ำหนักทั่วไป
2. S-Beam Load Cell – ใช้วัดแรงดึงหรือแรงกดในงานอุตสาหกรรม
3. Shear Beam Load Cell – เหมาะกับเครื่องชั่งแบบหลายหัว
4. Compression Load Cell – สำหรับวัดแรงกดโดยตรง เช่น ในแท่นไฮดรอลิก

Load Cell แบบ Strain Gauge

Load Cell ขนาด 5 กิโลกรัมที่ใช้ในงานทั่วไปมักเป็นแบบ “Half-bridge” หรือ “Full-bridge” Wheatstone Circuit ซึ่งมี Strain Gauge ติดอยู่ 2-4 ตัว จุดเด่นคือสามารถวัดแรงได้แม่นยำและมีความไวสูง

การทำงานภายใน

1. ภายใน Load Cell จะมีวัสดุโลหะหรืออะลูมิเนียมที่ออกแบบให้มีความยืดหยุ่นเล็กน้อย
2. Strain Gauge จะติดกับวัสดุชิ้นนั้น เมื่อมีแรงกระทำ (จากน้ำหนัก)
3. วัสดุจะเกิดการยืดหรือหด -> ส่งผลให้ Strain Gauge เปลี่ยนค่าความต้านทาน
4. วงจร Wheatstone Bridge จะเปลี่ยนค่าแรงต้านเป็นแรงดันไฟฟ้าเล็กน้อย (ประมาณไมโครโวลต์ถึงมิลลิโวลต์)
5. สัญญาณนี้ยังอ่อนเกินกว่าที่ไมโครคอนโทรลเลอร์จะอ่านได้ จำเป็นต้องมีการขยายด้วย HX711

1.5.5 Ultrasonic Module HC-SR04



ภาพที่ 9 HC-SR04

Ultrasonic Sensor หรือเซนเซอร์อัลตราโซนิก เป็นอุปกรณ์อิเล็กทรอนิกส์ที่ทำหน้าที่วัดระยะทางระหว่างเซนเซอร์กับวัตถุที่อยู่เบื้องหน้า โดยใช้หลักการสะท้อนของคลื่นเสียงความถี่สูง อุปกรณ์ชนิดนี้มีข้อดีคือสามารถตรวจจับวัตถุได้โดยไม่ต้องสัมผัส จึงได้รับความนิยมอย่างมากในงานอัตโนมัติ เช่น หุ่นยนต์หลบสิ่งกีดขวาง ระบบช่วยจอดรถ ระบบวัดระดับน้ำ หรือแม้แต่ในงานด้านความปลอดภัยภายในอาคาร

คลื่นเสียงที่เซนเซอร์อัลตราโซนิกใช้งานอยู่ในช่วงที่มนุษย์ไม่สามารถได้ยิน โดยทั่วไปประมาณ 40 กิโลเฮิรตซ์ การวัดระยะทางที่เกิดขึ้นไม่ขึ้นกับสีหรือความสว่างของวัตถุ ต่างจากเซนเซอร์แบบอินฟราเรด จึงทำให้ Ultrasonic Sensor เหมาะกับงานหลากหลายประเภทมากยิ่งขึ้น

โครงสร้างของอุปกรณ์

โครงสร้างพื้นฐานของ Ultrasonic Sensor รุ่นที่นิยมใช้มากที่สุด ได้แก่ รุ่น HC-SR04 จะประกอบด้วยส่วนประกอบหลักดังนี้

1. ตัวส่งคลื่นเสียง (Ultrasonic Transmitter) ทำหน้าที่ปล่อยคลื่นเสียงความถี่สูงออกไปจากตัวเซนเซอร์เพื่อให้เดินทางไปยังวัตถุที่อยู่ด้านหน้า
2. ตัวรับคลื่นเสียง (Ultrasonic Receiver) มีหน้าที่รับคลื่นเสียงที่สะท้อนกลับมาจากวัตถุที่อยู่ในระยะตรวจจับ
3. วงจรควบคุมภายใน ใช้ในการจัดการการทำงานของตัวส่งและตัวรับ รวมถึงสร้างสัญญาณควบคุมและวัดเวลาของคลื่นที่สะท้อนกลับ
4. ขาเชื่อมต่อ (Pins) สำหรับเชื่อมต่อกับวงจรควบคุมหรือไม่โครคอนโทรลเลอร์ โดยประกอบด้วย 4 ขา ได้แก่

- VCC สำหรับจ่ายไฟ (โดยทั่วไปคือ 5 โวลต์)
- GND สำหรับกราวด์
- Trig สำหรับส่งสัญญาณให้เซนเซอร์เริ่มทำงาน
- Echo สำหรับรับสัญญาณกลับ เพื่อคำนวณระยะทาง

การทำงานของอุปกรณ์

การทำงานของ Ultrasonic Sensor อาศัยหลักการของการสะท้อนของคลื่นเสียง โดยมีลำดับขั้นตอนการทำงานดังต่อไปนี้

1. เริ่มต้นการทำงานโดยไมโครคอนโทรลเลอร์ ไมโครคอนโทรลเลอร์จะส่งสัญญาณไฟฟ้าไปยังขา Trig เป็นระยะเวลาประมาณ 10 ไมโครวินาที เพื่อสั่งให้เซนเซอร์เริ่มส่งคลื่นเสียงออกไป
2. เซนเซอร์ส่งคลื่นเสียงออกไปในอากาศ ตัวส่งคลื่นจะปล่อยคลื่นเสียงความถี่ 40 กิโลเฮิร์ตซ์ ออกไปด้านหน้าอย่างต่อเนื่องเป็นจำนวน 8 ชุด
3. คลื่นเสียงเดินทางไปยังวัตถุ แล้วสะท้อนกลับมา เมื่อคลื่นเสียงไปกระทบวัตถุในแนวตรงหน้าของเซนเซอร์ จะเกิดการสะท้อนกลับมายังตัวรับของเซนเซอร์
4. เซนเซอร์วัดเวลาการเดินทางของคลื่น เมื่อคลื่นเสียงสะท้อนกลับถึงตัวรับ เซนเซอร์จะส่งสัญญาณผ่านขา Echo ไปยังไมโครคอนโทรลเลอร์ โดยความยาวของสัญญาณที่ส่งจะเท่ากับเวลาที่คลื่นเสียงใช้ในการเดินทางไปและกลับจากวัตถุ
5. คำนวณระยะทางจากเวลาที่วัดได้ ระยะทางสามารถคำนวณได้จากสูตรทางฟิสิกส์ คือ

$$\text{ระยะทาง} = \frac{\text{เวลา} \times \text{ความเร็วเสียง}}{2}$$

โดยใช้ค่าความเร็วเสียงในอากาศประมาณ 343 เมตรต่อวินาที หรือ 0.0343 เซนติเมตรต่อไมโครวินาทีและหารด้วยสองเพื่อแยกระยะทางขาไปกับขากลับ
ผลลัพธ์ที่ได้คือค่าระยะทางจากเซนเซอร์ไปยังวัตถุซึ่งสามารถแสดงออกเป็นเซนติเมตร หรือเมตร ตามการตั้งค่าของโปรแกรมควบคุม

1.5.6 Micro Servo SG90 and MG90S



ภาพที่ 10 Micro Servo SG90 และ MG90S

Micro Servo เป็นอุปกรณ์ชนิดหนึ่งที่ใช้สำหรับการควบคุมมุมหรือตำแหน่งของวัตถุอย่างแม่นยำในงานต่าง ๆ เช่น หุ่นยนต์ โมเดลวิทยาศาสตร์ และอุปกรณ์อัตโนมัติ โดยมีขนาดเล็ก น้ำหนักเบา และสามารถควบคุมได้ง่ายผ่านสัญญาณ PWM (Pulse Width Modulation)

Micro Servo SG90 และ MG90S เป็นเซอร์โวมอเตอร์ขนาดเล็กที่ได้รับความนิยมสูง เนื่องจากมีราคาประหยัดและประสิทธิภาพที่ดี เหมาะสำหรับงานที่ต้องการความแม่นยำในระดับเล็ก ๆ เช่น การควบคุมแขนหุ่นยนต์ หรือการปรับมุมเซนเซอร์ต่าง ๆ ทั้งสองรุ่นนี้มีขนาดและรูปแบบคล้ายคลึงกัน แต่มีความแตกต่างในวัสดุและประสิทธิภาพบางอย่าง

โครงสร้างอุปกรณ์

ทั้ง SG90 และ MG90S ประกอบด้วยชิ้นส่วนหลักดังนี้

1. มอเตอร์ไฟฟ้าขนาดเล็ก ทำหน้าที่หมุนแกนของเซอร์โวเพื่อปรับตำแหน่งตามคำสั่งที่ได้รับ
2. เกียร์ SG90 ใช้เกียร์พลาสติก ในขณะที่ MG90S ใช้เกียร์โลหะที่ทนทานกว่า ส่งผลให้ MG90S มีความแข็งแรงและอายุการใช้งานที่ยาวนานกว่า
3. แผงวงจรควบคุม (Control Circuit Board) ทำหน้าที่รับสัญญาณ PWM และควบคุมมอเตอร์ให้หมุนไปยังตำแหน่งที่ต้องการอย่างแม่นยำ
4. เพลาหรือแกนหมุน (Output Shaft) เป็นจุดที่เชื่อมต่อกับชิ้นส่วนที่ต้องการควบคุม เช่น แขนหุ่นยนต์ หรือคันโยก
5. สายสัญญาณ โดยทั่วไปจะมีสามสาย ได้แก่ สายไฟเลี้ยง (VCC), สายกราวด์ (GND) และสายสัญญาณควบคุม (Signal)

การทำงานของอุปกรณ์

Micro Servo ทำงานโดยการรับสัญญาณ PWM ซึ่งเป็นสัญญาณความกว้างพัลส์ที่มีช่วงเวลาซ้ำซ้อนตามความต้องการของตำแหน่งที่ต้องการให้แกนหมุนอยู่

1. สัญญาณ PWM จะถูกส่งมาที่ขาสัญญาณของเซอร์โว
2. แผงวงจรภายในจะเปรียบเทียบความกว้างของพัลส์กับตำแหน่งปัจจุบันของแกนหมุน
3. จากนั้นจะสั่งให้มอเตอร์หมุนไปในทิศทางที่เหมาะสมจนกว่าแกนจะอยู่ในตำแหน่งที่สอดคล้องกับสัญญาณ PWM นั้น
4. เมื่อแกนหมุนไปถึงตำแหน่งที่ต้องการ มอเตอร์จะหยุดหมุน ทำให้สามารถควบคุมได้อย่างแม่นยำ

โดยทั่วไป Micro Servo SG90 และ MG90S สามารถหมุนได้ประมาณ 180 องศา และมีความแม่นยำในการควบคุมอยู่ในระดับดีเพียงพอสำหรับงานประดิษฐ์และงานเรียนรู้

1.5.7 4x4 Matrix Membrane Keypad



ภาพที่ 11 4x4 Matrix Membrane Keypad

Keypad หรือแผงปุ่มกด เป็นอุปกรณ์อินพุต (Input Device) ที่ใช้สำหรับรับข้อมูลจากผู้ใช้งานเข้าสู่ระบบไมโครคอนโทรลเลอร์ เช่น Arduino, ESP32 หรือบอร์ดควบคุมอื่น ๆ โดยผู้ใช้งานจะทำการกดปุ่มที่อยู่บนแผง ซึ่งตัวอุปกรณ์จะส่งค่าที่สอดคล้องกับปุ่มที่ถูกกดเข้าสู่ระบบ

นิยมใช้งานในระบบที่ต้องการรับค่าตัวเลข ตัวอักษร หรือรหัสผ่าน เช่น เครื่องกดรหัสเปิดประตู เครื่องกดเบอร์โทรศัพท์ เครื่องคำนวณ (Calculator) ตู้กดสินค้าอัตโนมัติ หรือแผงควบคุมอุปกรณ์ต่าง ๆ ในระบบสมองกลฝังตัว

โครงสร้างอุปกรณ์

Keypad ที่ใช้ในงานทั่วไปมีหลายขนาด เช่น 1x4, 3x4, และ 4x4 โดยหมายถึงจำนวนแถวและจำนวนคอลัมน์ของปุ่มบนแผง ตัวอย่างเช่น Keypad แบบ 4x4 จะมีทั้งหมด 16 ปุ่ม เรียงเป็น 4 แถว 4 คอลัมน์ โครงสร้างพื้นฐานของ Keypad ประกอบด้วย

1. แผงปุ่มกด (Key Mat) เป็นแผ่นพลาสติกหรือยางที่มีปุ่มกดวางเรียงกันตามจำนวนแถวและคอลัมน์ บางรุ่นจะมีตัวอักษรหรือตัวเลขพิมพ์กำกับไว้ เช่น 0-9, A-D, *, #
2. แผงวงจรภายใน (Matrix Circuit) ภายในแผงปุ่มจะมีแผงวงจรที่เชื่อมต่อแต่ละปุ่มเป็นรูปแบบเมทริกซ์ กล่าวคือ การเชื่อมต่อแบบแถว (Row) และคอลัมน์ (Column) โดยการกดปุ่มหนึ่งปุ่ม จะทำให้วงจรไฟฟ้าระหว่างแถวและคอลัมน์ที่สอดคล้องกันปิดวงจร
3. ขาเชื่อมต่อ (Pins) Keypad จะมีขาออกเท่ากับจำนวนแถวคูณจำนวนคอลัมน์ เช่น Keypad 4x4 จะมีขาออกทั้งหมด 8 ขา โดยเชื่อมต่อกับไมโครคอนโทรลเลอร์เพื่ออ่านค่าสัญญาณของปุ่มที่ถูกกด

การทำงานของอุปกรณ์

Keypad ทำงานโดยอาศัยหลักการตรวจจับการปิดวงจรระหว่างแถวและคอลัมน์ เมื่อต้องการตรวจสอบว่ามีการกดปุ่มใดหรือไม่ ไมโครคอนโทรลเลอร์จะทำการส่งสัญญาณไฟฟ้า (เช่น สถานะ HIGH) ไปยังแต่ละแถวทีละแถว และตรวจสอบว่ามีสัญญาณกลับมาจากคอลัมน์หรือไม่

ขั้นตอนการทำงานสามารถอธิบายได้ดังนี้

1. ไมโครคอนโทรลเลอร์กำหนดแถวหนึ่งแถวให้เป็นสถานะ HIGH
2. ตรวจสอบแต่ละคอลัมน์ว่ามีสัญญาณ HIGH หรือไม่
3. หากพบสัญญาณจากคอลัมน์ใด หมายความว่าปุ่มในตำแหน่งของแถวนั้นและคอลัมน์นั้นถูกกด
4. ทำการวนซ้ำกระบวนการนี้กับทุกแถว เพื่อให้สามารถตรวจจับการกดปุ่มได้ครบทุกตำแหน่ง

การทำงานแบบเมทริกซ์นี้ช่วยลดจำนวนขา I/O ที่ใช้ จากเดิมที่ปุ่ม 16 ปุ่มอาจต้องใช้ขาถึง 16 ขา หากใช้การเชื่อมต่อแบบตรง แต่การจัดแบบแถว-คอลัมน์ทำให้ใช้เพียง 8 ขาเท่านั้นใน Keypad 4x4

1.5.8 PCF8574 8Bit Parallel Expander I2C-Bus



ภาพที่ 12 PCF8574

PCF8574 เป็นชิป I/O Expander หรืออุปกรณ์ขยายขาอินพุต/เอาต์พุตสำหรับไมโครคอนโทรลเลอร์ ถูกออกแบบมาเพื่อเพิ่มจำนวนขา I/O ของบอร์ด เช่น Arduino, ESP32 หรือ Raspberry Pi โดยเชื่อมต่อผ่าน I²C bus ซึ่งเป็นการสื่อสารแบบสองสาย (SDA และ SCL) ชิปตัวนี้ช่วยให้ผู้พัฒนาสามารถควบคุมอุปกรณ์จำนวนมาก เช่น LED, รีเลย์, ปุ่มกด หรือเซนเซอร์ ได้แม้บอร์ดต้นทางมีจำนวนขา I/O จำกัด

PCF8574 มีคุณสมบัติเด่นคือรองรับแรงดันไฟฟ้า 2.5V–6V และมี 8 ขา I/O ที่สามารถตั้งค่าเป็นอินพุตหรือเอาต์พุตได้อย่างอิสระ แต่ละขาสามารถจ่ายกระแสได้ประมาณ 25 mA ทำให้สามารถขับ LED ขนาดเล็กหรือสัญญาณดิจิทัลอื่น ๆ ได้โดยตรง

โครงสร้างของอุปกรณ์

โครงสร้างของ PCF8574 ประกอบด้วยส่วนประกอบหลักดังนี้

1. ขา I/O 8 ขา (P0–P7): สามารถตั้งเป็นอินพุตหรือเอาต์พุตได้
2. ขา VCC และ GND: สำหรับจ่ายไฟให้ชิป
3. ขา SDA และ SCL: ใช้เชื่อมต่อกับไมโครคอนโทรลเลอร์ผ่าน I²C bus
4. ขา A0–A2: ใช้ตั้ง I²C address ทำให้สามารถใช้ชิปหลายตัวในบัสเดียวกันได้
5. วงจร Pull-up ภายใน: ช่วยให้ขาอินพุตสามารถอ่านค่าดิจิทัลได้เสถียร
6. โครงสร้างภายใน: ประกอบด้วยตัว latch, buffer, และ driver เพื่อจัดการการอ่าน/เขียนข้อมูลระหว่างบัส I²C และขา I/O

PCF8575 มักมาในรูปแบบโมดูลที่มี핀แถวสำหรับเสียบกับบอร์ดทดลอง (Breadboard) และมีตัวต้านทาน Pull-up สำหรับ SDA/SCL ติดตั้งไว้เรียบร้อยแล้วเพื่อความสะดวกในการใช้งาน

การทำงานของอุปกรณ์

หลักการทำงานของ PCF8574 คือการส่งและรับข้อมูลระหว่างไมโครคอนโทรลเลอร์กับไอซีผ่านโปรโตคอล I²C เพื่อควบคุมสถานะของขา I/O ที่เชื่อมต่ออยู่

1. เชื่อมต่อกับไมโครคอนโทรลเลอร์: ต่อขา VCC, GND, SDA และ SCL เข้ากับบอร์ดต้นทาง เช่น Arduino หรือ ESP32
2. ตั้งค่า I²C Address: เลือก address ของชิปโดยตั้งค่าขา A0–A2 เพื่อให้สามารถใช้ชิปหลายตัวบนบัสเดียวกันได้
3. กำหนดโหมดขา I/O: เขียนโปรแกรมให้แต่ละขาของ PCF8574 เป็นอินพุตหรือเอาต์พุต
4. ส่งคำสั่งผ่าน I²C: ไมโครคอนโทรลเลอร์ส่งคำสั่งอ่านหรือเขียนข้อมูลไปยัง PCF8574 ผ่านบัส I²C
5. การอ่านค่าขาอินพุต: ถ้าขาใดตั้งเป็นอินพุต ชิปจะส่งสถานะ HIGH/LOW กลับไปยังไมโครคอนโทรลเลอร์
6. การเขียนค่าขาเอาต์พุต: ถ้าขาใดตั้งเป็นเอาต์พุต ไมโครคอนโทรลเลอร์สามารถส่งสัญญาณ HIGH/LOW เพื่อควบคุม LED, รีเลย์ หรืออุปกรณ์อื่น ๆ
7. ทำงานต่อเนื่อง: การสื่อสาร I²C จะวนซ้ำตลอดเวลา ทำให้ไมโครคอนโทรลเลอร์สามารถตรวจสอบสถานะและควบคุมอุปกรณ์ได้แบบ real-time

การควบคุม I/O ทั้งหมดนี้สามารถทำได้ผ่านไลบรารีที่รองรับใน Arduino IDE เช่น Wire.h หรือไลบรารีเฉพาะของ PCF8574 เพื่อให้่ายต่อการเขียนโปรแกรมและจัดการข้อมูล

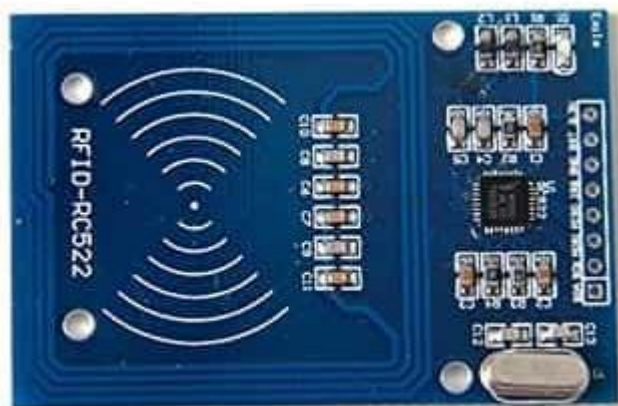
การทำงานร่วมกับระบบอื่น

PCF8574 สามารถทำงานร่วมกับไมโครคอนโทรลเลอร์ในระบบฝังตัวได้อย่างหลากหลาย เช่น Arduino UNO, MEGA, ESP32 หรือแม้แต่ Raspberry Pi ซึ่งจะช่วยขยายความสามารถในการควบคุมอุปกรณ์ภายนอกที่ใช้ขาดิจิทัลจำนวนมาก

การประยุกต์ใช้งานร่วมกับระบบอื่นที่นำมาใช้ในโครงการ

- แผงปุ่มกด (Keypad)
เมื่อต้องใช้ปุ่มจำนวนมาก เช่น 4x4 หรือมากกว่านั้น PCF8574 สามารถใช้เป็นตัวรับอินพุตของปุ่มโดยไม่ต้องใช้ขา I/O จำนวนมากของไมโครคอนโทรลเลอร์

1.5.9 RC522 RFID



ภาพที่ 13 RC522 RFID

RC522 เป็นโมดูล RFID (Radio Frequency Identification) ที่ทำหน้าที่อ่านและเขียนข้อมูลจากอุปกรณ์แท็กหรือบัตร RFID ซึ่งใช้เทคโนโลยีการสื่อสารไร้สายผ่านคลื่นวิทยุ โดยรองรับบัตรในระบบ 13.56 MHz ซึ่งเป็นความถี่มาตรฐานสำหรับ RFID แบบใกล้ระยะ (NFC – Near Field Communication)

โมดูล RC522 เป็นที่นิยมในวงการอิเล็กทรอนิกส์และไมโครคอนโทรลเลอร์ เนื่องจากมีราคาย่อมเยาใช้งานง่าย และมีไลบรารีสำหรับใช้งานร่วมกับบอร์ดยอดนิยม เช่น Arduino, ESP32 และ Raspberry Pi เหมาะสำหรับโครงการที่ต้องการระบบตรวจสอบตัวตน เช่น ระบบควบคุมประตู ระบบเช็คชื่อ หรือระบบสมาชิก

โครงสร้างอุปกรณ์

โมดูล RC522 ประกอบด้วยส่วนประกอบหลักดังนี้

1. ชิป MFRC522 เป็นชิปหลักของโมดูล ทำหน้าที่ควบคุมการสื่อสารระหว่างโมดูลกับบัตร RFID รวมถึงจัดการกระบวนการเข้ารหัสและถอดรหัสข้อมูลที่รับส่ง
2. สายอากาศ (Antenna) อยู่ในรูปแบบขดลวดทองแดงรอบบอร์ด ทำหน้าที่สร้างสนามแม่เหล็กไฟฟ้าสำหรับติดต่อกับบัตร RFID ที่อยู่ในระยะใกล้ โดยทั่วไปประมาณ 2–5 เซนติเมตร
3. อินเทอร์เฟซ SPI (Serial Peripheral Interface) ใช้เชื่อมต่อระหว่างโมดูล RC522 กับไมโครคอนโทรลเลอร์ มีขาต่อหลัก ๆ ได้แก่
 - SDA (SS): Slave Select
 - SCK: Clock
 - MOSI: Master Out Slave In
 - MISO: Master In Slave Out

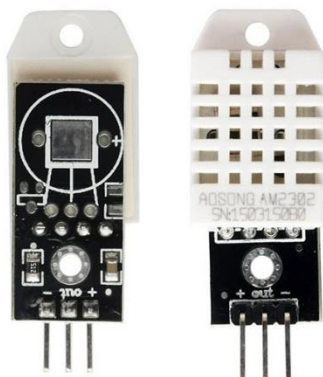
- RST: Reset
 - GND: กราวด์
 - 3.3V: จ่ายไฟเลี้ยง (ควรใช้ 3.3V เท่านั้น)
4. หน่วยความจำบนบัตร RFID โมดูลสามารถอ่านหรือเขียนข้อมูลไปยังบัตร RFID ที่มีชิปภายใน เช่น MIFARE 1K ซึ่งแบ่งเป็นบล็อกและเซกเตอร์สำหรับเก็บข้อมูลต่าง ๆ เช่น หมายเลขบัตร ชื่อผู้ใช้งาน หรือข้อมูลการเข้าออก

การทำงานของอุปกรณ์

กระบวนการทำงานของ RC522 สามารถแบ่งออกเป็นขั้นตอนหลักดังนี้

1. สร้างสนามแม่เหล็กความถี่ 13.56 MHz
โมดูลจะสร้างสนามแม่เหล็กไฟฟ้าเพื่อรอรับบัตร RFID ที่อยู่ในระยะใกล้ เมื่อบัตรเข้าใกล้สนามนี้ ชิปในบัตรจะได้รับพลังงานและทำงาน (ในกรณีของบัตรแบบ Passive)
2. การแลกเปลี่ยนข้อมูลระหว่างโมดูลกับบัตร
เมื่อบัตรอยู่ในระยะที่เหมาะสม ชิป MFRC522 จะเริ่มสื่อสารกับบัตรผ่านโปรโตคอล ISO/IEC 14443A โดยส่งคำสั่งไปยังบัตรและรับข้อมูลตอบกลับ เช่น UID (Unique Identifier) ซึ่งเป็นหมายเลขประจำตัวของบัตรแต่ละใบ
3. การตรวจสอบสิทธิ์ (Authentication)
หากต้องการอ่านหรือเขียนข้อมูลจากบัตร ระบบจะต้องทำการยืนยันสิทธิ์ก่อน ด้วยการใส่รหัสผ่าน (Key A หรือ Key B) เพื่อป้องกันการเข้าถึงข้อมูลโดยไม่ได้รับอนุญาต
4. การอ่านหรือเขียนข้อมูล
หลังจากยืนยันสิทธิ์สำเร็จ ระบบสามารถอ่านข้อมูลจากแต่ละบล็อกของบัตร หรือเขียนข้อมูลใหม่ลงในบัตรได้ โดยแต่ละบัตร MIFARE 1K จะมีความจุประมาณ 1 กิโลไบต์ แบ่งออกเป็นหลายเซกเตอร์
5. สิ้นสุดการทำงานเมื่อบัตรถูกนำออกจากระยะอ่าน
เมื่อไม่มีบัตรอยู่ในระยะใกล้ สนามแม่เหล็กจะไม่มีผลต่อบัตรใด ๆ และระบบจะกลับเข้าสู่สถานะรอการตรวจจับบัตรใหม่อีกครั้ง

1.5.10 DHT22



ภาพที่ 14 DHT22

DHT22 หรือที่รู้จักในอีกชื่อหนึ่งว่า AM2302 เป็นเซนเซอร์วัดอุณหภูมิและความชื้นแบบดิจิทัล ที่ได้รับความนิยมสูงในวงการอิเล็กทรอนิกส์และระบบฝังตัว เนื่องจากใช้งานง่าย มีความแม่นยำสูง และมีราคาที่เหมาะสม

DHT22 ถูกออกแบบมาเพื่อใช้ในการตรวจวัดสภาพแวดล้อม เช่น อุณหภูมิในห้อง ความชื้นในดิน หรือ การตรวจสอบคุณภาพอากาศภายในระบบอัตโนมัติ เซนเซอร์รุ่นนี้เหมาะสำหรับงานในภาคเกษตรกรรมอัจฉริยะ (Smart Farming), ระบบควบคุมอุณหภูมิภายในบ้านอัจฉริยะ (Smart Home), ระบบตู้ควบคุมอุณหภูมิ, และระบบ IoT อื่น ๆ ที่ต้องการข้อมูลสภาพแวดล้อมแบบเรียลไทม์

โครงสร้างของอุปกรณ์

DHT22 มีลักษณะเป็นโมดูลขนาดเล็ก บรรจุอยู่ในแพ็คเกจพลาสติก มีขาเชื่อมต่อโดยทั่วไป 3 หรือ 4 ขา ซึ่งประกอบด้วยส่วนสำคัญดังนี้

1. เซนเซอร์วัดอุณหภูมิ (Temperature Sensor)

ใช้เทอร์มิสเตอร์แบบดิจิทัล (Digital Thermistor) ซึ่งมีความสามารถในการตรวจจับอุณหภูมิในช่วงประมาณ -40°C ถึง $+80^{\circ}\text{C}$ ด้วยความแม่นยำประมาณ $\pm 0.5^{\circ}\text{C}$

2. เซนเซอร์วัดความชื้น (Humidity Sensor)

ใช้เซนเซอร์แบบความจุ (Capacitive Humidity Sensor) ที่สามารถตรวจวัดค่าความชื้นสัมพัทธ์ในช่วง 0% ถึง 100% RH (Relative Humidity) ด้วยความแม่นยำประมาณ $\pm 2-5\%$ RH

3. วงจรแปลงสัญญาณแบบดิจิทัล (Digital Signal Processor)

ทำหน้าที่ประมวลผลข้อมูลจากเซนเซอร์และส่งข้อมูลออกมาในรูปแบบดิจิทัล ทำให้ง่ายต่อการอ่านค่าด้วยไมโครคอนโทรลเลอร์

4. ขาเชื่อมต่อ (Pins)

โดยทั่วไปมี 3 ขาหลัก คือ

- VCC (จ่ายไฟเลี้ยง 3.3V – 6V)
- GND (กราวด์)
- DATA (ส่งข้อมูลแบบดิจิทัล)

ในบางรุ่นจะมีขาที่ 4 ซึ่งไม่ใช้งานหรือใช้สำหรับการเชื่อมต่อเพิ่มเติม

การทำงานของอุปกรณ์

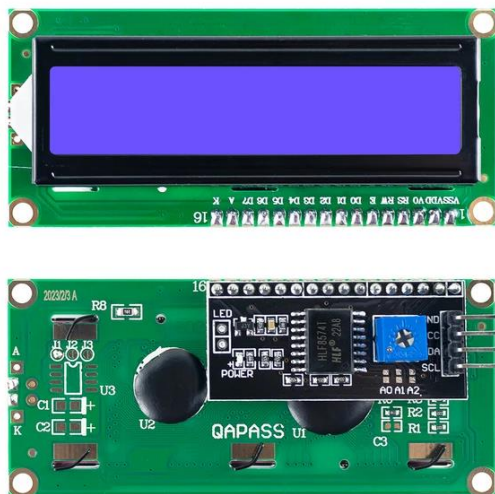
การทำงานของ DHT22 อาศัยกระบวนการตรวจวัดค่าทางฟิสิกส์ (อุณหภูมิและความชื้น) แล้วแปลงเป็นสัญญาณดิจิทัลโดยวงจรภายใน จากนั้นจึงส่งข้อมูลออกจากขา DATA โดยใช้โปรโตคอลการสื่อสารแบบ one-wire ซึ่งเป็นรูปแบบที่ใช้สายสัญญาณเพียงเส้นเดียวสำหรับส่งข้อมูล

ขั้นตอนการทำงานสามารถสรุปได้ดังนี้

1. เริ่มต้นการสื่อสาร ไมโครคอนโทรลเลอร์จะส่งสัญญาณเริ่มต้น (Start Signal) ไปยังเซนเซอร์ โดยการดึงขา DATA ลง LOW เป็นระยะเวลาหนึ่ง (ประมาณ 1 มิลลิวินาที)
2. การตอบกลับจากเซนเซอร์ เซนเซอร์จะตอบกลับด้วยสัญญาณ LOW ตามด้วย HIGH เพื่อยืนยันการเชื่อมต่อ จากนั้นจึงเริ่มส่งข้อมูล
3. การส่งข้อมูล เซนเซอร์จะส่งข้อมูลทั้งหมด 40 บิต แบ่งเป็น:
 - 16 บิตแรก: ค่าความชื้นสัมพัทธ์
 - 16 บิตถัดมา: ค่าอุณหภูมิ
 - 8 บิตสุดท้าย: ค่า checksum สำหรับตรวจสอบความถูกต้องของข้อมูล
4. การประมวลผลข้อมูล ไมโครคอนโทรลเลอร์จะอ่านข้อมูลที่ได้รับ แล้วแปลงเป็นค่าที่สามารถนำไปแสดงผลหรือใช้ประมวลผลต่อไป เช่น แสดงผลบนหน้าจอ หรือนำไปใช้ในระบบควบคุมอื่น ๆ

เนื่องจากเซนเซอร์มีความไวต่อการสื่อสาร จึงควรเว้นช่วงเวลาประมาณ 2 วินาทีระหว่างการอ่านค่าทุกครั้ง เพื่อให้ได้ข้อมูลที่ถูกต้องและไม่รบกวนการทำงานของเซนเซอร์

1.5.11 1602 LCD (Blue Screen) 16x2 LCD with I2C Adapter



ภาพที่ 15 LCD (Blue Screen) 16x2 LCD with I2C Adapter

LCD หรือ Liquid Crystal Display คืออุปกรณ์แสดงผลที่ใช้เทคโนโลยีของผลึกเหลว (Liquid Crystal) ในการควบคุมการผ่านของแสง เพื่อสร้างภาพหรือตัวอักษรบนหน้าจอ อุปกรณ์ประเภทนี้มีจุดเด่นที่ใช้พลังงานต่ำ น้ำหนักเบา และสามารถแสดงข้อมูลได้อย่างชัดเจนแม้ในสภาวะแสงน้อย

ในวงการไมโครคอนโทรลเลอร์ LCD แบบอักษร เช่น LCD 16x2 และ LCD 20x4 เป็นที่นิยมใช้งานอย่างแพร่หลาย โดยสามารถแสดงตัวอักษรและตัวเลขได้อย่างชัดเจน เหมาะสำหรับโครงการต่าง ๆ เช่น ระบบควบคุมอัตโนมัติ, เครื่องมือวัด, ตู้กดสินค้า, หรือเครื่องคิดเลขดิจิทัล

โครงสร้างอุปกรณ์

LCD มีองค์ประกอบสำคัญหลายส่วน ซึ่งทำงานร่วมกันเพื่อแสดงผลข้อมูล ได้แก่

1. แผงผลึกเหลว (Liquid Crystal Panel) เป็นชั้นกลางที่ประกอบด้วยผลึกเหลวที่สามารถเปลี่ยนการจัดเรียงตัวของโมเลกุลเมื่อได้รับแรงดันไฟฟ้า เพื่อควบคุมการผ่านของแสง
2. ขั้วไฟฟ้า (Electrodes) เป็นตัวนำไฟฟ้าที่ควบคุมการทำงานของผลึกเหลวในแต่ละจุด (พิกเซลหรือเซกเมนต์) เมื่อจ่ายไฟแรงดันต่ำ ผลึกเหลวจะปรับตัวและแสดงผลตามตำแหน่ง
3. แผ่นโพลาไรซ์ (Polarizer) ทำหน้าที่กรองแสงให้เข้าสู่ผลึกเหลวในทิศทางเดียว และกรองแสงออกเพื่อให้แสดงเป็นตัวอักษรหรือตัวเลขบนหน้าจอ
4. แหล่งแสงพื้นหลัง (Backlight) เป็นส่วนที่ให้แสงสว่างจากด้านหลังแผงผลึกเหลว เพื่อให้ภาพที่แสดงมีความชัดเจนมากขึ้น โดยเฉพาะในที่แสงน้อย

5. ไอซีควบคุม (Controller IC) สำหรับ LCD 16x2 หรือ 20x4 จะใช้ชิปควบคุมมาตรฐาน เช่น HD44780 ซึ่งช่วยจัดการข้อมูลที่ส่งเข้ามาจากไมโครคอนโทรลเลอร์ และแปลงเป็นสัญญาณควบคุม พิกเซลแต่ละตำแหน่ง
6. ขาเชื่อมต่อ (Interface Pins) LCD มาตรฐานมักมี 16 ขา สำหรับจ่ายไฟ ควบคุม และส่งข้อมูล โดยสามารถเลือกใช้โหมด 8 บิต หรือ 4 บิต เพื่อลดจำนวนขาที่ใช้

การทำงานของอุปกรณ์

การทำงานของ LCD แบบตัวอักษร เช่น 16x2 (แสดงผลได้ 2 แถว แถวละ 16 ตัวอักษร) อาศัยการควบคุมผ่านชุดคำสั่งและการส่งข้อมูลจากไมโครคอนโทรลเลอร์ไปยังไอซีควบคุมของ LCD เพื่อแสดงผล กระบวนการทำงานแบ่งได้ดังนี้

1. เริ่มต้นระบบ (Initialization) เมื่อต่อไฟเลี้ยง ไมโครคอนโทรลเลอร์จะส่งชุดคำสั่งเริ่มต้นให้ LCD เพื่อกำหนดรูปแบบการแสดงผล เช่น จำนวนแถว/คอลัมน์, การเลื่อนข้อความ, และการเปิดไฟพื้นหลัง
2. ส่งคำสั่งหรือข้อมูล ข้อมูลที่ส่งไปยัง LCD แบ่งเป็น 2 ประเภท คือ
 - คำสั่ง (Command) เช่น เคลื่อนตำแหน่งเคอร์เซอร์ ล้างหน้าจอ
 - ข้อมูล (Data) คือ ตัวอักษรที่ต้องการแสดงผล

การส่งสัญญาณควบคุมประกอบด้วยการกำหนดสถานะของขา RS (Register Select), RW (Read/Write), E (Enable) และข้อมูลผ่านบิต D0–D7 (หรือ D4–D7 สำหรับโหมด 4 บิต)

3. การอัปเดตหน้าจอ เมื่อได้รับข้อมูลแล้ว ไอซีควบคุมจะทำการอัปเดตหน้าจอโดยเปลี่ยนการจัดเรียงของพิกเซลในตำแหน่ง ให้แสงผ่านหรือไม่ผ่าน เพื่อแสดงเป็นตัวอักษรตามที่ต้องการ
4. การวนรอบการทำงาน ไมโครคอนโทรลเลอร์สามารถส่งข้อมูลใหม่ไปยังหน้าจอได้ทุกเมื่อเพื่อเปลี่ยนข้อความหรือปรับปรุงสถานะที่แสดงผล

1.5.12 Tact Switch Push Button



ภาพที่ 16 Tact Switch Push Button

Tact Switch หรือที่เรียกทั่วไปว่า Push Button Switch คืออุปกรณ์สวิตช์แบบสัมผัสที่ทำหน้าที่เป็นตัวเปิด-ปิดวงจรไฟฟ้าแบบชั่วคราว โดยผู้ใช้งานจะต้องกดเพื่อให้เกิดการเชื่อมต่อวงจร และเมื่อปล่อยนิ้ว วงจรจะกลับมาอยู่ในสถานะเปิดอีกครั้ง ปุ่มกดชนิดนี้มักใช้ในงานควบคุมอิเล็กทรอนิกส์ขนาดเล็ก เช่น รีโมตคอนโทรล, เครื่องคิดเลข, อุปกรณ์ควบคุมด้วยไมโครคอนโทรลเลอร์, และแผงวงจรควบคุมต่าง ๆ

Tact Switch ได้รับความนิยมอย่างแพร่หลาย เนื่องจากมีขนาดเล็ก ราคาถูก ใช้งานง่าย และสามารถติดตั้งลงบนแผงวงจรพิมพ์ (PCB) ได้โดยสะดวก เหมาะสำหรับโครงการระบบฝังตัวที่ต้องการปุ่มกดจำนวนมากในพื้นที่จำกัด

โครงสร้างของอุปกรณ์

Tact Switch มีโครงสร้างที่เรียบง่ายแต่สามารถใช้งานได้อย่างมีประสิทธิภาพ โดยส่วนประกอบหลักประกอบด้วย

1. ตัวเรือน (Housing) ทำจากพลาสติกหรือวัสดุฉนวนไฟฟ้า ทำหน้าที่เป็นโครงสร้างของปุ่มกด และป้องกันส่วนประกอบภายในไม่ให้เสียหาย
2. หน้าปุ่มกด (Actuator) เป็นส่วนบนสุดของสวิตช์ที่ผู้ใช้งานกดลงไป ทำจากพลาสติกแข็งหรือยาง ขนาดเล็กประมาณ 6x6 มิลลิเมตรในรุ่นมาตรฐาน
3. แผ่นโลหะยืดหยุ่น (Metal Dome) เป็นแผ่นโลหะบางรูปโดมที่มีแรงสปริงในตัว เมื่อถูกกดจะยุบลง ทำให้ขั้วไฟฟ้าภายในสัมผัสกัน เมื่อปล่อยนิ้ว แผ่นโลหะจะเต่งกลับสู่สภาพเดิม
4. ขาเชื่อมต่อ (Terminals / Pins) โดยทั่วไปจะมี 4 ขา ซึ่งเชื่อมต่อกับวงจรไฟฟ้า ขาเหล่านี้จะถูกเชื่อมต่อกันสองคู่แบบแนวตรง และวงจรจะถูกปิดเฉพาะเมื่อกดปุ่มให้แผ่นโลหะสัมผัสกับขาทั้งสองคู่
5. หน้าสัมผัสภายใน (Internal Contacts) ทำจากวัสดุนำไฟฟ้า เช่น ทองแดงหรือโลหะอื่น ๆ ทำหน้าที่เป็นจุดเชื่อมต่อของวงจรเมื่อเกิดการกดปุ่ม

การทำงานของอุปกรณ์

หลักการทำงานของ Tact Switch เป็นแบบ Normally Open (NO) หรือวงจรเปิดโดยปกติ จนกว่าจะมีการกดปุ่ม ซึ่งทำให้เกิดการเชื่อมต่อวงจรอย่างชั่วคราว เมื่อละมือจากปุ่ม สวิตช์จะคืนกลับสู่สถานะเปิดโดยอัตโนมัติ

ขั้นตอนการทำงานมีดังนี้

1. สถานะปกติ (ไม่กด) ขาสวิตช์ทั้งสองชุดจะไม่เชื่อมต่อกัน วงจรไฟฟ้าจึงยังไม่ทำงาน กระแสไฟไม่สามารถไหลผ่าน
2. สถานะกด (ทำงาน) เมื่อผู้ใช้งานกดปุ่ม แผ่นโลหะภายในจะยุบตัวลงและสัมผัสกับขาสวิตช์ทั้งสอง ทำให้วงจรไฟฟ้าเชื่อมต่อกัน และกระแสไฟสามารถไหลผ่านไปยังส่วนอื่นของระบบ
3. สถานะปล่อย (หยุดทำงาน) เมื่อปล่อยปุ่ม แผ่นโลหะจะคืนรูป และหน้าสัมผัสจะแยกออกจากกัน ทำให้วงจรไฟฟ้ากลับเข้าสู่สถานะเปิดอีกครั้ง

Tact Switch ทำงานเป็นแบบ Momentary Switch คือมีการเชื่อมต่อวงจรเฉพาะขณะกด ไม่ใช่การเปิด/ปิดแบบถาวร จึงเหมาะกับงานที่ต้องการคำสั่งเพียงชั่วขณะ เช่น การเริ่มต้นโปรแกรม การรีเซ็ต หรือการนับจำนวน

1.5.13 LED (Light-Emitting Diode)



ภาพที่ 17 LED

LED หรือ Light-Emitting Diode คืออุปกรณ์อิเล็กทรอนิกส์ประเภทไดโอดชนิดหนึ่ง ที่สามารถเปล่งแสงได้เมื่อมีกระแสไฟฟ้าไหลผ่าน โดยทั่วไปจะเปล่งแสงในช่วงความยาวคลื่นต่าง ๆ เช่น สีแดง เขียว น้ำเงิน ขาว หรือแม้กระทั่งอินฟราเรด

LED ได้รับความนิยมอย่างแพร่หลายในงานอิเล็กทรอนิกส์ทุกระดับ ตั้งแต่การแสดงผลในอุปกรณ์พื้นฐาน เช่น สัญญาณไฟแสดงสถานะ จนถึงการใช้งานเชิงอุตสาหกรรม เช่น ไฟถนน ไฟรถยนต์ หรือหน้าจอแสดงผล LED Display นอกจากนี้ยังเป็นอุปกรณ์หลักที่ใช้บ่อยในโครงงานไมโครคอนโทรลเลอร์ เช่น Arduino, ESP32 หรือ Raspberry Pi เนื่องจากสามารถแสดงผลได้ชัดเจน ใช้พลังงานต่ำ และควบคุมได้ง่าย

โครงสร้างของอุปกรณ์

โครงสร้างของ LED ประกอบด้วยส่วนประกอบหลักดังต่อไปนี้

1. สารกึ่งตัวนำ (Semiconductor Material) เป็นวัสดุหลักที่ใช้ผลิตขึ้น P-type และ N-type ซึ่งรวมตัวกันเป็นรอยต่อไดโอด (PN Junction) โดยเลือกใช้วัสดุที่สามารถเปล่งแสงได้เมื่ออิเล็กตรอนรวมตัวกับโฮล
2. รอยต่อ PN (PN Junction) เป็นบริเวณที่กระแสไฟฟ้าไหลผ่านเมื่อไดโอดอยู่ในสถานะไบอัสตรง (Forward Bias) และเป็นจุดที่เกิดการเปล่งแสง (Electroluminescence) เมื่อมีการปล่อยพลังงานของอิเล็กตรอน
3. เลนส์หรือเปลือกหุ้ม (Encapsulation) ทำจากพลาสติกโปร่งแสงรูปโดมหรือแบน ทำหน้าที่เป็นทั้งเลนส์รวมแสง และปกป้องสารกึ่งตัวนำภายในจากความเสียหาย
4. ขั้วไฟฟ้า (Leads / Pins) โดยทั่วไปจะมี 2 ขา ได้แก่
 - ขา Anode (ขั้วบวก): ขาที่ยาวกว่า
 - ขา Cathode (ขั้วลบ): ขาที่สั้นกว่า หรือในบางรุ่นมีการเจียรฐานให้รู้ว่าเป็นขาใด

5. ฐานรอง (Reflective Cavity) ทำหน้าที่สะท้อนแสงจากชิป LED ออกไปด้านหน้า เพื่อเพิ่มความสว่าง
การทำงานของอุปกรณ์

การทำงานของ LED อาศัยหลักการทางฟิสิกส์ของสารกึ่งตัวนำ เมื่อมีกระแสไฟฟ้าไหลผ่านจากขา Anode ไปยังขา Cathode (สถานะไบอัสตรง) อิเล็กตรอนจากฝั่ง N-type จะเคลื่อนที่ไปจับกับโฮลที่ฝั่ง P-type ที่รอยต่อ PN การรวมตัวนี้จะปล่อยพลังงานออกมาในรูปของแสง

แสงที่เปล่งออกมาจะมีความยาวคลื่นและสีแตกต่างกันขึ้นอยู่กับชนิดของวัสดุที่ใช้ในการผลิตชั้นกึ่งตัวนำ เช่น:

- GaAs (Gallium Arsenide) สำหรับแสงอินฟราเรด
- GaP (Gallium Phosphide) สำหรับแสงเขียวหรือแดง
- InGaN (Indium Gallium Nitride) สำหรับแสงฟ้าและขาว

คุณสมบัติพื้นฐานของ LED คือ ต้องใช้แรงดันต่ำ (โดยเฉลี่ย 1.8 – 3.3 โวลต์ ขึ้นอยู่กับสี) และ ต้องมีตัวต้านทานจำกัดกระแส (Current-Limiting Resistor) เพื่อป้องกันไม่ให้เกิดกระแสไหลมากเกินไปจนทำให้ LED เสียหาย

1.5.14 Webcam NWC-560



ภาพที่ 18 NWC-560

Webcam NWC-560 เป็นกล้องเว็บแคมที่ผลิตโดยแบรนด์ Nubwo (ไทย) ออกแบบมาเพื่อการใช้งานในคอมพิวเตอร์สำหรับการประชุมออนไลน์,ไลฟ์สตรีม, การเรียนออนไลน์ หรือการบันทึกวิดีโอ กล้องตัวนี้โดดเด่นที่ความเรียบง่าย ใช้งานได้สะดวก และมีฟีเจอร์พื้นฐานที่เพียงพอสำหรับงานทั่วไป เช่น ความละเอียดระดับ HD, ไมโครโฟนในตัว และโครงสร้างที่สามารถติดตั้งกับจอคอมพิวเตอร์หรือขาตั้งได้ จากสเปกทางการ กล้อง NWC-560 ให้ “ความละเอียดแบบไดนามิก” ที่ 1280×720 พิกเซล (HD 720p) และอัตราเฟรมเรต (Frame Rate) ที่ 30 เฟรมต่อวินาที (30 fps) กล้องมาพร้อมไมโครโฟนในตัวที่รองรับการรับเสียงภายในระยะประมาณ 10 เมตร ตามคำโฆษณาของผู้ผลิต โดยรวม NWC-560 เป็นเว็บแคมราคาย่อมเยา เหมาะสำหรับผู้ใช้ทั่วไปที่ต้องการกล้องเสริมสำหรับคอมพิวเตอร์ โดยไม่ต้องการฟีเจอร์ระดับสูง เช่น 1080p หรือ 4K, autofocus อัตโนมัติ หรือระบบลดเสียงรบกวนขั้นสูง

โครงสร้างของอุปกรณ์

โครงสร้างภายในและภายนอกของเว็บแคม NWC-560 ประกอบด้วย:

1. เลนส์และเซ็นเซอร์ภาพ: ใช้เลนส์ออปติคอลคุณภาพพื้นฐาน ร่วมกับเซ็นเซอร์ภาพที่รองรับความละเอียด HD ช่วยให้ภาพที่ได้มีความคมชัดเหมาะกับการใช้งานทั่วไป
2. ไมโครโฟนในตัว: ติดตั้งไมโครโฟนแบบลดเสียงรบกวนขั้นพื้นฐาน เพื่อบันทึกเสียงพร้อมกับภาพเหมาะสำหรับการสนทนาและการประชุมออนไลน์
3. ตัวบอดี้และฐานยึด: มีโครงสร้างที่สามารถปรับมุมกล้องได้ ติดตั้งได้ทั้งกับขอบหน้าจคอมพิวเตอร์หรือใช้วางบนโต๊ะ รองรับการใช้งานหลายลักษณะ

4. สายเชื่อมต่อ USB: ใช้สำหรับเชื่อมต่อกับคอมพิวเตอร์เพื่อทั้งการส่งสัญญาณภาพ เสียง และจ่ายพลังงานให้กับกล้อง

การทำงานของอุปกรณ์

กล้อง NWC-560 ทำงานตามลำดับขั้นตอนดังนี้

1. การจ่ายพลังงาน: เมื่อเสียบสาย USB กล้องจะได้รับพลังงานและเริ่มต้นการทำงาน
2. การตรวจจับอุปกรณ์: ระบบปฏิบัติการบนคอมพิวเตอร์จะตรวจพบกล้องในฐานะอุปกรณ์วิดีโอ (UVC – USB Video Class) และไมโครโฟน (USB Audio Class) โดยอัตโนมัติ
3. การจับภาพและเสียง: เลนส์และเซ็นเซอร์ภาพจะทำงานร่วมกันเพื่อบันทึกวิดีโอที่ความละเอียดสูงสุด 720p ที่อัตราเฟรม 30 fps ขณะเดียวกันไมโครโฟนจะบันทึกเสียง
4. การประมวลผลและส่งข้อมูล: สัญญาณภาพและเสียงจะถูกประมวลผลขั้นต้น แล้วส่งผ่านสาย USB ไปยังคอมพิวเตอร์
5. การใช้งานในโปรแกรม: ผู้ใช้สามารถเปิดโปรแกรมต่าง ๆ เช่น Zoom, Microsoft Teams, OBS หรือ Skype เพื่อรับภาพและเสียงจากกล้องมาใช้งานได้ทันที
6. การปรับโฟกัส: ผู้ใช้สามารถหมุนเลนส์เพื่อปรับโฟกัสให้ภาพชัดเจนตามระยะทางของผู้ใช้งาน

บทที่ 2

การออกแบบและพัฒนา

2.1 แผนดำเนินการของทีม

Sprint	Goal	งานหลัก	Deliverable
Sprint 1 (วันที่ 1–7)	- ระบบ Sensor และ AI เบื้องต้น	- ติดตั้ง Ultrasonic Sensor + ระบบล็อกประตู - ติดตั้ง Load Cell + Buzzer	- Sensor ทำงานอ่านค่าได้
Setup Hardware & AI Prototype	AI เบื้องต้นทำงานได้	- อ่านค่าน้ำหนักเริ่มต้นและส่งข้อมูลขึ้น MCU/Serial - เก็บ Dataset ของสินค้า (ขนม, น้ำ)	- AI Prototype จำแนกสินค้าได้
	- ทดสอบอุปกรณ์ว่าใช้งานได้จริง	- เทรนโมเดล AI เบื้องต้น (จำแนกสินค้า 2 ประเภท) - ออกแบบ Database Schema เบื้องต้น + ราคาสินค้า - ทำ UI เบื้องต้นแสดงน้ำหนักขึ้นวาง - ทดสอบ L298 with DC Motor 12V - ออกแบบระบบสายพาน	- Database + UI เบื้องต้นใช้งานได้ - L298 สามารถควบคุม DC Motor ได้
Sprint 2 (วันที่ 8–14)	- เชื่อม Sensor + AI Core System Integration	- รวม Ultrasonic + Load Cell → MCU → ส่งข้อมูลเข้าฐานข้อมูล - ปรับ AI ให้แม่นยำขึ้น และส่งผลลัพธ์สินค้าไป Database - เขียนระบบคำนวณราคาสินค้าอัตโนมัติ - ทดสอบระบบสายพาน	- ระบบอ่านน้ำหนัก + AI → Database ทำงานร่วมกันได้ - UI แสดงรายการสินค้า + ราคา - มี QR Code สำหรับชำระเงิน - ระบบสายพานไม่เป็นตามต้องการ
Sprint 3 (วันที่ 15–21)	- ระบบครบ Loop พร้อม ทดสอบและ Debug	- ทำระบบ Cross Verification (เปรียบเทียบน้ำหนักขึ้นวาง กับรายการจาก AI) - แสดงผล Success / Error และปลดล็อกประตูตามผล	- ระบบ Prototype ทำงานครบทุกขั้นตอน - ระบบ Cross-check ทำงาน ป้องกันการโกง
Full Prototype & Testing	- สร้างระบบ Pusher	- เชื่อมระบบ QR Payment จริง (หรือ Mock API)	- ลูกค้าสามารถซื้อสินค้าและจ่ายเงินได้จริง

Sprint	Goal	งานหลัก	Deliverable
		- Log ประวัติการซื้อขาย + แจ้งเตือนกรณีผิดปกติ- - Integration Test (Run Full Loop: ลูกค้าเข้า → เลือกสินค้า → ชำระเงิน → ออก) - Debug รอบแรกและรอบสอง - ปรับ UI/UX ให้ชัดเจน- เตรียม Demo + Present Prototype - ออกแบบระบบ Pusher ในการผลักดันสินค้า	- Pusher ยังคงต้องปรับแต่งโครงสร้างและเปลี่ยนแบบ
Sprint 4 (วันที่ 22-30) Tuning & Decorating	- ปรับแต่งระบบและอุปกรณ์ให้ดีขึ้น - ปรับการจัดวางพร้อมตกแต่ง- จัดทำรายงาน	- เพิ่มระบบ Slider ทำงานร่วมกับ Pusher ในการทำ Clustering อย่างง่าย - ออกแบบกล่องรองรับสินค้าที่ผ่านระบบ Pusher และ Slider - จัดวางโครงสร้างและสายไฟทั้งหมดให้เป็นระเบียบ - ตกแต่งโครงสร้างให้มีความน่าสนใจ ดึงดูดลูกค้า - จัดเตรียมข้อมูล และจัดทำรายงานการทำงานทั้งหมด	- Pusher สามารถทำงานได้ดีขึ้น - Slider สามารถยกขนมได้จริง - โครงสร้างมีความเป็นระเบียบมากขึ้น - โครงสร้างผ่านการตกแต่ง - รายงานจัดทำได้เสร็จตามกำหนด

ตารางสรุป Sprint Plan

2.2 แผนงบประมาณ

ลำดับ	รายการอุปกรณ์	จำนวน	ราคาต่อหน่วย (บาท)	ราคารวม (บาท)	หมายเหตุ
1	Arduino UNO R3	2	137	274	*ของส่วนตัว
2	ESP32	1	164	164	*ของส่วนตัว
3	ODROID C4	1	4,200	4,200	*ยืมแลป
4	Micro Servo SG90	2	70	140	
5	Mini Servo MG90S	3	60	180	
6	RC522 RFID	1	39	39	*ของส่วนตัว
7	HC-SR04 Ultrasonic	1	40	40	

ลำดับ	รายการอุปกรณ์	จำนวน	ราคาต่อ หน่วย (บาท)	ราคารวม (บาท)	หมายเหตุ
8	HX711 and Loadcell	2	129	258	
9	4x4 Matrix Membrane Keypad	1	28	28	
10	PCF8574	1	100	100	*ยืมแลป
11	DHT22	1	50	50	*ยืมแลป
12	16x2 LCD with I2C	2	140	280	*ยืมแลป
13	LED สี	2	2	4	
14	Tact Switch Push Button	1	20	20	
15	Webcam NWM-560	1	690	690	*ยืมแลป
16	Bread Board	3	29	87	*ของส่วนตัว
17	กระดาสลิ่ง	10	15	150	
18	ไส้กา	23	3	69	
19	กระดาสี	6	15	80	
20	แผ่นโฟมเคลือบ	1	55	55	
21	สายไฟ 22 AWG สีดำ 100 ฟุต	1	160	160	*ของส่วนตัว
22	สายไฟ 20 AWG สีดำ 1 เมตร	3	20	60	
23	สายไฟ 20 AWG สีแดง 1 เมตร	3	20	60	
24	สายจัมเปอร์ ชุดละ 40 เส้น	4	25	100	
25	เทปกาวสองหน้า แบบบาง	5	10	50	
รวมงบประมาณทั้งหมดที่ใช้ไป				7,338	

2.2.1 การจัดสรรงบประมาณ

- งบประมาณจริง

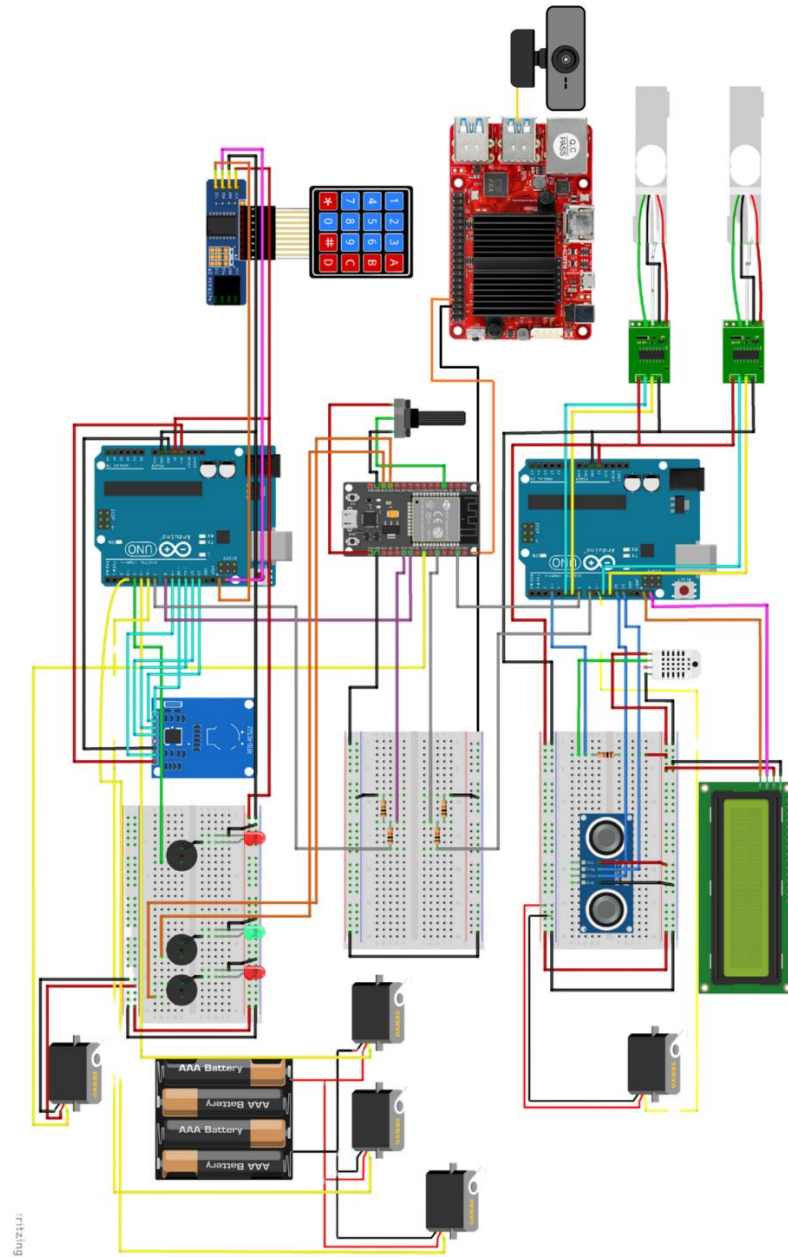
โดยคำนวณจากตามราคาอุปกรณ์ทั้งหมดที่ใช้จริง รวมทั้งสิ้น 7,338 บาท

- งบประมาณจากการสนับสนุน

โดยคำนวณจากงบประมาณจริง หักลบราคาอุปกรณ์ที่ได้จากการยืมห้องแลปฮาร์ดแวร์

คิดเป็นงบประมาณทั้งสิ้น 2,018 บาท (อุปกรณ์ที่ยืมจากห้องแลปฮาร์ดแวร์คิดเป็นงบประมาณทั้งสิ้น 5,320 บาท)

2.3 Schematic



ภาพที่ 19 Schematic ระบบทั้งหมด

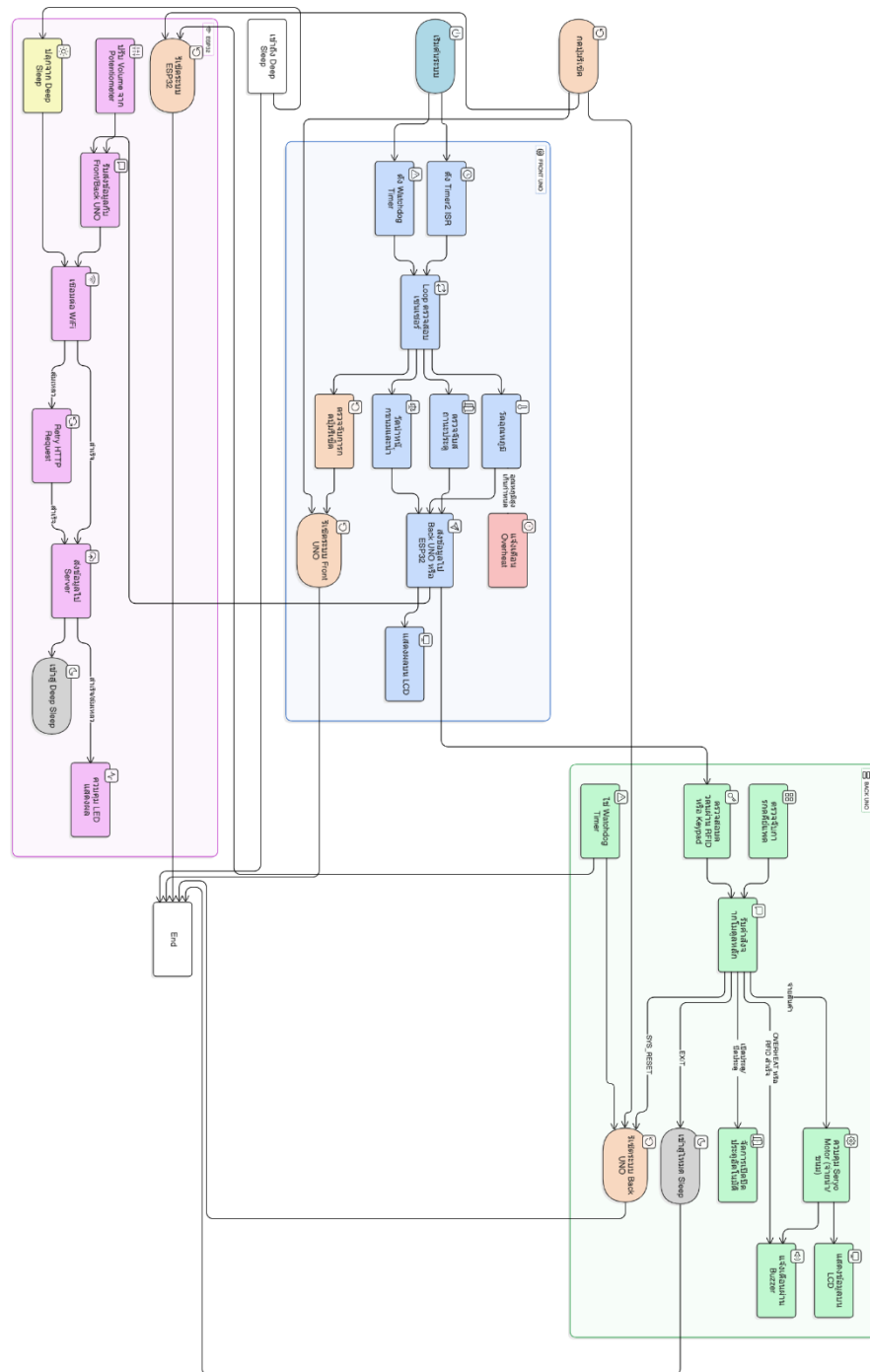
Front UNO ทำหน้าที่เป็นหน่วยรับข้อมูลจากเซนเซอร์หลักและอุปกรณ์ตรวจวัดต่าง ๆ โดยมีการเชื่อมต่อปุ่มรีเซ็ตไปยังขา D2 เพื่อใช้ตรวจจับการกดปุ่มและส่งคำสั่งรีเซ็ตระบบผ่าน External Interrupt สำหรับการตรวจวัดสภาพอากาศ ภายในตู้มีเซนเซอร์ DHT22 ต่ออยู่กับขา D3 เพื่ออ่านค่าอุณหภูมิและความชื้น นอกจากนี้ยังมีการติดตั้งโหลดเซลล์ 2 ชุดสำหรับตรวจวัดน้ำหนักขนมและน้ำ โดยโหลดเซลล์ขนมเชื่อมต่อกับ HX711 ผ่านขา D5 (SCK) และ D6 (DOUT) ขณะที่โหลดเซลล์น้ำเชื่อมต่อกับขา D10 (SCK) และ D11 (DOUT) การวัดระยะทางเพื่อป้องกันสิ่งกีดขวางใช้เซนเซอร์อัลตราโซนิก HC-SR04 ซึ่งต่อกับขา D12 (ECHO) และ D13 (TRIG) การแสดงผลข้อมูลทำได้ผ่านจอ LCD I2C ที่ต่อกับขา A4 และ A5 ข้อมูลจากอุปกรณ์เหล่านี้ถูกส่งต่อไปยัง ESP32 ผ่านพอร์ต SoftwareSerial (ขา D7 – RX, D8 – TX) เพื่อการประมวลผลต่อไป นอกจากนี้ ขา D9 ยังเชื่อมต่อกับ Servo Motor เพื่อควบคุมการเปิดและปิดประตูตู้ และขา D4 ใช้เป็นสัญญาณควบคุมอุปกรณ์เสริม เช่น พัดลมหรือไฟ

Back UNO ทำหน้าที่ควบคุมการจ่ายสินค้าและรับข้อมูลจากอุปกรณ์เสริมต่าง ๆ โดยมีเซอร์โวมอเตอร์สำหรับดันขนม (Pusher) ต่อกับขา D3 และเซอร์โวสำหรับจ่ายน้ำสองช่องต่อกับขา D5 และ D6 นอกจากนี้ ยังมี Buzzer ต่อกับขา D4 เพื่อแจ้งเตือนเหตุการณ์ต่าง ๆ เช่น การอ่าน RFID หรือเกิดเหตุฉุกเฉิน ระบบอ่าน RFID ใช้ขา D9 – D13 โดยขา D9 ใช้รีเซ็ตโมดูล RFID ขา D10 ใช้เลือกอุปกรณ์ (SS/CS) และขา D11 – D13 ใช้สำหรับส่งและรับข้อมูลผ่าน SPI ขา A0 เชื่อมต่อกับ Servo Motor หลักที่ใช้ควบคุมการเปิด/ปิดประตูเครื่อง Back UNO ยังสามารถรับและส่งข้อมูลกับ ESP32 ผ่าน SoftwareSerial (D7 – RX, D8 – TX)

ESP32 ทำหน้าที่เป็นหน่วยประมวลผลหลักของระบบ โดยรวบรวมข้อมูลจาก Front UNO, Back UNO และอุปกรณ์ภายนอกอื่น ๆ เช่น แผงควบคุม (Keypad/Display) และกล้อง AI (Odroid) ขา 16 และ 17 ใช้เชื่อมต่อกับแผงควบคุมผ่าน UART Serial2 เพื่อรับคำสั่งจากผู้ใช้และส่งข้อมูลตอบกลับ เช่น การยืนยันรหัสสินค้าหรือปรับระดับเสียง ขา 21 และ 22 ใช้เชื่อมต่อกับ Arduino ขึ้นวาง (Front/Back UNO) ผ่าน UART Serial1 เพื่อรับข้อมูลน้ำหนัก, สถานะประตู และคำสั่งรีเซ็ต รวมถึงส่งข้อมูลการแสดงผล ขา 23 เชื่อมต่อกับกล้อง AI เพื่อรับรหัสสินค้าที่ตรวจจับได้ ขา 18 ใช้ควบคุม Servo Motor สำหรับเปิด/ปิดประตู ขณะที่ LED แสดงสถานะการทำงานต่อกับขา 12 (LED_FAIL) และ 13 (LED_SUCCESS) ส่วนขา 32 เป็นปุ่มปลุกระบบจาก Deep Sleep

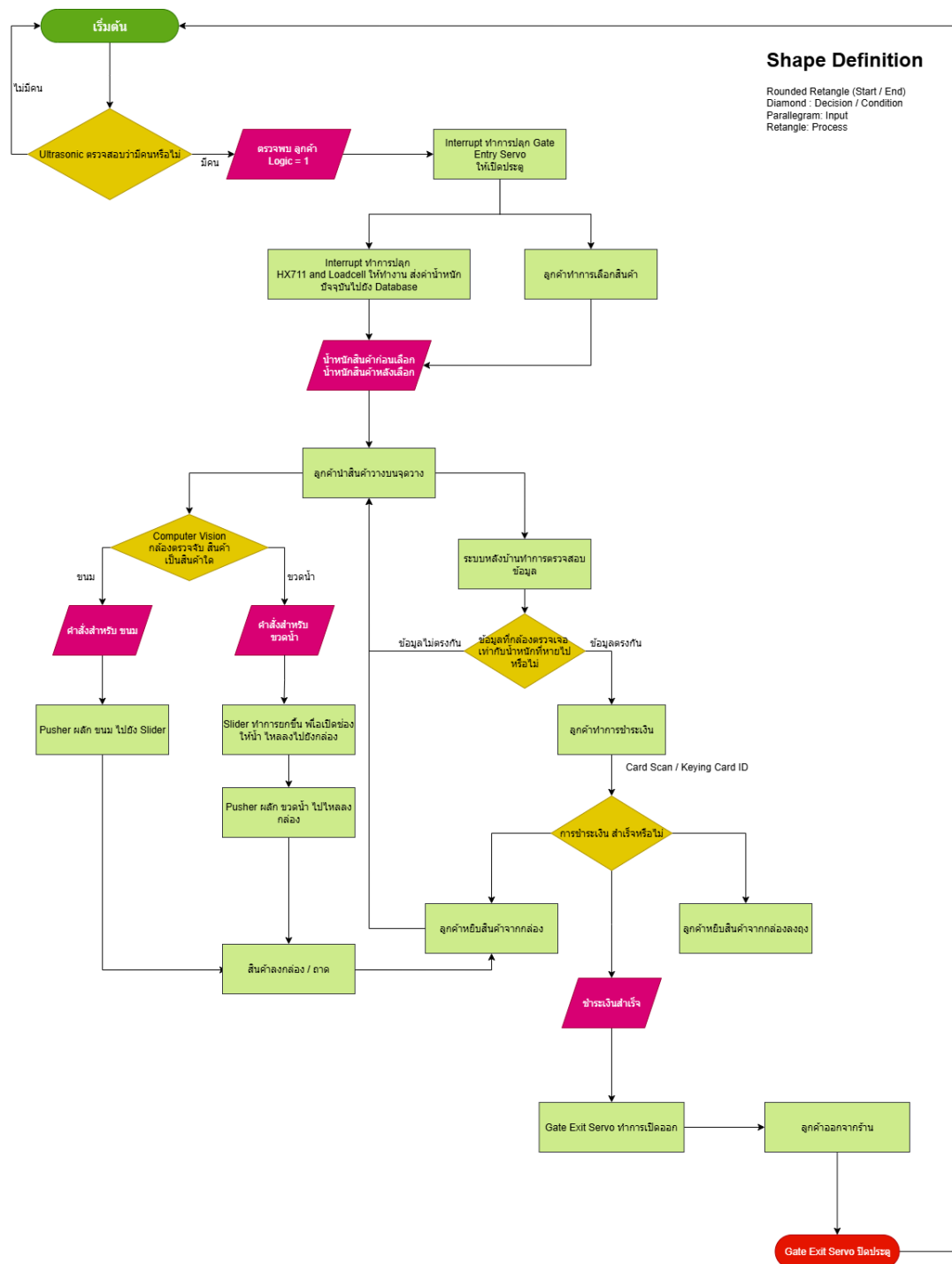
โดยรวมแล้ว ระบบนี้ทำงานเป็นลำดับขั้นตอน เริ่มจาก การตรวจวัดน้ำหนักและสภาพแวดล้อมในตู้, การตรวจสอบการกดปุ่มและ RFID, การประมวลผลคำสั่งจากผู้ใช้, การควบคุม Servo Motor และแจ้งสถานะผ่าน LED/Buzzer/Display ข้อมูลทั้งหมดถูกประสานงานผ่าน ESP32 ทำให้ระบบสามารถทำงานแบบอัตโนมัติได้ครบทุกฟังก์ชันและมีความปลอดภัยสูง

2.4 State Diagram ของระบบ



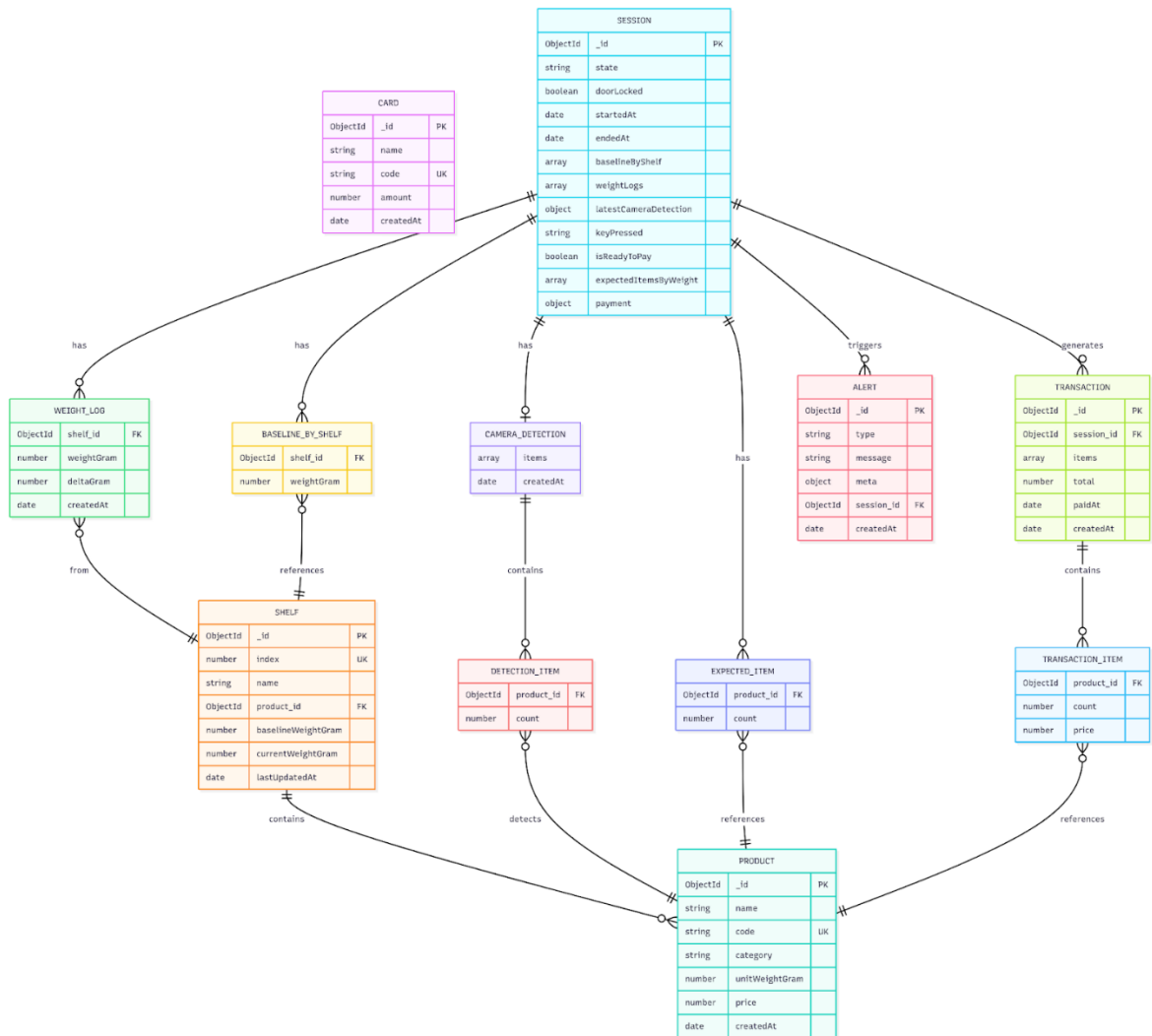
ภาพที่ 20 State Diagram

2.5 Flowchart ของระบบ



ภาพที่ 21 Flow Chart

2.6 Database Schema



ภาพที่ 22 Database Schema

ระบบฐานข้อมูลของตู้ขายสินค้าอัจฉริยะออกแบบด้วยโครงสร้าง Collection หลัก 6 รายการ ดังนี้:

1) PRODUCT – จัดเก็บข้อมูลสินค้า

- ฟิลด์หลัก: ชื่อสินค้า, รหัสสินค้า (Unique Code), ประเภท (หมวดหมู่: SNACK, WATER, OTHER), น้ำหนักต่อหน่วย, ราคา
- ใช้สำหรับอ้างอิงสินค้าทุกชนิดภายในระบบ

2) SHELF – จัดเก็บข้อมูลชั้นวางสินค้า

- ฟิลด์หลัก: รหัสชั้นวาง, น้ำหนักฐาน (Baseline Weight), น้ำหนักปัจจุบัน, รายการสินค้าที่วางอยู่
- ใช้ติดตามการเปลี่ยนแปลงน้ำหนักแบบเรียลไทม์เพื่อตรวจจับการหยิบ/วางสินค้า

3) SESSION – จัดเก็บข้อมูลเซสชันการใช้งานของลูกค้า

- ฟิลด์หลัก: สถานะ (OPEN → CHECKOUT_PENDING → PAID → CLOSED), บันทึกการเปลี่ยนแปลงน้ำหนักทุกชั้น, ผลการตรวจจับจากกล้อง, ข้อมูลการชำระเงิน (เช่น QR Code, สถานะการชำระเงิน), สถานะการควบคุมประตู (ล็อก/ปลดล็อก)
- เป็นหัวใจของระบบ ทำหน้าที่เชื่อมโยงทุกกระบวนการตั้งแต่เริ่มใช้งานจนถึงปิดการซื้อ

4) TRANSACTION – จัดเก็บข้อมูลใบเสร็จ/รายการซื้อขายที่เสร็จสมบูรณ์

- ฟิลด์หลัก: รายการสินค้า (รหัสสินค้า, จำนวน, ราคาต่อหน่วย), ยอดรวม, รหัสอ้างอิง Session
- สร้างขึ้นเมื่อการชำระเงินเสร็จสิ้น และเชื่อมโยงกลับไปยัง Session ที่เกี่ยวข้อง

5) ALERT – จัดเก็บบันทึกการแจ้งเตือน

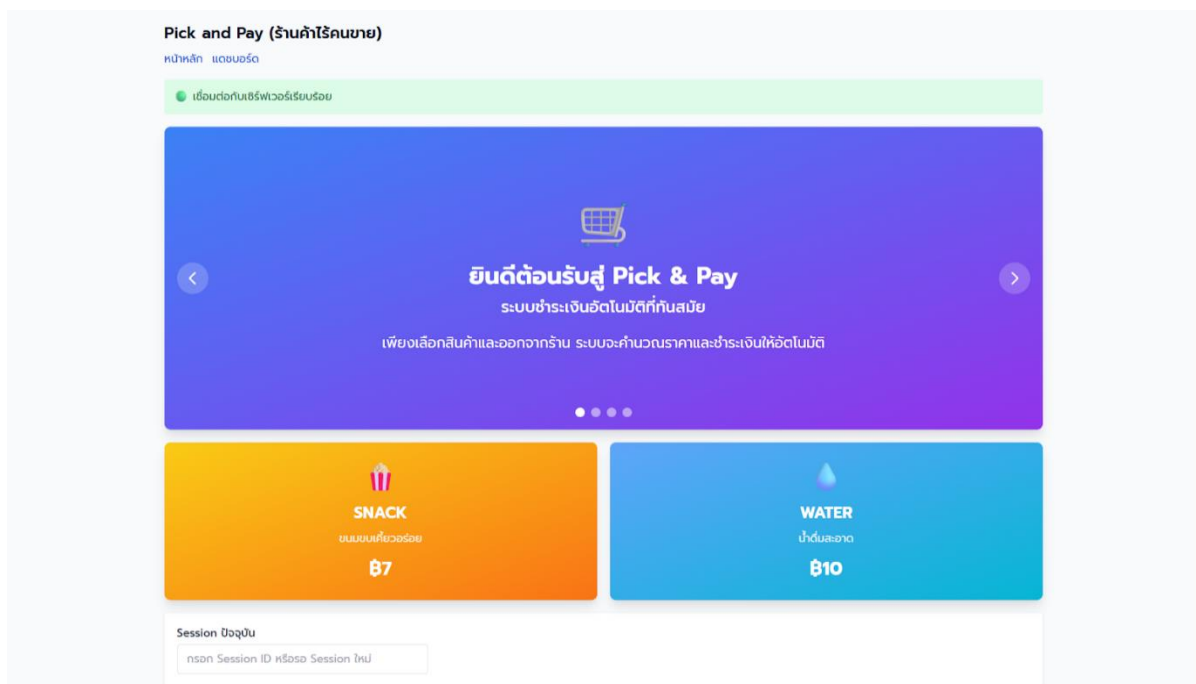
- ฟิลด์หลัก: ประเภทการแจ้งเตือน (ANOMALY, SECURITY, SYSTEM), รายละเอียดเหตุการณ์, รหัสอ้างอิง Session (ถ้ามี), วันเวลาเกิดเหตุ
- ใช้สำหรับตรวจสอบและติดตามเหตุการณ์ผิดปกติหรือปัญหาด้านความปลอดภัย

6) CARD – จัดเก็บข้อมูลบัตร RFID

- ฟิลด์หลัก: รหัสบัตร (UID), ประเภทบัตร (สมาชิก/เงินสด), ยอดเงินคงเหลือ (สำหรับระบบ Prepaid)

- รองรับการใช้งานบัตรเพื่อเข้าถึงตู้หรือชำระเงิน

2.7 การออกแบบ UI/UX



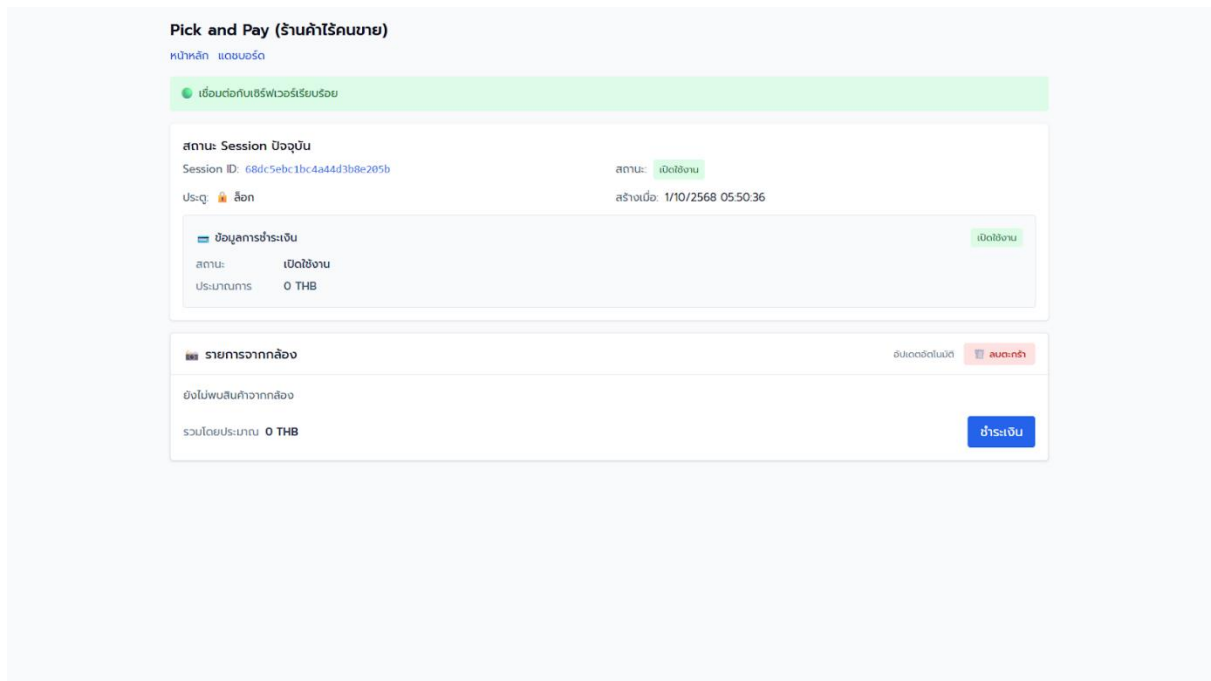
ภาพที่ 23 หน้าแรกของเว็บไซต์ร้านค้า

หน้าแรก (Home Page) – โหมดแนะนำร้านค้าและสินค้า

ในกรณีที่ไม่มี Session ใดๆ กำลังทำงานอยู่ในระบบ (เช่น ผู้ใช้งานยังไม่ได้เริ่มเซสชันการซื้อสินค้า) หน้าแรกจะแสดงหน้าต้อนรับพร้อมแนะนำร้านค้าและสินค้าที่มีให้บริการ โดยมีรายละเอียดดังนี้:

- 1) หัวข้อหลัก: “ยินดีต้อนรับสู่ Pick & Pay”
- 2) คำอธิบายระบบ: ระบบที่ช่วยให้ผู้ใช้งานสามารถเลือกสินค้าจากภายในร้าน คำนวณราคาและชำระเงินได้โดยไม่ต้องเดินไปยังจุดชำระเงิน
- 3) รายการสินค้าแนะนำ:
 - SNACK – ขนมป๊อปคอร์น เริ่มต้นที่ 7 บาท
 - WATER – น้ำดื่มสะอาด เริ่มต้นที่ 10 บาท

นอกจากนี้ ยังมีช่องกรอก “Session ID” สำหรับผู้ใช้งานที่ต้องการเข้าสู่เซสชันที่มีอยู่แล้ว หรือเริ่มเซสชันใหม่



ภาพที่ 24 การแสดงผลเมื่อเซนเซอร์ตรวจจับเจอลูกค้า

เมื่อเซนเซอร์ตรวจจับการเคลื่อนไหวของลูกค้าเข้าสู่พื้นที่ร้านค้า ระบบจะเริ่มต้น Session อัตโนมัติ โดยระบุ Session ID และบันทึกเวลาเริ่มต้น เพื่อติดตามกิจกรรมของผู้ใช้งานตลอดกระบวนการเลือกสินค้า ในระหว่างที่ลูกค้าเลือกสินค้า ระบบจะตรวจสอบความสอดคล้องระหว่าง รายการสินค้าที่ถูกหยิบ กับ การเปลี่ยนแปลงน้ำหนักบนชั้นวางสินค้า ซึ่งเป็นกลไกหลักในการยืนยันความถูกต้องของรายการสินค้าที่เลือก

เงื่อนไขการอนุญาตให้ชำระเงิน

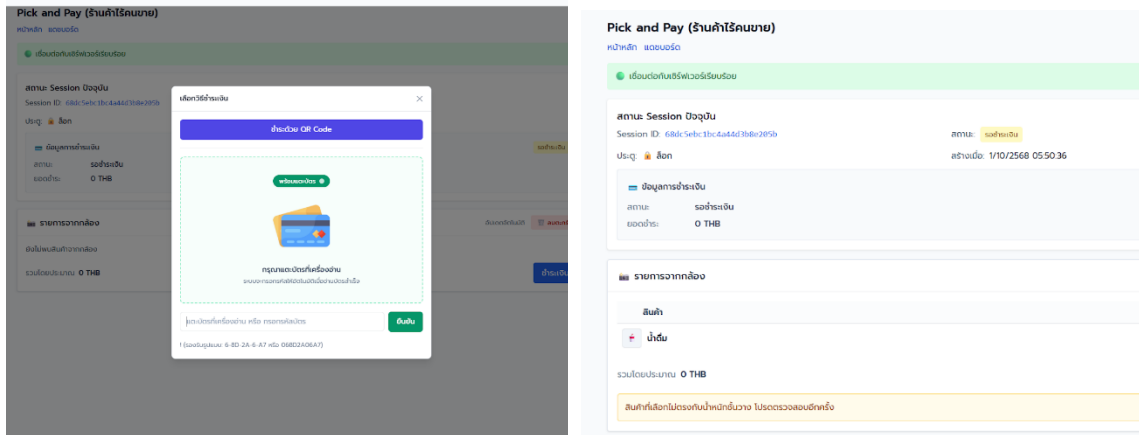
หากน้ำหนักที่ลดลงบนชั้นวางตรงกับน้ำหนักของสินค้าที่ถูกหยิบ ระบบจะทำการแสดงสถานะ “รอชำระเงิน” และเปิดให้ผู้ใช้งานสามารถดำเนินการชำระเงินได้

เมื่อกดปุ่ม “ชำระเงิน” ระบบจะแสดงหน้าต่าง pop-up สำหรับเลือกวิธีชำระเงิน เช่น ชำระผ่าน QR Code หรือบัตรเครดิต/เดบิต

กรณีไม่อนุญาตให้ชำระเงิน

หากพบว่า น้ำหนักที่ลดลงไม่ตรงกับสินค้าที่ถูกหยิบ (เช่น หยิบสินค้าแต่ไม่ได้หยิบจากชั้นวางที่มีเซนเซอร์ หรือมีการแทรกแซง) ระบบจะ ไม่อนุญาตให้ชำระเงิน และแสดง แจ้งเตือน พร้อมข้อความแจ้งเหตุผล เช่น “สินค้าที่เลือกไม่ตรงกับน้ำหนักที่ลดลง โปรดตรวจสอบสินค้าอีกครั้ง”

ระบบจะยังคงอยู่ในสถานะ “รอชำระเงิน” จนกว่าผู้ใช้งานจะแก้ไขหรือวางสินค้าคืน แล้วทำการเลือกใหม่ เพื่อให้เกิดความสอดคล้องกับข้อมูลน้ำหนัก



ภาพที่ 25 การแสดงผลเมื่อสินค้าที่เลือกตรง และไม่ตรงกับน้ำหนักของชั้นวาง

หน้า Dashboard

Pick and Pay (ร้านค้าไร้คนขาย)

หน้าหลักแดชบอร์ด

แดชบอร์ด

จำนวนออเดอร์ทั้งหมด

50

รายได้รวม (THB)

190

จำนวนสินค้า

2

สรุปสถานะเซสชัน

PAID: 9

UNKNOWN: 23

OPEN: 16

CHECKOUT_PENDING: 2

สินค้าปัจจุบัน

สินค้า	หมวดหมู่	จำนวน	ราคา
ขนม	SNACK	2	7
น้ำดื่ม	WATER	0	10

ประวัติเซสชัน (จากเครื่องนี้)

Session ID	สถานะ	เวลา	ยอดชำระ
68dc6d731bc4a44d3b8e282d	PAID	1/10/2568 06:56:19	10
68dc6c8f1bc4a44d3b8e266c	PAID	1/10/2568 06:52:00	10
68dc6c081bc4a44d3b8e25d0	เปิดใช้งาน	1/10/2568 06:47:20	-
68dc6b621bc4a44d3b8e2436	OPEN	1/10/2568 06:46:22	-
68dc6a991bc4a44d3b8e2389	PAID	1/10/2568 06:42:07	10

ภาพที่ 26 การแสดงผล Dashboard

หน้า Dashboard ของระบบ Pick and Pay (ร้านค้าไร้คนขาย) เป็นหน้าที่แสดงข้อมูลสรุปการขายและสถานะของสินค้าภายในร้านค้าอัตโนมัติ โดยมีการจัดวางข้อมูลเป็นส่วน ๆ เพื่อให้ผู้ใช้งานสามารถติดตามสถานะของระบบและยอดขายได้อย่างสะดวก

ในส่วนบนของหน้า Dashboard จะแสดงข้อมูลสรุป จำนวนเซสชันทั้งหมด, รายได้รวม และจำนวนสินค้าที่ขายไป จากภาพตัวอย่าง จำนวนเซสชันทั้งหมดคือ 50 รายการ รายได้รวม 190 บาท และจำนวนสินค้าที่ขายได้ทั้งหมด 2 ชิ้น ข้อมูลเหล่านี้ช่วยให้ผู้ดูแลระบบเห็นภาพรวมการขายแบบเรียลไทม์

สรุปสถานะเซสชัน ซึ่งแบ่งตามสถานะของการทำธุรกรรม ได้แก่

- PAID หมายถึงเซสชันที่ลูกค้าชำระเงินเรียบร้อยแล้ว
- UNKNOWN หมายถึงเซสชันที่สถานะไม่สามารถระบุได้
- OPEN หมายถึงเซสชันที่ยังเปิดอยู่และลูกค้ายังไม่ได้ทำรายการชำระเงิน
- CHECKOUT_PENDING หมายถึงเซสชันที่กำลังอยู่ระหว่างการชำระเงิน

ในตัวอย่างมีเซสชันที่ชำระเงินแล้ว 9 เซสชัน, UNKNOWN 23 เซสชัน, OPEN 16 เซสชัน และ CHECKOUT_PENDING 2 เซสชัน

รายการสินค้าปัจจุบัน แสดงรายละเอียดของสินค้าแต่ละประเภท ประกอบด้วยชื่อสินค้า, หมวดหมู่, จำนวน และราคา จากภาพ ตัวอย่างสินค้ามี

- ขนม (SNACK) จำนวน 2 ชิ้น ราคา 7 บาทต่อชิ้น
- น้ำดื่ม (WATER) จำนวน 0 ชิ้น ราคา 10 บาทต่อชิ้น
- สุดท้ายเป็น ประวัติเซสชันจากเครื่องขาย (Session History) ซึ่งแสดงรายละเอียดของแต่ละเซสชัน ประกอบด้วย
- Session ID รหัสประจำตัวของเซสชัน
- สถานะ ของเซสชัน เช่น PAID หรือ OPEN
- เวลา ที่เซสชันเกิดขึ้น
- ยอดชำระ หากลูกค้าชำระเงินแล้ว

ข้อมูลเหล่านี้ช่วยให้ผู้ดูแลระบบสามารถติดตามรายการขายย้อนหลัง ตรวจสอบปัญหา และประเมินประสิทธิภาพของร้านค้าไร้คนขายได้อย่างครบถ้วน

บทที่ 3

การเขียนโปรแกรมและการทำงาน

3.1 การจัดการระบบ

ในการพัฒนาระบบได้ทำการแบ่งส่วนการทำงานหลักออกเป็น 2 ส่วน คือ Embedded System และ Database System

3.1.1 Embedded System

แบ่งเป็นส่วนย่อย 3 ส่วน ได้แก่:

- 1) Front UNO ควบคุมการทำงานส่วนหน้า ซึ่งประกอบด้วย Servo Gate Entry, Ultrasonic, Loadcell 1 & 2, LCD , DHT22, Reset Switch
- 2) Back UNO ควบคุมการทำงานส่วนหลัง ซึ่งประกอบด้วย Keypad I2C, RFID, Servo Pusher, Servo Slider 1 & 2, Servo Gate Exit
- 3) ESP32 ควบคุมการยิง API ส่งค่าไปยัง Database

3.1.2 Database System

สำหรับจัดเก็บข้อมูลสินค้าและการทำงานของระบบ

3.2 โปรแกรมและการทำงานของ Front UNO

3.2.1 หน้าที่หลัก

บอร์ด Front UNO ทำหน้าที่ควบคุมและตรวจสอบการทำงานของตู้ขนมและน้ำอัจฉริยะ โดยสามารถตรวจจับสถานะการเปิด-ปิดประตู วัดน้ำหนักของขนมและน้ำ ตรวจสอบอุณหภูมิภายในตู้ และส่งข้อมูลทั้งหมดไปยัง Back UNO หรือ ESP32 เพื่อนำไปประมวลผลและแสดงผลต่อไป การทำงานในส่วนนี้ถือเป็นหัวใจสำคัญของระบบ ทำให้สามารถติดตามสภาพสินค้าและควบคุมความปลอดภัยได้อย่างมีประสิทธิภาพ

3.2.2 เซนเซอร์และอุปกรณ์

1. HC-SR04: ใช้วัดระยะเพื่อควบคุมการเปิด-ปิดประตู
2. DHT22: ตรวจจับอุณหภูมิภายในตู้และควบคุมระบบระบายความร้อน
3. HX711 + โหลดเซลล์: วัดน้ำหนักของขนมและน้ำ
4. Servo Motor: ขับกลไกเปิด-ปิดประตู
5. LCD I²C: แสดงผลน้ำหนักและจำนวนสินค้า
6. SoftwareSerial: ใช้ส่งและรับข้อมูลระหว่าง Front UNO กับ Back UNO/ESP32
7. สวิตช์รีเซ็ต: ส่งสัญญาณรีเซ็ตระบบเมื่อเกิด

3.2.3 การทำงานหลัก

- 1) ใช้ Timer2 ISR ตั้ง flag เพื่อให้ฟังก์ชัน loop() อ่านค่าจาก DHT22 และ HX711 แบบไม่บล็อกเวลา
- 2) ใช้ Watchdog Timer (WDT) ป้องกันไมโครคอนโทรลเลอร์ค้างและรีเซ็ตตัวเองอัตโนมัติ
- 3) อ่านและตรวจนับสถานะการเปิด-ปิดประตูผ่าน HC-SR04 และ Servo Motor
- 4) ตรวจสอบอุณหภูมิภายในตู้ และส่งสัญญาณแจ้งเตือน ALERT:OVERHEAT หากเกินกำหนด
- 5) วัดน้ำหนักขนมและน้ำ และส่งข้อมูลในรูปแบบ SNACK_WEIGHT:xxx และ WATER_WEIGHT:xxx
- 6) ส่งสถานะทั้งหมดไปยัง Back UNO หรือ ESP32 ผ่าน SoftwareSerial
- 7) ตรวจจบบการกดสวิตช์รีเซ็ต และดำเนินการรีเซ็ตระบบพร้อมส่งข้อมูลรีเซ็ตไปยังโมดูลหลัก

3.2.4 ขาและการใช้งาน (PIN Mapping)

ขา (PIN)	ต่อกับอุปกรณ์ใด	หน้าที่
2	ปุ่มรีเซ็ต (SW)	ตรวจจบบการกดปุ่ม → ส่งค่าขอรีเซ็ต (ใช้ External Interrupt)
3	DHT22	อ่านค่าอุณหภูมิ
5	HX711 (Snack Scale – SCK)	สัญญาณ Clock สำหรับโหลดเซลล์ขนม
6	HX711 (Snack Scale – DOUT)	สัญญาณข้อมูลจากโหลดเซลล์ขนม
7	SoftwareSerial RX	รับข้อมูลจากโมดูลภายนอก
8	SoftwareSerial TX	ส่งข้อมูลไปยังโมดูลภายนอก
9	Servo Motor	ควบคุมการเปิด/ปิดประตู
10	HX711 (Water Scale – SCK)	สัญญาณ Clock สำหรับโหลดเซลล์น้ำ
11	HX711 (Water Scale – DOUT)	สัญญาณข้อมูลจากโหลดเซลล์น้ำ
12	HC-SR04 (ECHO)	รับสัญญาณสะท้อนจากอัลตราโซนิก
13	HC-SR04 (TRIG)	ส่งสัญญาณกระตุ้นอัลตราโซนิก
4	WAKING_BACK (Output)	สัญญาณควบคุม (ใช้เปิด/ปิดอุปกรณ์เสริม เช่น พัดลมหรือไฟ)
A4, A5	LCD I2C (0x27)	สื่อสารกับจอ LCD ผ่านโปรโตคอล I2C

3.2.5 โปรแกรม

```
#include <Servo.h>
#include <SoftwareSerial.h>
#include "HX711.h"
#include "DHT.h"
#include "LiquidCrystal_I2C.h"
#include <avr/wdt.h>
#include <avr/interrupt.h>

#define TRIG_PIN 13
#define ECHO_PIN 12
#define SERVO_PIN 9
#define DHTPIN 3
#define DHTTYPE DHT22

const int SNACK_DOUT_PIN = 6;
const int SNACK_SCK_PIN = 5;
const int WATER_DOUT_PIN = 11;
const int WATER_SCK_PIN = 10;

const int SW = 2;

const int WAKING_BACK = 4;

SoftwareSerial mySerial(7, 8);
Servo myServo;

HX711 snackScale;
HX711 waterScale;
DHT dht(DHTPIN, DHTTYPE);
```

```
bool coolingMode = false;
unsigned long coolingStartTime = 0;
const unsigned long coolingDuration = 120000;

bool doorOpened = false;
unsigned long doorOpenTime = 0;

volatile unsigned long lastInterruptTime = 0;
const unsigned long debounceDelay = 50;
volatile bool resetRequested = false;

int previousSnackWeight = 0;
int previousWaterWeight = 0;

String totalSnack = "0";
String snackWeight = "0";
String totalWater = "0";
String waterWeight = "0";

LiquidCrystal_I2C lcd(0x27, 16, 2);

volatile uint8_t timer2Count = 0;
volatile bool dhtTrigger = false;
volatile bool weightTrigger = false;
volatile bool readSnackNext = true;

ISR(TIMER2_OVF_vect) {
```

```

timer2Count++;

if (timer2Count % 31 == 0) {
    weightTrigger = true;
    readSnackNext = !readSnackNext;
}

if (timer2Count >= 122) {
    dhtTrigger = true;
    timer2Count = 0;
}
}

void setupTimers() {
    TCCR2A = 0;
    TCCR2B = 0;
    TCNT2 = 0;

    TCCR2B |= (1 << CS22) | (1 << CS21) | (1 << CS20);

    TIMSK2 |= (1 << TOIE2);
}

void setup() {
    Serial.begin(9600);
    mySerial.begin(9600);

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(SW, INPUT_PULLUP);
}

```

```

pinMode(WAKING_BACK, OUTPUT);

digitalWrite(WAKING_BACK, 1);

myServo.attach(SERVO_PIN);
myServo.write(90);

snackScale.begin(SNACK_DOUT_PIN, SNACK_SCK_PIN);
snackScale.tare();
snackScale.set_scale(408.12);

waterScale.begin(WATER_DOUT_PIN, WATER_SCK_PIN);
waterScale.tare();
waterScale.set_scale(444.44);

lcd.init();
lcd.backlight();
lcd.home();
lcd.print("ZZZZZZZZZ");

dht.begin();

attachInterrupt(digitalPinToInterrupt(SW), resetHandler, FALLING);

setupTimers();
wdt_enable(WDTO_8S);

Serial.println(F("=== ระบบตู้ขนมอัจฉริยะเริ่มทำงาน ==="));
}

```

```

void loop() {
    wdt_reset();

    processDHTSensor();
    processWeightSensors();
    processCoolingSystem();
    processDoorControl();
    processSerialMessages();
    processResetRequest();
}

void resetHandler() {
    static unsigned long lastInterruptTime = 0;
    unsigned long now = micros();
    if (now - lastInterruptTime > 50000) {
        resetRequested = true;
        lastInterruptTime = now;
    }
}

long getDistance() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);
    if (duration == 0) return -1;
    return duration * 0.034 / 2;
}

```

```

}

void openDoor() {
  for (int pos = 90; pos >= 0; pos--) {
    myServo.write(pos);
    delay(10);
  }
  digitalWrite(WAKING_BACK, 0);
  doorOpened = true;
  doorOpenTime = millis();
}

void closeDoor() {
  for (int pos = 0; pos <= 90; pos++) {
    myServo.write(pos);
    delay(10);
  }
  digitalWrite(WAKING_BACK, 1);
  doorOpened = false;
}

void processDHTSensor() {
  if (!dhtTrigger) return;
  dhtTrigger = false;

  float t = dht.readTemperature();
  Serial.print("TEMP = ");
  Serial.println(t);
  if (isnan(t)) {
    Serial.println(F("✗ ไม่สามารถอ่านค่าจาก DHT sensor!"));
  }
}

```

```

    return;
}

if (t >= 35 && !coolingMode) {
    Serial.println(F("🔥 อุณหภูมิสูงเกิน! เริ่มระบายความร้อน..."));
    mySerial.println("ALERT:OVERHEAT");
    if (!doorOpened) {
        openDoor();
    }
    coolingMode = true;
    coolingStartTime = millis();
}
}

void processWeightSensors() {
    if (!weightTrigger) return;
    weightTrigger = false;

    if (readSnackNext) {
        int currentSnackWeight = snackScale.get_units(1);
        if (currentSnackWeight >= 0 && currentSnackWeight <= 10000) {
            int delta = currentSnackWeight - previousSnackWeight;
            const int WEIGHT_THRESHOLD = 15;
            if (abs(delta) > WEIGHT_THRESHOLD) {
                if (delta > WEIGHT_THRESHOLD) {
                    Serial.println("🟢 มีคนเติมของเข้าตู้!");
                } else {
                    Serial.println("🔴 มีคนหยิบของออกจากตู้!");
                }
            }
            previousSnackWeight = currentSnackWeight;
        }
    }
}

```

```

    mySerial.print("SNACK_WEIGHT:");
    mySerial.println(currentSnackWeight);
}

Serial.print("🚰 น้ำหนักขนม: ");
Serial.print(currentSnackWeight, 1);
Serial.println(" g");
}

} else {
    int currentWaterWeight = waterScale.get_units(1);
    if (currentWaterWeight >= 0 && currentWaterWeight <= 10000) {
        int delta = currentWaterWeight - previousWaterWeight;
        const int WEIGHT_THRESHOLD = 100;
        if (abs(delta) > WEIGHT_THRESHOLD) {
            if (delta > WEIGHT_THRESHOLD) {
                Serial.println("💧 มีคนเติมน้ำเข้าตู้!");
            } else {
                Serial.println("🚰 มีคนหยิบน้ำออกจากตู้!");
            }
        }
        previousWaterWeight = currentWaterWeight;
        mySerial.print("WATER_WEIGHT:");
        mySerial.println(currentWaterWeight);
    }

    Serial.print("💧 น้ำหนักน้ำ: ");
    Serial.print(currentWaterWeight, 1);
    Serial.println(" g");
}

}

}

```

```

void processCoolingSystem() {
    if (!coolingMode) return;

    if (millis() - coolingStartTime >= coolingDuration) {
        Serial.println(F("✅ ครบเวลา 2 นาที ปิดประตูระบายความร้อน"));
        closeDoor();
        coolingMode = false;
    }
}

void processDoorControl() {
    long distance = getDistance();

    if (distance > 0 && distance < 5 && !doorOpened) {
        openDoor();
        mySerial.println("DOOR_OPENED");
    }

    if (doorOpened &&
        (millis() - doorOpenTime >= 2000) &&
        (distance == -1 || distance >= 20) &&
        !coolingMode) {
        closeDoor();
        mySerial.println("DOOR_CLOSED_BY_ULTRASONIC");
    }
}

void processSerialMessages() {
    if (!mySerial.available()) return;

```

```

String msg = mySerial.readStringUntil('\n');
msg.trim();
Serial.println(msg);

if (msg.startsWith("TOTAL_SNACK:")) {
    totalSnack = msg.substring(12);
} else if (msg.startsWith("WEIGHT_SNACK:")) {
    snackWeight = msg.substring(13);
} else if (msg.startsWith("TOTAL_wATER:")) {
    totalWater = msg.substring(12);
} else if (msg.startsWith("WEIGHT_WATER:")) {
    waterWeight = msg.substring(13);
}

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Water: ");
lcd.print(waterWeight);

lcd.setCursor(0, 1);
lcd.print("Snack: ");
lcd.print(snackWeight);
}

void processResetRequest() {
    if (!resetRequested) return;

    resetRequested = false;
    Serial.println("🔄 ตรวจพบคำขอรีเซ็ต!");
}

```

```

float snackW = snackScale.get_units(2);
float waterW = waterScale.get_units(2);

mySerial.print("SNACK_RESET:");
mySerial.println(snackW);
delay(2000);

mySerial.print("WATER_RESET:");
mySerial.println(waterW);
delay(2000);

mySerial.println("SYS_RESET:");
delay(8000);
}

```

หมายเหตุสำคัญ

1. ใช้ Flag ใน ISR แทนที่จะทำงานในนั้นเลย

การอ่าน DHT22 หรือ HX711 โดยตรงใน ISR (Interrupt Service Routine) เป็น สิ่งที่ไม่แนะนำอย่างยิ่ง — และในทางปฏิบัติ มักจะล้มเหลวหรือทำให้ระบบค้าง เนื่องจาก:

ปัญหาหลัก

- 1) DHT22 ใช้การสื่อสารแบบ one-wire ที่ต้องใช้ delayMicroseconds() และการรอสัญญาณ → ห้ามใช้ใน ISR
- 2) HX711 ใช้การสื่อสารแบบ bit-banging (SCK + DOUT) → ต้องใช้ digitalWrite() และ pulseIn() ซึ่ง ไม่ปลอดภัยใน ISR
- 3) ISR ต้องสั้นมาก (ไม่เกินไม่กี่ไมโครวินาที) — การอ่านเซนเซอร์ใช้เวลา 10–100 ms → ทำให้เกิด missed interrupts, ระบบค้าง, หรือ ข้อมูลผิด

กฎทองของ ISR

- 1) ห้ามใช้ delay(), millis(), Serial.print(), String, new, หรือฟังก์ชันที่ใช้เวลานาน
- 2) ห้ามเรียกไลบรารีที่ไม่ออกแบบมาสำหรับ ISR
- 3) ควรใช้แค่ set flag, increment counter, หรือ toggle pin

2. การใช้งาน Timer2 ใน Front UNO

งาน	ช่วงเวลา
อ่าน DHT22	ทุก 2,000 ms
สลับอ่านน้ำหนัก (snack/water)	ทุก 500 ms

แต่ Timer2 เป็น 8-bit → นับได้แค่ 256 ค่า → ต้องใช้ prescaler + นับรอบหลายครั้ง

การตั้งค่า Timer2 (ในฟังก์ชัน setupTimers())

(1) รีเซ็ต register

```
TCCR2A = 0; // โหมด Normal (ไม่ใช่ PWM)
TCCR2B = 0;
TCNT2 = 0; // เริ่มนับจาก 0
```

(2) ตั้ง prescaler = 1024

```
TCCR2B |= (1 << CS22) | (1 << CS21) | (1 << CS20);
```

- ความถี่ CPU ของ Uno = 16 MHz
- ด้วย prescaler 1024 → ความถี่ Timer = $16,000,000 / 1024 = 15,625$ Hz
- คาบเวลา (period) = $1 / 15,625 \approx 64 \mu\text{s}$ ต่อ tick

(3) คำนวณเวลา overflow

- Timer2 นับ 0 → 255 → overflow ที่ 256
- เวลาต่อ 1 รอบ overflow = $256 \times 64 \mu\text{s} = 16,384 \mu\text{s} = 16.384$ ms

ดังนั้น ทุก 16.384 ms → ISR จะถูกเรียก 1 ครั้ง

การนับเวลาใน ISR (TIMER2_OVF_vect)

เนื่องจาก 16.384 ms สั้นเกินไป เราจึง นับจำนวนรอบ เพื่อให้ได้เวลาที่ต้องการ:

(ก) สำหรับ 500 ms

- $500 \text{ ms} / 16.384 \text{ ms} \approx 30.52$
- เราใช้ 31 รอบ → $31 \times 16.384 \approx 508 \text{ ms}$ (คลาดเคลื่อน $\sim 1.6\%$ — ยอมรับได้)

```
if (timer2Count % 31 == 0) {
    weightTrigger = true; // สั่งให้อ่านน้ำหนัก
    readSnackNext = !readSnackNext; // สลับระหว่าง snack/water
}
```

(ข) สำหรับ 2,000 ms

- $2000 / 16.384 \approx 122.07$
- ใช้ 122 รอบ $\rightarrow 122 \times 16.384 \approx 1998.8$ ms (ใกล้เคียง 2 วินาที)

```
if (timer2Count >= 122) {  
  dhtTrigger = true; // สั่งให้อ่าน DHT  
  timer2Count = 0;   // รีเซ็ตตัวนับ
```

* ใช้ volatile กับตัวแปรที่ถูกเข้าถึงทั้งใน ISR และ loop() เพื่อป้องกัน compiler optimize ผิด

เหตุผลที่ไม่ทำทุกอย่างใน ISR

เนื่องจาก ISR ต้องสั้นมาก การอ่าน DHT22 หรือ HX711 ใช้เวลา 10–100 ms

หากทำใน ISR:

- 1) จะปิด global interrupt ชั่วคราว $\rightarrow$ ทำให้ Timer อื่นหยุด
- 2) อาจทำให้ Servo กระตุก, Serial หาย, หรือ WDT รีเซ็ต
- 3) ถ้าเซนเซอร์มีปัญหา $\rightarrow$ MCU ค้างใน ISR ตลอดกาล

ดังนั้นเราจึงใช้ flag (weightTrigger, dhtTrigger) $\rightarrow$ แล้วไปอ่านเซนเซอร์ใน loop() แทน

เหตุผลในการเลือกใช้งาน Timer2

TIMER	ใช้โดยใคร	ใช้ได้หรือไม่
Timer0	millis(), delay()	ห้ามแตะ
Timer1	Servo.h	ห้ามใช้ $\rightarrow$ จะเกิด multiple definition of __vector_11
Timer2	Aviabl	ใช้ได้ปลอดภัย

3.3 โปรแกรมและการทำงานของ Back UNO

3.3.1 หน้าที่หลัก

บอร์ด Back UNO ทำหน้าที่เป็น เครื่องจ่ายน้ำและขนมอัตโนมัติ โดยควบคุมการจ่ายสินค้าและตรวจสอบตัวตนของผู้ใช้งานผ่าน RFID หรือคีย์แพด ระบบนี้ถูกออกแบบมาให้ประหยัดพลังงานด้วยการเข้าสู่โหมด Sleep เมื่อไม่มีการใช้งาน และสามารถสื่อสารกับโมดูลหลัก เช่น ESP32 เพื่อส่งข้อมูลการจ่ายสินค้า ตลอดจนรับคำสั่งรีเซ็ตหรือแจ้งเตือนเหตุฉุกเฉิน ทำให้ระบบทำงานได้อย่างต่อเนื่อง ปลอดภัย และแม่นยำ

3.3.2 เซนเซอร์และอุปกรณ์

1. RFID Reader (SS_PIN 10, RST_PIN 9): ตรวจสอบตัวตนของผู้ใช้งานหรือสัตว์เลี้ยง
2. Keypad 4x4 (I²C, Addr 0x20): ใช้ป้อนคำสั่งหรือกดปุ่มควบคุมระบบ
3. Servo Motor 3 ตัว:
 - Left/Right Servo → ควบคุมช่องน้ำ
 - Pusher Servo → ดันขนมออกมา
4. LCD 16x2 I²C (0x27): แสดงจำนวนขนมและน้ำหนักปัจจุบัน
5. Buzzer (ขา 6): แจ้งเตือนเมื่ออ่าน RFID สำเร็จ หรือเกิดเหตุฉุกเฉิน
6. SoftwareSerial (ขา 7/8): สื่อสารกับโมดูลหลัก เช่น ESP32
7. INT0 (ขา D2): ใช้ปลุกบอร์ดจากโหมด Sleep

3.3.3 การทำงานหลัก

- 1) ตรวจสอบตัวตนผ่าน RFID Reader และส่งข้อมูลไปยังโมดูลหลักผ่าน SoftwareSerial
- 2) รับคำสั่งจากโมดูลหลัก เช่น
 - WATER001_200_OK → เริ่มจ่ายน้ำ
 - SNACK001_200_OK → จ่ายขนม
 - OVERHEAT → เปิดประตูระบายความร้อน
 - EXIT → ปิดประตูและเข้าสู่โหมด Sleep
 - VOLUME:xxx → ปรับระดับเสียง Buzzer
- 3) ใช้ State Machine ควบคุม Servo Motor ทั้งสามตัว สำหรับลำดับการจ่ายน้ำและขนม เช่น ปิด → ค้าง → ปิด
- 4) ตรวจสอบการกด คีย์แพด โดยรองรับการกดปุ่มปกติและกดค้าง (เช่น กดปุ่ม 0 ค้างเพื่อล้างข้อมูล)

- 5) แสดงข้อมูลจำนวนขนมและน้ำหนักปัจจุบันบน LCD 16x2 I²C
- 6) ให้เสียงแจ้งเตือนผ่าน Buzzer เมื่ออ่าน RFID สำเร็จ หรือเกิดเหตุฉุกเฉิน
- 7) ใช้ Watchdog Timer ป้องกันระบบค้าง และรองรับคำสั่งรีเซ็ตระบบ (SYS_RESET:) เพื่อเริ่มการนับถอยหลังและรีบูตบอร์ด
- 8) จัดการการเปิด-ปิดประตูอัตโนมัติ และใช้ AUTO_CLOSE_MS ปิดประตูหลังระยะเวลาที่กำหนด
- 9) รองรับการทำงานโหมด Sleep เพื่อประหยัดพลังงาน และปลุกบอร์ดด้วยสัญญาณจากขา D2 (INT0)

3.3.4 ขาและการใช้งาน (PIN Mapping)

ขา (PIN)	ต่อกับอุปกรณ์ใด	หน้าที่
D2	สวิตช์ / เซนเซอร์ปลุก (ภายนอก)	ใช้เป็นInterrupt 0 (INT0)เพื่อปลุกระบบจากโหมด Sleep
D3	Pusher Servo	ควบคุมการดันขนม (ดันออก/ถอยกลับ)
D6	Right Servo	ควบคุมช่องจ่ายน้ำด้านขวา
D5	Left Servo	ควบคุมช่องจ่ายน้ำด้านซ้าย
D4	Buzzer	สร้างเสียงแจ้งเตือน (เมื่ออ่าน RFID หรือเกิดเหตุฉุกเฉิน)
D7	RX ของโมดูลภายนอก (เช่น ESP32)	รับข้อมูลผ่าน SoftwareSerial
D8	TX ของโมดูลภายนอก	ส่งข้อมูลผ่าน SoftwareSerial
D9	RFID Module (ขา RST)	รีเซ็ต RFID Reader
D10	RFID Module (ขา SS / CS)	เลือกอุปกรณ์ RFID ในการสื่อสารแบบ SPI
D11	RFID Module (ขา MOSI)	ส่งข้อมูล SPI จาก Arduino ไปยัง RFID
D12	RFID Module (ขา MISO)	รับข้อมูล SPI จาก RFID มายัง Arduino
D13	RFID Module (ขา SCK)	สัญญาณนาฬิกา SPI
A0	Main Door Servo	ควบคุมการเปิด/ปิดประตูเครื่อง

3.3.5 โปรแกรม

```
#include <SPI.h>
#include <SoftwareSerial.h>
#include "RFID.h"
#include "Keypad_I2C.h"
#include "Keypad.h"
#include <Wire.h>
#include <Servo.h>
#include <avr/wdt.h>
#include <avr/sleep.h>
#include <avr/interrupt.h>
#include <avr/power.h>

enum DispenseState {
    DISPENSE_IDLE,
    DISPENSE_WATER_START,
    DISPENSE_WATER_HOLDING,
    DISPENSE_WATER_RETURN,
    DISPENSE_SNACK_START,
    DISPENSE_SNACK_HOLDING,
    DISPENSE_SNACK_RETURN
};

DispenseState currentState = DISPENSE_IDLE;
unsigned long actionStartTime = 0;

#define I2CADDR 0x20

#define SS_PIN 10
#define RST_PIN 9
```

```

RFID rfid(SS_PIN, RST_PIN);
unsigned long lastCardReadTime = 0;
const unsigned long DEBOUNCE_DELAY = 2000;

#define SERVO_PIN A0
#define SERVO_OPEN_ANGLE 90
#define SERVO_CLOSED_ANGLE 0
#define SERVO_STEP_DELAY 10
long AUTO_CLOSE_MS = 3000;

Servo myServo;

Servo leftSer;
Servo rightSer;
Servo PusherSer;

#define LEFT_SERVO_PIN 5
#define RIGHT_SERVO_PIN 4
#define PUSHER_SERVO_PIN 3

#define LEFT_INIT_ANGLE 0
#define RIGHT_INIT_ANGLE 90
#define LEFT_WATER_ANGLE 90
#define RIGHT_WATER_ANGLE 0
#define PUSHER_RETRACTED 180
#define PUSHER_PUSHED 0
#define ACTION_DELAY_MS 2000

bool doorOpened = false;
unsigned long doorOpenTime = 0;

```

```

const int buzzPin = 6;

#define RX_PIN2 8
#define TX_PIN2 7
SoftwareSerial mySerial(RX_PIN2, TX_PIN2);

String totalSnack;
String currentSnackWeight;

uint8_t vol;

unsigned long pressStartTime = 0;
bool isZeroPressed = false;
const unsigned long longPressThreshold = 3000;

const byte ROWS = 4;
const byte COLS = 4;

char hexaKeys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

byte rowPins[ROWS] = {7, 6, 5, 4};
byte colPins[COLS] = {3, 2, 1, 0};

LiquidCrystal_I2C lcd(0x27, 16, 2);

```

```

Keypad_I2C keypad( makeKeymap(hexaKeys), rowPins, colPins, ROWS, COLS, I2CADDR);

int serNum0, serNum1, serNum2, serNum3, serNum4;

volatile bool systemActive = false;

ISR(INT0_vect) {
    systemActive = true;
}

void handleKeypadInput();
void handleRFIDReading();
void handleSerialCommunication();
void handleAutoDoorClose();

void beep(unsigned int freq, unsigned int durMs, uint8_t volume = 255) {
    if (volume == 0) {
        digitalWrite(buzzPin, LOW);
        delay(durMs);
        return;
    }

    uint8_t duty = (volume > 255) ? 128 : (volume >> 1);

    unsigned long startTime = millis();
    unsigned long period = 1000000UL / freq;

    while (millis() - startTime < durMs) {
        digitalWrite(buzzPin, HIGH);
        delayMicroseconds(duty * (period / 128));
    }
}

```

```

    digitalWrite(buzzPin, LOW);
    delayMicroseconds((128 - duty) * (period / 128));
}

digitalWrite(buzzPin, LOW);
}

void openDoor() {
    Serial.println("Opening door...");
    for (int pos = SERVO_CLOSED_ANGLE; pos <= SERVO_OPEN_ANGLE; pos++) {
        myServo.write(pos);
        delay(SERVO_STEP_DELAY);
    }
    doorOpened = true;
    doorOpenTime = millis();
}

void closeDoor() {
    Serial.println("Closing door...");
    for (int pos = SERVO_OPEN_ANGLE; pos >= SERVO_CLOSED_ANGLE; pos--) {
        myServo.write(pos);
        delay(SERVO_STEP_DELAY);
    }
    doorOpened = false;
}

void handleKeypadInput() {
    char key = keypad.getKey();
    if (key != NO_KEY) {

```

```

    if (key == '0') {
        isZeroPressed = true;
        pressStartTime = millis();
        mySerial.print("KEY:");
        mySerial.println(key);
    } else {
        mySerial.print("KEY:");
        mySerial.println(key);
        isZeroPressed = false;
    }
} else {
    if (isZeroPressed) {
        unsigned long pressDuration = millis() - pressStartTime;
        if (pressDuration >= longPressThreshold) {
            mySerial.println("CLEAR_KEY");
            isZeroPressed = false;
        }
    }
}
}

void handleRFIDReading() {
    if (rfid.isCard() && rfid.readCardSerial()) {
        if (millis() - lastCardReadTime < DEBOUNCE_DELAY) {
            rfid.halt();
        } else {
            serNum0 = rfid.serNum[0];
            serNum1 = rfid.serNum[1];
            serNum2 = rfid.serNum[2];
            serNum3 = rfid.serNum[3];

```

```

serNum4 = rfid.serNum[4];

Serial.print("Hex: ");
Serial.print(serNum0, HEX); Serial.print(" ");
Serial.print(serNum1, HEX); Serial.print(" ");
Serial.print(serNum2, HEX); Serial.print(" ");
Serial.print(serNum3, HEX); Serial.print(" ");
Serial.println(serNum4, HEX);

String cardData = String(serNum0, HEX) + "-" +
                  String(serNum1, HEX) + "-" +
                  String(serNum2, HEX) + "-" +
                  String(serNum3, HEX) + "-" +
                  String(serNum4, HEX);

mySerial.println("CARD:" + cardData);
beep(3000, 500, vol);
lastCardReadTime = millis();
rfid.halt();
}
}
}

void handleSerialCommunication() {
  if (mySerial.available() > 0) {
    String msg = mySerial.readStringUntil('\n');
    msg.trim();
    Serial.println(msg);

    if (msg.startsWith("VOLUME:")) {

```

```

    vol = msg.substring(7).toInt();
}

if (msg == "SYS_RESET:") {
    mySerial.println("SYS_RESET_ACK");
    wdt_disable();
    for (int i = 5; i >= 1; i--) {
        beep(1000, 150);
        delay(1000);
    }
    wdt_enable(WDTO_15MS);
    while (1);
}

if (msg == "WATER001_200_OK") {
    Serial.println("I_GOT_WATER");
    if (currentState == DISPENSE_IDLE) {
        currentState = DISPENSE_WATER_START;
    }
} else if (msg == "SNACK001_200_OK") {
    if (currentState == DISPENSE_IDLE) {
        currentState = DISPENSE_SNACK_START;
    }
}

if (msg == "EXIT") {
    if (doorOpened) {
        closeDoor();
    }
    isZeroPressed = false;
}

```

```

    systemActive = false;
    Serial.println("ได้รับ EXIT → กลับสู่โหมด sleep");
}

if (msg == "OVERHEAT") {
    openDoor();
    AUTO_CLOSE_MS = 3000;
    doorOpenTime = millis();
}

if (msg.startsWith("TOTAL:")) {
    totalSnack = msg.substring(6);
} else if (msg.startsWith("WEIGHT:")) {
    currentSnackWeight = msg.substring(7);
}

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Snack: ");
lcd.print(totalSnack);
lcd.setCursor(0, 1);
lcd.print("Weight: ");
lcd.print(currentSnackWeight);
lcd.print(" g");
}
}

void handleDispensingStateMachine() {
    if (currentState == DISPENSE_IDLE) return;

```

```

unsigned long currentMillis = millis();

switch (currentState) {
  case DISPENSE_WATER_START:
    Serial.println("Water");
    leftSer.write(LEFT_WATER_ANGLE);
    rightSer.write(RIGHT_WATER_ANGLE);
    PusherSer.write(PUSHER_PUSHED);
    actionStartTime = currentMillis;
    currentState = DISPENSE_WATER_HOLDING;
    break;

  case DISPENSE_WATER_HOLDING:
    if (currentMillis - actionStartTime >= ACTION_DELAY_MS) {
      Serial.println("Returning servos...");
      leftSer.write(LEFT_INIT_ANGLE);
      rightSer.write(RIGHT_INIT_ANGLE);
      PusherSer.write(PUSHER_RETRACTED);
      actionStartTime = currentMillis;
      currentState = DISPENSE_WATER_RETURN;
    }
    break;

  case DISPENSE_WATER_RETURN:
    if (currentMillis - actionStartTime >= 500) {
      Serial.println("Done.");
      currentState = DISPENSE_IDLE;
    }
    break;

```

```

case DISPENSE_SNACK_START:
    Serial.println("Snack");
    PusherSer.write(PUSHER_PUSHED);
    actionStartTime = currentMillis;
    currentState = DISPENSE_SNACK_HOLDING;
    break;

case DISPENSE_SNACK_HOLDING:
    if (currentMillis - actionStartTime >= ACTION_DELAY_MS) {
        PusherSer.write(PUSHER_RETRACTED);
        actionStartTime = currentMillis;
        currentState = DISPENSE_SNACK_RETURN;
    }
    break;

case DISPENSE_SNACK_RETURN:
    if (currentMillis - actionStartTime >= 500) {
        currentState = DISPENSE_IDLE;
    }
    break;

default:
    currentState = DISPENSE_IDLE;
    break;
}
}

void handleAutoDoorClose() {
    if (systemActive && doorOpened && (millis() - doorOpenTime >= AUTO_CLOSE_MS)) {
        closeDoor();
    }
}

```

```

    }
}

void enterSleep() {
    power_adc_disable();
    power_spi_disable();
    power_timer0_disable();
    power_timer1_disable();
    power_timer2_disable();
    power_twi_disable();

    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    sleep_enable();

    EIFR |= (1 << INTF0);
    EIMSK |= (1 << INT0);

    sei();

    sleep_cpu();

    sleep_disable();

    power_all_enable();
}

void setup() {
    Serial.begin(9600);
    pinMode(2, INPUT_PULLUP);
}

```

```

EICRA = 0;
EIMSK |= (1 << INTO);
sei();

leftSer.attach(LEFT_SERVO_PIN);
rightSer.attach(RIGHT_SERVO_PIN);
PusherSer.attach(PUSHER_SERVO_PIN);

leftSer.attach(LEFT_SERVO_PIN);
rightSer.attach(RIGHT_SERVO_PIN);
PusherSer.attach(PUSHER_SERVO_PIN);

leftSer.write(LEFT_INIT_ANGLE);
rightSer.write(RIGHT_INIT_ANGLE);
PusherSer.write(PUSHER_RETRACTED);

SPI.begin();
rfid.init();
Wire.begin();
keypad.begin();
lcd.init();
lcd.backlight();
myServo.attach(SERVO_PIN);
myServo.write(SERVO_CLOSED_ANGLE);
pinMode(buzzPin, OUTPUT);
digitalWrite(buzzPin, LOW);
mySerial.begin(9600);

Serial.println("ระบบเริ่มต้น... รอสัญญาณปลุกที่ขา D2");
}

```

```

void loop() {
  wdt_reset();

  if (!systemActive) {
    enterSleep();
    if (!systemActive) return;
  }

  handleKeypadInput();
  handleRFIDReading();
  handleSerialCommunication();
  handleAutoDoorClose();
  handleDispensingStateMachine();

  delay(20);
}

```

หมายเหตุ

- 2C Bus (SDA = A4, SCL = A5): ใช้โดยอัตโนมัติสำหรับ LCD (0x27) และ Keypad I2C (0x20) → ไม่ต้องระบุพินในโค้ด
- SPI Bus (MOSI=11, MISO=12, SCK=13): ใช้กับ RFID module → ถูกเรียกผ่านไลบรารี SPI.h

ตารางนี้รวมเฉพาะ พินที่ต่ออุปกรณ์จริง เท่านั้น (ไม่รวมพินเสมือน เช่น A4/A5 ที่ใช้ I2C โดยอัตโนมัติ)

3.4 โปรแกรมและการทำงานของ ESP32

3.4.1 หน้าที่หลัก

เชื่อมต่อกับ WiFi, ส่งข้อมูลไป Server, ควบคุม LED แจ้งผล

3.4.2 เซนเซอร์และอุปกรณ์

1. LED_SUCCESS / LED_FAIL
2. Potentiometer: ปรับ Volume ส่งไป Back UNO
3. SoftwareSerial: รับส่งข้อมูลกับ Front/Back UNO

3.4.3 การทำงานหลัก

- 1) ใช้ WiFi + HTTPClient ส่งข้อมูลไป Server
- 2) รองรับ Retry สำหรับ HTTP request
- 3) ใช้ Deep Sleep + Wake Pin ประหยัดพลังงาน
- 4) ควบคุม LED แสดงผลสำเร็จ/ล้มเหลว

3.4.4 ขาและการใช้งาน (PIN Mapping) เนื่องจากมี WiFi และ UART หลายพอร์ตโดยกำหนด PIN ดังนี้:

ขา (PIN)	ต่อกับอุปกรณ์ใด	หน้าที่	UART ที่ใช้
12	LED แสดงสถานะ ล้มเหลว (LED_FAIL)	แสดงเมื่อเกิดข้อผิดพลาด (เช่น ส่งข้อมูลไม่ได้)	—
13	LED แสดงสถานะ สำเร็จ (LED_SUCCESS)	แสดงเมื่อส่งข้อมูล/สแกนสินค้าสำเร็จ	—
16	RX ของแผงควบคุม (Keypad/Display)	รับคำสั่งจากผู้ใช้ (รหัส, บัตร, ลบ)	Serial2
17	TX ของแผงควบคุม	ส่งข้อมูลกลับ (เช่น ยืนยัน, ปรับเสียง)	Serial2
18	Servo Motor	ควบคุมการเปิด/ปิดประตู	—
21	RX ของชั้นวาง (Arduino)	รับข้อมูลน้ำหนัก, สถานะประตู, คำขอรีเซ็ต	Serial1
22	TX ของชั้นวาง	ส่งข้อมูลแสดงผล (น้ำหนัก, จำนวนสินค้า)	Serial1
23	RX ของ Odroid (กล้อง AI)	รับรหัสสินค้าที่ตรวจจับได้จากกล้อง	Serial3
32	ปุ่มปลุก (WAKE_PIN)	ใช้ปลุก ESP32 จาก Deep Sleep	—

หมายเหตุ

- 1) Serial3 ใช้ `rxPin3 = 23` และ `txPin3 = -1` → รับอย่างเดียว (RX only) จาก Odroid
- 2) PIN 34 เป็น ADC เท่านั้น → ไม่สามารถใช้เป็น output ได้ (ตามข้อจำกัดของ ESP32)
- 3) ระบบใช้ Deep Sleep + External Wakeup เพื่อประหยัดพลังงานเมื่อไม่มีผู้ใช้งาน

3.4.5 โปรแกรม

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <SoftwareSerial.h>
#include "esp_sleep.h"
#include <ESP32Servo.h>

#define LED_SUCCESS 13
#define LED_FAIL 12

const char* ssid = "realme";
const char* password = "12345678";

#define WAKE_PIN 32
RTC_DATA_ATTR bool systemActive = false;

String serverBase = "http://172.20.10.3:8080";
String apiKeyMCU = "mcu_0d1a8e9f4c7b2e1d8a3c7f5d9b4e0f2a";
String apiKeyAI = "ai_7f8b2c9d6a1e3b5c4d2f0e8a9c7d3b1a";

String sessionId = "";

int rxPin1 = 21;
int txPin1 = 22;
int rxPin2 = 16;
int txPin2 = 17;
int rxPin3 = 23;
int txPin3 = -1;

SoftwareSerial Serial3(rxPin3, txPin3);
```

```

Servo myServo;
int servoPin = 18;

struct ShelfData {
    int totalSnack;
    int totalWater;
    int snackWeight;
    int waterWeight;
};

int totalSnack;
int totalWater;

const int potPin = 34;
int volume;

unsigned long ledTimer = 0;
bool ledActive = false;
int activeLedPin = -1;
const unsigned long LED_DURATION = 500;

bool safeHttpRequest(String method, String url, String body, String apiKey, String&
responseOut) {
    const int MAX_RETRIES = 3;
    int delayMs = 1000;

    for (int attempt = 1; attempt <= MAX_RETRIES; attempt++) {
        HTTPClient http;
        http.setTimeout(20000);
    }
}

```

```

http.begin(url);

http.addHeader("x-api-key", apiKey);
http.addHeader("Content-Type", "application/json");

int httpCode = -1;
if (method == "GET") {
    httpCode = http.GET();
} else if (method == "POST") {
    httpCode = http.POST(body);
} else if (method == "PATCH") {
    httpCode = http.PATCH(body);
} else if (method == "DELETE") {
    httpCode = http.sendRequest("DELETE", body);
}

Serial.printf("🔍 Attempt %d → httpCode=%d\n", attempt, httpCode);

if (httpCode > 0) {
    if (httpCode >= 200 && httpCode < 300) {
        responseOut = http.getString();
        http.end();
        return true;
    } else {
        Serial.printf("❌ HTTP %d, attempt %d\n", httpCode, attempt);
        responseOut = http.getString();
    }
} else {
    Serial.printf("❌ HTTP request failed, attempt %d, error: %s\n",
        attempt, http.errorToString(httpCode).c_str());
}

```

```

    }

    http.end();
    if (attempt < MAX_RETRIES) {
        delay(delayMs);
        delayMs *= 2;
    }
}

return false;
}

void setup() {
    Serial.begin(9600);
    Serial1.begin(9600, SERIAL_8N1, rxPin1, txPin1);
    Serial2.begin(9600, SERIAL_8N1, rxPin2, txPin2);
    Serial3.begin(9600);

    myServo.attach(servoPin);

    for (int pos = 0; pos <= 90; pos++) {
        myServo.write(pos);
        delay(15);
    }

    delay(500);

    for (int pos = 90; pos >= 0; pos--) {
        myServo.write(pos);
        delay(15);
    }
}

```

```

esp_sleep_wakeup_cause_t wakeup_reason = esp_sleep_get_wakeup_cause();
if (wakeup_reason == ESP_SLEEP_WAKEUP_EXT0) {
    Serial.println("💡 ตื่นจาก EXT0 interrupt");
    systemActive = true;
}

if (!systemActive) {
    Serial.println("zz Sleep mode, รอปลุกจาก GPIO32...");
    pinMode(WAKE_PIN, INPUT_PULLUP);
    esp_sleep_enable_ext0_wakeup((gpio_num_t)WAKE_PIN, 0);
    delay(100);
    esp_deep_sleep_start();
}

Serial.println("Connecting to WiFi...");
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("\n✅ WiFi connected!");

pinMode(LED_SUCCESS, OUTPUT);
pinMode(LED_FAIL, OUTPUT);
digitalWrite(LED_SUCCESS, LOW);
digitalWrite(LED_FAIL, LOW);

ShelveData totals = getShelveData();
totalSnack = totals.totalSnack;

```

```

totalWater = totals.totalWater;
updateDisplay(totals);
}

void loop() {
  int potValue = analogRead(potPin);
  int volumeValue = map(potValue, 0, 4095, 0, 255);

  if (abs(volume - volumeValue) > 20) {
    volume = volumeValue;
    Serial2.print("VOLUME:");
    Serial2.println(volume);
  }

  if (ledActive && (millis() - ledTimer >= LED_DURATION)) {
    digitalWrite(LED_SUCCESS, LOW);
    digitalWrite(LED_FAIL, LOW);
    ledActive = false;
    activeLedPin = -1;
  }

  if (Serial1.available()) {
    String msg = Serial1.readStringUntil('\n');
    msg.trim();
    Serial.println(msg);

    if (msg.equals("DOOR_OPENED")) {
      if (sessionId.isEmpty()) {
        Serial.println("  Door opened! Creating new session...");
        createSession();
      }
    }
  }
}

```

```

    } else {
        Serial.println("  Ignoring DOOR_OPENED: session already active");
    }
}

else if (msg.startsWith("SNACK_WEIGHT:")) {
    int value = msg.substring(13).toInt();
    if (updateShelveWeight("snack", value, 0)) {
        ShelveData totals = getShelveData();
        updateDisplay(totals);
    }
}

else if (msg.startsWith("WATER_WEIGHT:")) {
    int value = msg.substring(13).toInt();
    if (updateShelveWeight("water", value, 0)) {
        ShelveData totals = getShelveData();
        updateDisplay(totals);
    }
}

else if (msg.startsWith("SYS_RESET:")) {
    Serial2.println("SYS_RESET:");
}

else if (msg.startsWith("SNACK_RESET:")) {
    int value = msg.substring(12).toInt();
    if (updateShelveWeight("snack", value, 0)) {
        ShelveData totals = getShelveData();
    }
}

else if (msg.startsWith("WATER_RESET:")) {
    int value = msg.substring(12).toInt();

```

```

    if (updateShelveWeight("water", value, 0)) {
        ShelveData totals = getShelveData();
    }
}

if (Serial2.available()) {
    String msg = Serial2.readStringUntil('\n');
    msg.trim();
    Serial.println(msg);

    if (msg == "CLEAR_KEY") codeDelete();
    if (msg.startsWith("KEY:")) codePressed(msg.substring(4));
    if (msg.startsWith("PRODUCT:") && sessionId != "") sendItem(msg.substring(8));
    if (msg.startsWith("CARD:") && sessionId != "") payWithCard(msg.substring(5));

    if (msg == "SYS_RESET_ACK") {
        Serial.println("🔄 ได้รับ ACK จากชั้นวาง — กำลังรีเซ็ตระบบหลัก...");

        delay(8000);

        ESP.restart();
    }
}

if (Serial3.available()) {
    String data = Serial3.readString();
    data.trim();

    Serial.println("📡 จาก Odriod: " + data);
    sendItem(data);
}

```

```

    }
}

ShelveData getShelveData() {
    ShelveData data = {0,0,0,0};
    String response;

    if (safeHttpRequest("GET", serverBase + "/shelves/lcd", "", apiKeyMCU, response)) {
        Serial.println("✅ Got shelves: " + response);

        int start = response.indexOf("\"totalSnack\":") + 13;
        int end = response.indexOf(",", start);
        data.totalSnack = response.substring(start, end).toInt();

        start = response.indexOf("\"totalWater\":") + 13;
        end = response.indexOf(",", start);
        data.totalWater = response.substring(start, end).toInt();

        start = response.indexOf("\"snackWeight\":") + 14;
        end = response.indexOf(",", start);
        data.snackWeight = response.substring(start, end).toInt();

        start = response.indexOf("\"waterWeight\":") + 14;
        end = response.indexOf("}", start);
        data.waterWeight = response.substring(start, end).toInt();
    } else {
        Serial.println("❌ Failed to fetch shelves");
    }
    return data;
}

```

```

bool updateShelveWeight(String type, int value, bool isReset) {
    if (WiFi.status() != WL_CONNECTED) return false;

    String shelfId = (type == "snack") ? "68c6cf1c1ddf12f0b4891721" :
        (type == "water") ? "68c6cf1c1ddf12f0b4891723" : "";

    if (shelfId == "") return false;

    String url = serverBase + "/shelves/" + shelfId;
    String jsonBody = isReset ? "{\"baselineWeightGram\":" + String(value) + "}" :
        "{\"currentWeightGram\":" + String(value) + "};";
    String response;
    return safeHttpRequest("PATCH", url, jsonBody, apiKeyAl, response);
}

void updateDisplay(ShelveData totals) {
    Serial1.print("TOTAL_SNACK:");
    Serial1.println(totals.totalSnack);
    Serial1.print("WEIGHT_SNACK:");
    Serial1.println(totals.snackWeight);

    Serial1.print("TOTAL_WATER:");
    Serial1.println(totals.totalWater);
    Serial1.print("WEIGHT_WATER:");
    Serial1.println(totals.waterWeight);

    totalSnack = totals.totalSnack;
    totalWater = totals.totalWater;

    Serial1.printf("📊 Display updated - Snack: %d, Water: %d\n", totalSnack, totalWater);
}

```

```

}

void createSession() {
    if (WiFi.status() != WL_CONNECTED) return;
    sessionId = "";

    String response;
    if (safeHttpRequest("POST", serverBase + "/sessions/enter", "{}", apiKeyMCU, response))
    {
        int idStart = response.indexOf("\"sessionId\":");
        if (idStart != -1) {
            idStart += 13;
            int idEnd = response.indexOf("\"", idStart);
            sessionId = response.substring(idStart, idEnd);

            Serial.println("🔑 Session: " + sessionId);
        }
    }
}

void codePressed(String key) {
    if (WiFi.status() != WL_CONNECTED || sessionId == "") return;
    key.trim();
    String jsonBody = "{\"key\": \"" + key + "\"}";
    String response;
    safeHttpRequest("PATCH", serverBase + "/sessions/" + sessionId + "/press", jsonBody,
    apiKeyMCU, response);
}

void codeDelete() {
    if (WiFi.status() != WL_CONNECTED || sessionId == "") return;

```

```

String response;
safeHttpRequest("DELETE", serverBase + "/sessions/" + sessionId + "/press", "",
apiKeyMCU, response);
}

void payWithCard(String cardId) {
if (WiFi.status() != WL_CONNECTED || sessionId == "") return;
cardId.trim();
String jsonBody = "{\"code\":\"" + cardId + "\"}";
String response;
if (safeHttpRequest("POST", serverBase + "/sessions/" + sessionId + "/pay/card",
jsonBody, apiKeyAI, response)) {
for (int pos = 0; pos <= 90; pos++) {
myServo.write(pos);
delay(15);
}

delay(500);

for (int pos = 90; pos >= 0; pos--) {
myServo.write(pos);
delay(15);
}
Serial2.println("EXIT");
Serial.println(" 🚪 EXIT detected → กลับไปหลับ");
systemActive = false;

pinMode(WAKE_PIN, INPUT_PULLUP);
esp_sleep_enable_ext0_wakeup((gpio_num_t)WAKE_PIN, 0);
delay(200);
}

```

```

    esp_deep_sleep_start();
}
}

void sendItem(String itemStr) {
    if (WiFi.status() != WL_CONNECTED || sessionId == "") {
        triggerLed(LED_FAIL);
        return;
    }

    itemStr.trim();
    if (itemStr == "") return;

    String jsonBody = "{\"productCode\":\"" + itemStr + "\"}";
    String response;

    bool success = safeHttpRequest("PATCH", serverBase + "/sessions/" + sessionId +
    "/camera", jsonBody, apiKeyAI, response);

    if (success) {
        triggerLed(LED_SUCCESS);
        Serial2.print(itemStr);
        Serial2.println("_200_OK");
    } else {
        triggerLed(LED_FAIL);
    }
}

void triggerLed(int pin) {
    digitalWrite(LED_SUCCESS, LOW);

```

```
digitalWrite(LED_FAIL, LOW);  
  
digitalWrite(pin, HIGH);  
  
activeLedPin = pin;  
ledActive = true;  
ledTimer = millis();  
}
```

บทที่ 4

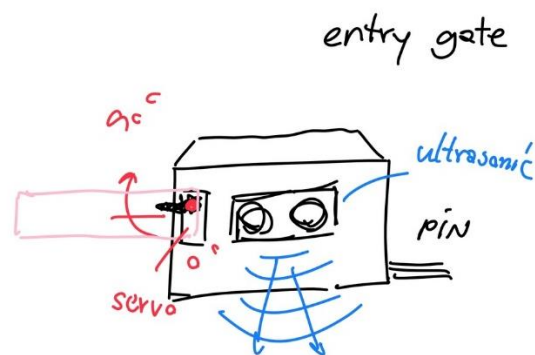
ชิ้นงานและการทำงาน

4.1 ชิ้นงาน

ชิ้นงานส่วนใหญ่มีการประดิษฐ์ขึ้นเองจากวัสดุที่สามารถพบหาได้ โดยได้ทำการประกอบและจัดวางไว้ 9 ส่วนด้วยกัน ดังนี้

4.1.1 Gate Entry

การออกแบบ

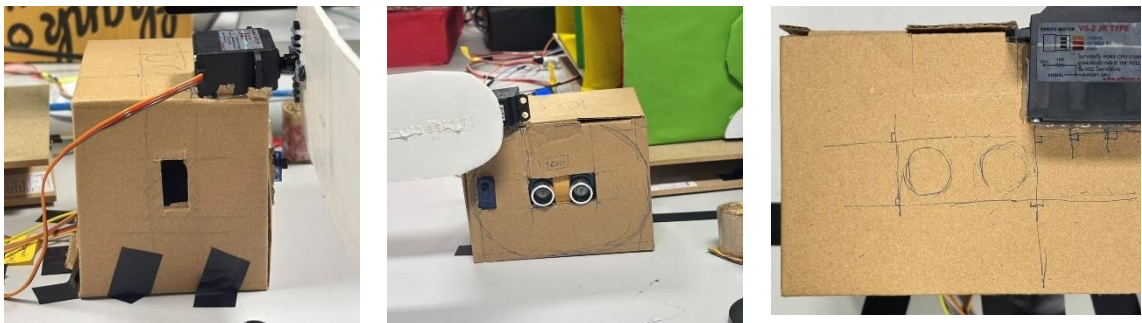


ภาพที่ 27 Gate Design

การทำงาน

- 1) กลไกการเปิด-ปิดโดยใช้มอเตอร์/เซอร์โวมอเตอร์
- 2) ติดตั้งเซนเซอร์ตรวจจับการเข้าออก (Ultrasonic)
- 3) ระบบเชื่อมต่อกับบอร์ดควบคุมเพื่อปลดล็อกเมื่อตรวจพบลูกค้า

ชิ้นงานจริง



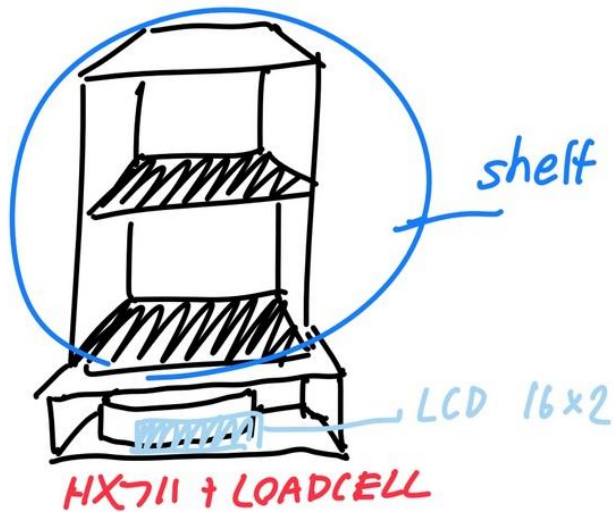
ภาพที่ 28 Gate Entry ชิ้นงานจริง

การทดสอบ

Ultrasonic และ Micro Servo สามารถทำงานร่วมกันได้ แต่ระบบเปิด / ปิด ประตูหากตรวจพบเจอลูกค้าสามารถใช้งานได้

4.1.2 Product Shelf

การออกแบบ



ภาพที่ 29 Shelf Design

การทำงาน

- 1) ชั้นวางติดตั้ง Load Cell เพื่อวัดน้ำหนักสินค้า
- 2) วางตำแหน่งให้สอดคล้องกับการตรวจจับสินค้าโดย AI
- 3) เชื่อมต่อ MCU ส่งข้อมูลเข้าฐานข้อมูลแบบ Real-time

ชิ้นงานจริง



ภาพที่ 30 Shelf ชิ้นงานจริง

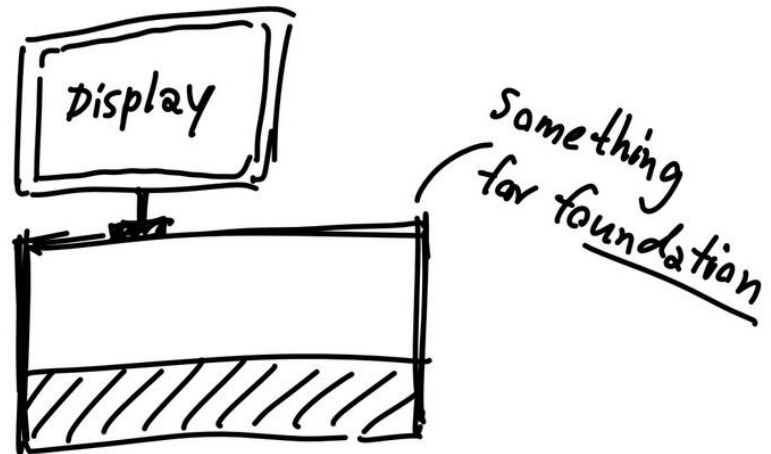
การทดสอบ

การทดสอบครั้งแรก พบเจอปัญหาเนื่องจากฐานรองรับชั้นวางไม่มั่นคง ขาดสมดุล ซึ่งทำให้ Load Cell ทำงานได้ไม่เต็มประสิทธิภาพทั้งหมด และไม่มีเสถียรภาพ

ต่อมาได้มีการปรับแก้ไข โครงสร้างฐานรองรับให้มีความแข็งแรง สมดุลขึ้นทำให้ทดสอบแล้วสามารถวัดค่าได้ถูกต้องและเสถียรมากขึ้น

4.1.3 Display Screen

การออกแบบ

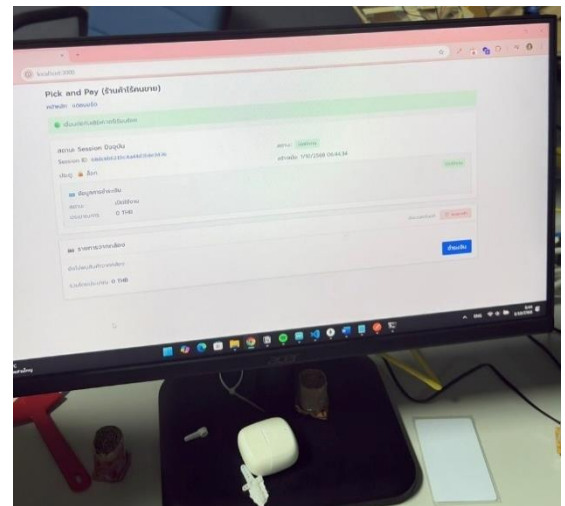
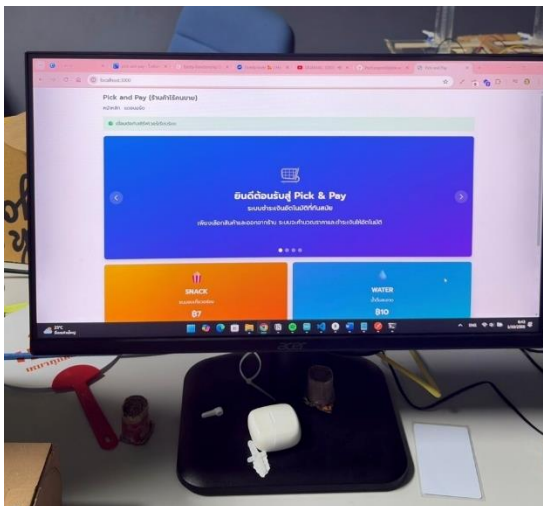


ภาพที่ 31 Display Set up

การทำงาน

- 1) รองรับการทำงานเชื่อมต่อกับฐานข้อมูลผ่าน Ordroid
- 2) UI แสดงรายการปัจจุบัน และ QR Code สำหรับชำระเงิน

ชั้นงานจริง



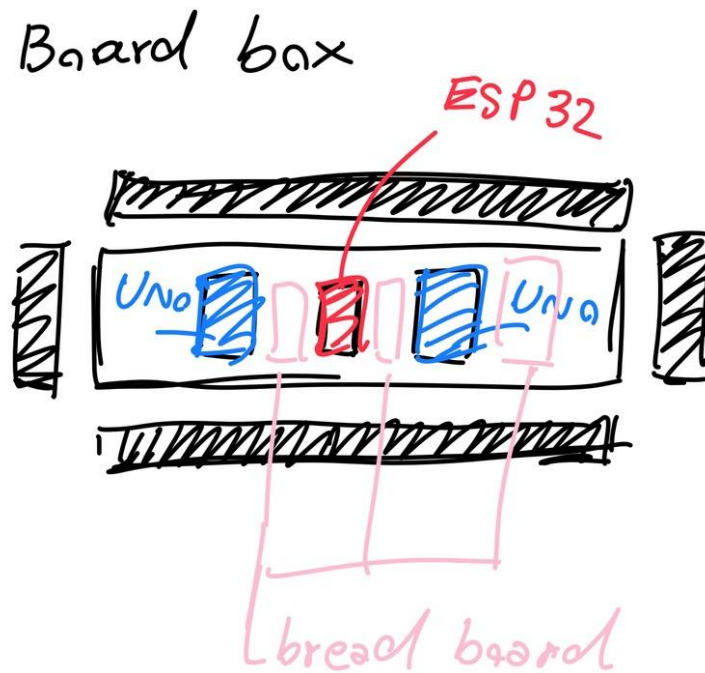
ภาพที่ 32 การจัดวางหน้าจอจริง

การทดสอบ

หน้าจอสามารถเชื่อมต่อกับ ODROID เพื่อรัน Ubuntu ได้อย่างราบรื่นไม่พบเจอความติดขัด

4.1.4 Combination Board Box

การออกแบบ

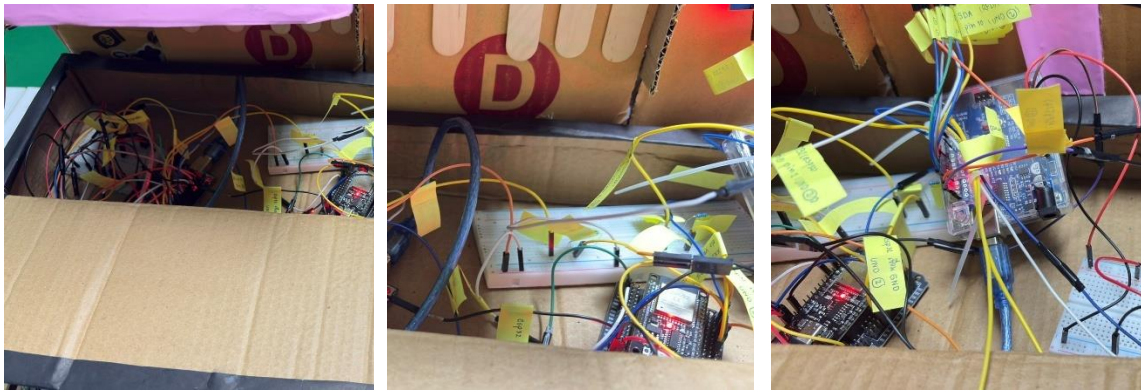


ภาพที่ 33 Combination Board Box Design

การทำงาน

- 1) กล่องรวมวงจร (UNO, ESP32, Bread Board)
- 2) จัดวางสายไฟและอุปกรณ์ให้เป็นระเบียบ
- 3) ออกแบบเพื่อความปลอดภัยและบำรุงรักษาง่าย

ชิ้นงานจริง



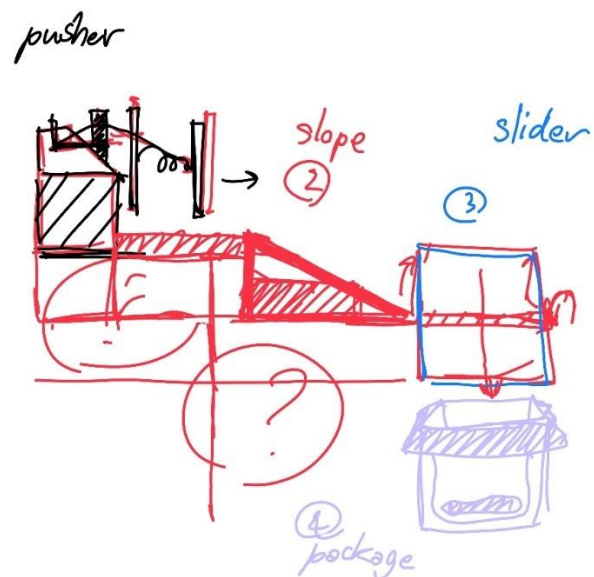
ภาพที่ 34 Combination Board Box ชิ้นงานจริง

การทดสอบ

จากการทดสอบใช้งานพบว่า กล่องรวมบอร์ดนี้สามารถช่วยให้ระบบดูเป็นระเบียบ และง่ายต่อการจัดการได้จริงตามเป้าประสงค์

4.1.5 Pusher

การออกแบบ



ภาพที่ 35 Pusher and Platform Design

การทำงาน

- 1) กลไกผลักสินค้าให้เลื่อนไปด้านหน้า
- 2) ใช้เซอร์โวมอเตอร์ในการประยุกต์การทำงาน
- 3) ช่วยผลักสินค้าไปยังส่วนต่อไป

ชิ้นงานจริง



ภาพที่ 36 Pusher and Platform ชิ้นงานจริง

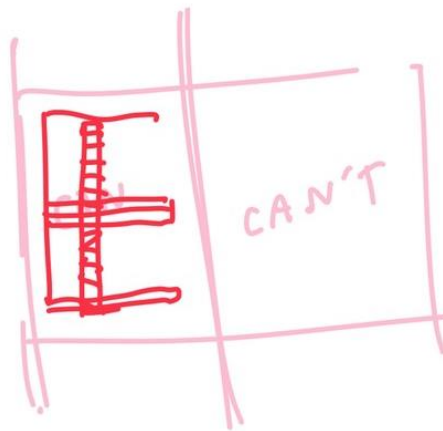
การทดสอบ

การทดสอบครั้งแรก เนื่องจากต้นแบบเดิมแขนผลักมีความยาวเกินไปทำให้แรงผลักที่ส่งผ่านมาจากเซอร์โวมอเตอร์ลดลง ทำให้ไม่สามารถตอบโจทย์การทำงานได้ จึงมีการนำไปออกแบบใหม่เพื่อแก้ไขโครงสร้าง หลังทำงานแก้ไขและปรับแต่งโครงสร้างพบว่า สามารถนำแรงบิดของเซอร์โวมอเตอร์ใช้งานได้มากขึ้น

4.1.6 Slider

การออกแบบ

slider * 1

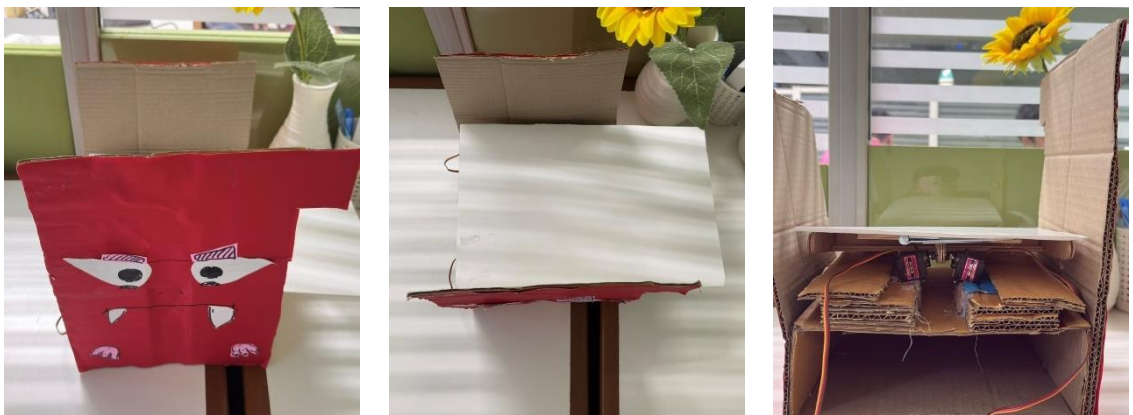


ภาพที่ 37 Slider and Frame Design

การทำงาน

- 1) กลไกที่แยก ขนม และ ขวดน้ำ จากกัน
- 2) ใช้เซอร์โวมอเตอร์ในการประยุกต์การทำงาน เพื่อดันแผ่นโฟมขึ้นให้เป็นทางลาด

ชิ้นงานจริง



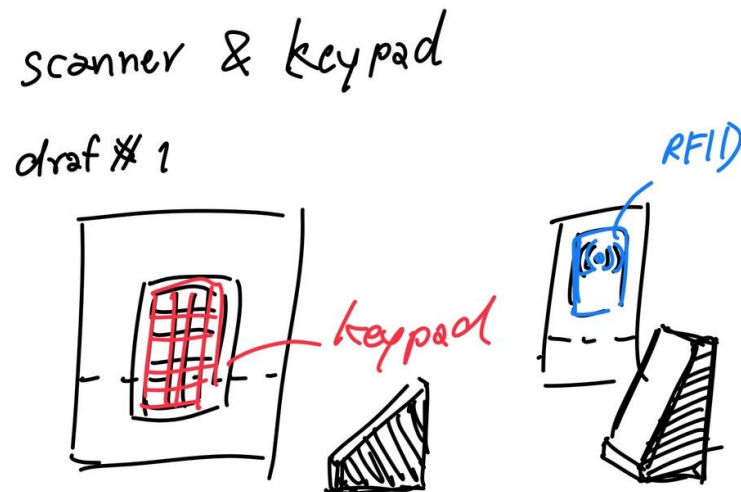
ภาพที่ 38 รูปภาพ Slider จริง

การทดสอบ

จากการทดสอบใช้งาน พบว่าจำเป็นต้องแก้ไขโครงสร้างและ เพิ่มท่อน้ำหักในส่วนปลายให้ยกง่าย ขึ้นเนื่องจากไฟที่กระชากของเซอร์โมเตอร์ไม่เพียงพอเมื่อทำงานร่วมกับระบบอื่นๆ

4.1.7 Card Scanner and Keypad

การออกแบบ



ภาพที่ 39 Card Scanner (RFID) and Keypad Design

การทำงาน

- 1) เครื่องสแกนบัตรสำหรับ Credit Card
- 2) Keypad ใช้กรอก Credit Card ID

ชิ้นงานจริง



ภาพที่ 40 Card Scanner (RFID) and Keypad ชิ้นงานจริง

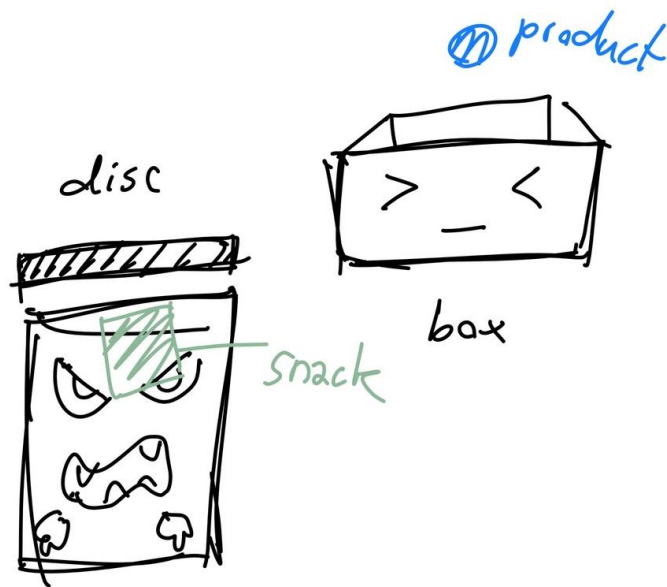
การทดสอบ

การทดสอบช่วงแรกพบว่า RFID ไม่สามารถสแกนได้ ซึ่งอาจจะเกิดได้หลายสาเหตุจึงทำการแก้ไข และค้นหาสาเหตุที่เกิดขึ้น ผลการวิเคราะห์พบว่า สาเหตุหลักเกิดจากความยาวของสายสัญญาณที่ต่อกับ โมดูล RC522 มากเกินไป ซึ่งทำให้สัญญาณ SPI ลดประสิทธิภาพ และเกิดความผิดพลาดในการสื่อสาร ระหว่าง Arduino และ RC522

เพื่อแก้ไขปัญหานี้ จึงได้ทำการ ลดความยาวสายสัญญาณให้น้อยที่สุด และปรับตำแหน่งการวางโมดูลให้อยู่ใกล้กับบอร์ด Arduino มากขึ้น ผลลัพธ์หลังการปรับปรุงพบว่า RFID สามารถอ่านบัตรได้อย่างถูกต้อง และเสถียร ทำให้ระบบสามารถทำงานตามฟังก์ชันที่ออกแบบไว้ได้ครบถ้วน

4.1.8 Product Receiving Box / Disc

การออกแบบ



ภาพที่ 41 Product Receiving Box Design

การทำงาน

- 1) พื้นที่รับสินค้าหลังจากเลือกซื้อ

2) ป้องกันการกระแทกหรือความเสียหายของสินค้า

ชิ้นงานจริง



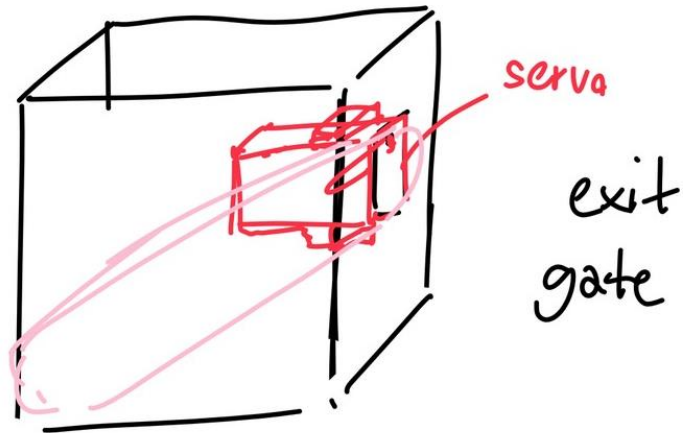
ภาพที่ 42 Product Receiving Box ชิ้นงานจริง

การทดสอบ

จากการทดสอบได้มีการรองใช้งานรับสินค้าที่ตกลงมาจาก Slider พบว่าสามารถรองรับได้ แต่เนื่องจากความสูง และน้ำหนักทำให้สินค้าตกมาแรงไป ดังนั้นจึงทำการเพิ่ม Bubble กันกระแทกในการลดแรงที่ตกลงมา

4.1.9 Gate Exit

การออกแบบ



ภาพที่ 43 Gate Exit Design

การทำงาน

- 1) กลไกเปิด-ปิดเมื่อชำระเงินสำเร็จ โดยใช้เซอร์โวมอเตอร์
- 2) ป้องกันการออกโดยไม่ได้ชำระเงิน

ชิ้นงานจริง



ภาพที่ 44 Gate Exit ชิ้นงานจริง

การทดสอบ

สามารถรับ Response API เมื่อทำการชำระเงินสำเร็จแล้วสั่งการเปิดประตูได้ตามเป้าหมาย

4.2 ประสิทธิภาพ

4.2.1 Gate Entry

จากการทดสอบใช้งานจริงพบว่า หากไม่สามารถยัดเซอร์โวมอเตอร์ให้ติดกับฐานได้แน่นพอจะทำให้ไม่สามารถยกประตูขึ้นได้อย่างเต็มที่ แต่เมื่อผ่านการทดสอบและปรับแต่งการยัดทำให้สามารถใช้ได้จริง ในส่วนของ Ultrasonic เองก็สามารถวัดระยะแต่ตรวจสอบการเข้ามาของลูกค้าส่งข้อมูลไปยัง Server ได้ตามเป้าหมาย สามารถประเมินประสิทธิภาพได้ร้อยละ 100

4.2.2 Product Shelf

จากการทดสอบใช้งานจริงพบว่า ตัว Load Cell นั้น Sensitive ต่อแรงภายนอกง่าย และหากทำฐานไม่มั่นคงพอจะทำให้ค่ามีความคลาดเคลื่อนสูง แต่หลังจากการปรับแต่งแล้วพบว่ามีความคลาดเคลื่อนอยู่ในเกณฑ์ยอมรับได้และสามารถทำงานได้จริงตามเป้าหมาย สามารถประเมินประสิทธิภาพได้ร้อยละ 95

4.2.3 Display Screen

จากการทดสอบใช้งานจริงพบว่าหน้าจอสามารถเชื่อมต่อ กับ Odriod เพื่อเปิดหน้าจอร้านค้าได้อย่างราบรื่นตามเป้าหมาย สามารถประเมินประสิทธิภาพได้ร้อยละ 100

4.2.4 Combination Board Box

จากการทดสอบจริงพบว่ากล่องสามารถใส่บอร์ดการทำงาน และเซนเซอร์ต่างๆ รวมถึงสายไฟได้จึง อีกทั้งสามารถทำให้จัดการแก้ไข หรือซ่อมแซมได้อย่างง่ายได้มากขึ้น และเมื่อเพิ่มต่อมการติดป้ายระบุสายการทำงาน และจัดเรียงสายไฟแล้วทำให้ทำงานได้อย่างราบรื่นและง่ายมากขึ้นจริงตามเป้าหมาย สามารถประเมินประสิทธิภาพได้ร้อยละ 100

4.2.5 Pusher

จากการทดสอบใช้งานจริงพบว่า Pusher เมื่อเจอวัตถุที่มีน้ำหนักมาก จะมีแรงไม่เพียงพอต่อการดันหรือผลักวัตถุนั้น เนื่องจากแรงบิดเซอร์โวมอเตอร์ไม่เพียงพอ แต่หลังจากการปรับแต่ง เปลี่ยนเซอร์โวมอเตอร์จากแกนพลาสติกเป็นแกนเหล็ก แล้วพบว่าสามารถดันวัตถุมีน้ำหนักได้มากขึ้น แต่หากไม่จัดวางในรูปแบบที่ช่วยผ่อนแรงจะดันไม่ได้ และจำเป็นต้องมีการเพิ่มตัวหนุนน้ำหนักของแผ่นวางให้เกิดมุมเงยเพื่อช่วยส่งให้วัตถุไปได้ไกลยิ่งขึ้น สามารถประเมินประสิทธิภาพได้ร้อยละ 75

4.2.6 Slider

จากการทดสอบใช้งานจริงพบว่า Slider เมื่อเจอวัตถุที่มีน้ำหนักมาก จะมีแรงไม่เพียงพอต่อการยกแผ่นโฟม พร้อมกับวัตถุได้ เนื่องจากแรงบิดเซอร์โวไม่เพียงพอ แต่หลังจากการปรับแต่ง เพิ่มจำนวนเซอร์โวมอเตอร์และปรับโครงไม้ภายใน แล้วพบว่าหากจำทำให้สามารถยกวัตถุมีน้ำหนักได้จำเป็นต้องมีการเพิ่มตัวหนุนน้ำหนักของแผ่นวางให้เกิดมุมเงยเพื่อช่วยยกแผ่นโฟมได้ง่ายขึ้น สามารถประเมินประสิทธิภาพได้ร้อยละ 85

4.2.7 Card Scanner and Keypad

จากการทดสอบใช้งานจริงพบว่า Card Scanner and Keypad หากทำการต่อสายเยอะ และยาวเกินไป จะทำให้การทำงานเกิดความขัดข้อง แต่หลังจากการปรับแต่งลดระยะสายไฟให้สั้นลง สามารถประเมินประสิทธิภาพได้ร้อยละ 80

4.2.8 Product Receiving Box / Disc

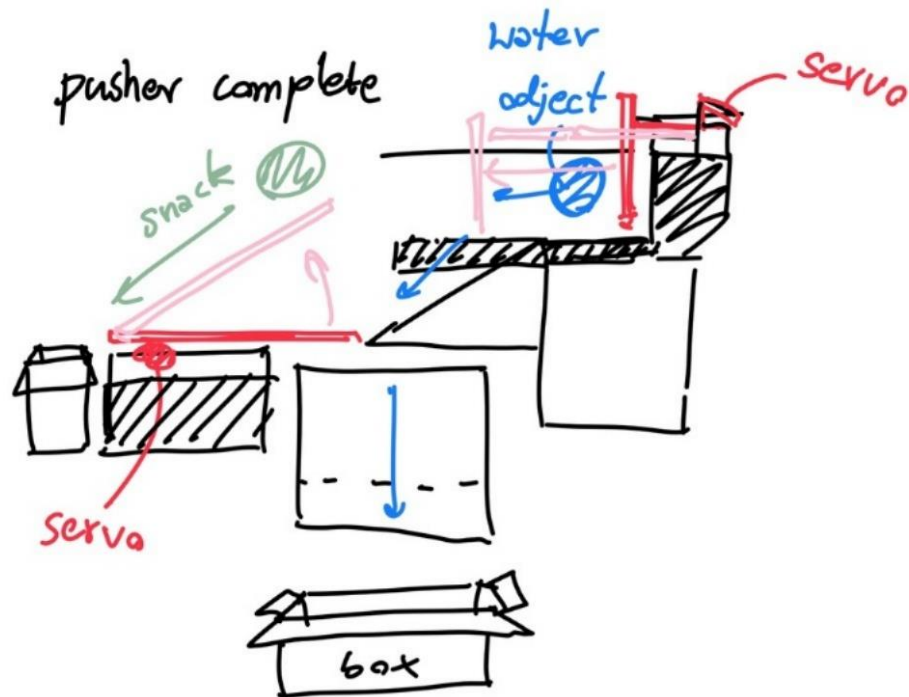
จากการทดสอบใช้งานจริงพบว่า กล่องและถาดสามารถรองรับแรงกระแทกของวัตถุได้ตามเป้าหมาย สามารถประเมินประสิทธิภาพได้ร้อยละ 100

4.2.9 Gate Exit

จากการทดสอบใช้งานจริงพบว่า สามารถรับ API จากการชำระเงินสำเร็จ เพื่อเปิดประตูได้แต่ต้องยึดตัวเซอร์โวมอเตอร์ให้แน่นขึ้น หลังปรับแต่งการยึดทำให้สามารถใช้ได้จริงตามเป้าหมาย สามารถประเมินประสิทธิภาพได้ร้อยละ 100

4.3 การพัฒนา

4.3.1 Pusher Remodel

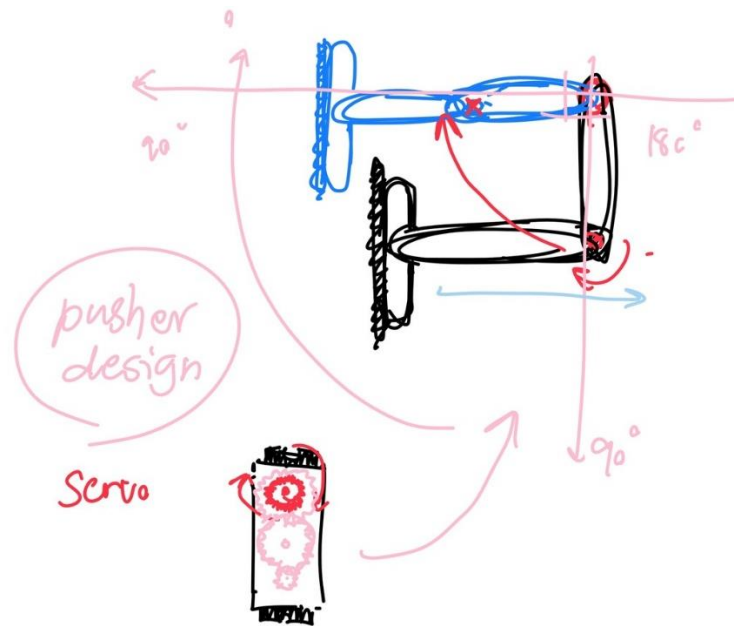


ภาพที่ 45 Pusher Remodel Design

จากการที่ทดสอบการทำงานในสัปดาห์ที่ 2-3 พบเจอปัญหา ที่ทำให้เกิดการพัฒนาสู่แบบปัจจุบัน ดังนี้

- 1) ระยะทางการผลักสั้นไป
- 2) แรงผลักไม่เพียงพอต่อการใช้งาน
- 3) โครงสร้างไม่มั่นคง

จึงได้นำมาแก้ไขตัวหลักแต่ฐานให้มั่นคงมากยิ่งขึ้น ดังภาพข้างต้น และ ภาพต่อไปนี้



ภาพที่ 46 Pusher Arm Design

เนื่องจากได้นำเซอร์โวมอเตอร์ 180 องศา มาประยุกต์โดยการทำให้เกิดแรงจากได้น้ำมัน ของไม้ไผ่ที่ได้น้ำมันอัดไว้แต่แรงที่ส่งผ่านจากแรงอัดของไม้ไผ่ที่ติดกับตัวแกนหมุนเซอร์โวมอเตอร์ยังไม้ไผ่ที่ขันน็อตไว้ทำให้เกิดการดันไปด้านหน้า ซึ่งจะทำให้เกิดการผลักรวดที่มาจากข้างใต้ ซึ่งหากไม้ไผ่ยาวเกินไปจะทำให้แรงที่ส่งผ่านลดลง จนไม่สามารถดันวัตถุได้ตามเป้าหมาย ซึ่งการทำงานของ Pusher นั้นเหมือนการทำงานของแขน ขา ของมนุษย์ที่ยึดติดได้ด้วยการงอหรือพับข้อต่อ ในการเพิ่มหรือลดระยะแขน ขา

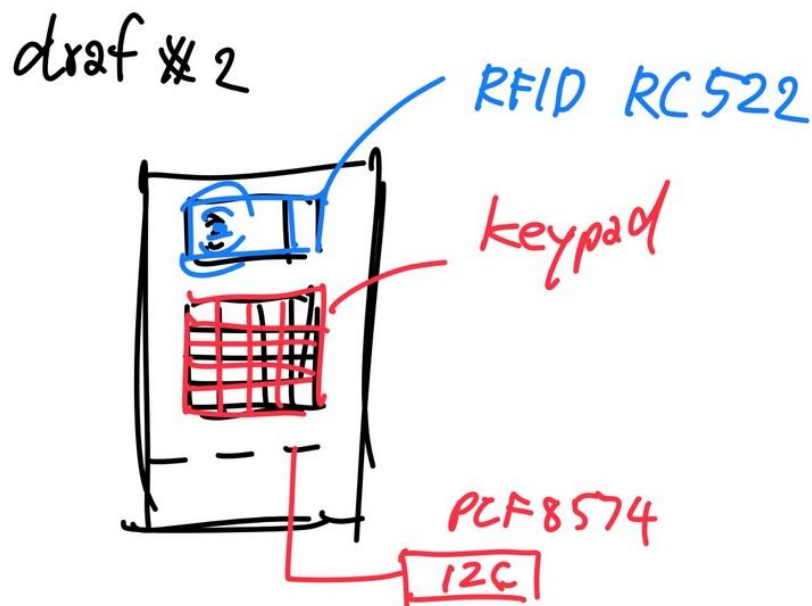
การแก้ไข

- 1) ลดความยาวของไม้ไผ่ที่ยึดของไปผลักรวด ทำให้มีแรงส่งผ่านเหลือมากขึ้น
- 2) เสริมความแข็งแรงของแกนให้มากขึ้น ทำให้รองรับแรงได้อย่างเต็มที่

ผลลัพธ์

จากการทดสอบการปรับแต่งชิ้นงานพบว่ามีความแข็งแรงเพิ่มขึ้น จนสามารถดันวัตถุได้ตามเป้าหมายและมีความคงทนมากขึ้นกว่าเดิม

4.3.2 Card Scanner & Keypad Remodel



ภาพที่ 47 Scanner and Keypad Remodel Design

จากการที่ทดสอบการทำงานในสัปดาห์ที่ 2-3 พบเจอปัญหา ที่ทำให้เกิดการพัฒนาสู่แบบปัจจุบัน ดังนี้

- 1) รูปแบบเดิมใช้พื้นที่เยอะเกินไป
- 2) รูปแบบเดิมแยกส่วนกันทำให้ดูไม่สะดวกต่อผู้ใช้งาน (ลูกค้า)
- 3) RFID ทำงานขัดข้องจากสายสัญญาณที่ยาวเกินไป

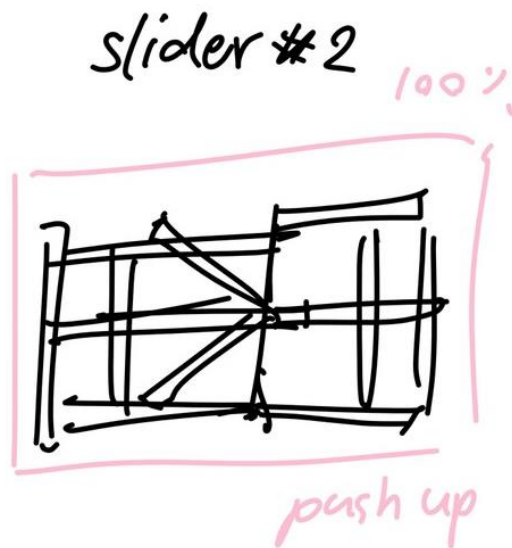
การแก้ไข

- 1) ลดขนาดพื้นที่
- 2) ลดความยาวสายไฟ

ผลลัพธ์

จากการทดสอบการปรับแต่งชิ้นงานพบว่าใช้งานได้ง่ายขึ้น ใช้พื้นที่ลดลง ทำงานได้อย่างถูกต้อง

4.3.3 Slider Frame



ภาพที่ 48 Slider Remodel Design

จากการที่ทดสอบการทำงานในสัปดาห์ที่ 2-3 พบเจอปัญหา ที่ทำให้เกิดการพัฒนาสู่แบบปัจจุบัน ดังนี้

- 1) โครงสร้างเดิมไม่แข็งแรง กระจายแรงได้ไม่ดี
- 2) แรงไม่เพียงพอต่อการยก

การแก้ไข

- 1) แก้ไขโครงให้กระจายแรงได้ดีขึ้น
- 2) เพิ่มจำนวนเซอร์โวมอเตอร์
- 3) เพิ่มการท่อน้ำหนักตรงส่วนปลาย

ผลลัพธ์

จากการทดสอบการปรับแต่งชิ้นงานพบว่า สามารถยกวัตถุได้ง่ายขึ้นและโครงสร้างมีความมั่นคงกว่าเดิม

4.4 ภาพรวมชิ้นงานทั้งหมด

เมื่อนำชิ้นงานทั้งหมดมาประกอบกันเป็นชิ้นเดียว จะเป็นดังภาพต่อไปนี้



ภาพที่ 49 Pick and Pay

ภาพรวมการทำงาน

จากการทดสอบสามารถทำงานร่วมกันได้แต่ในช่วงแรก พบเจอกับปัญหาไฟเลี้ยงไม่เพียงพอ เนื่องจากเมื่อถึงส่วนการทำงานของ Pusher and Slider ที่ใช้แรงขับของ Servo เยอะกว่าปกติทำให้เกิดการดึงแรงดันและกระแสแบบฉับพลัน จึงทำให้ไฟไม่เพียงพอต่อระบบอื่นๆ และเกิดการทำงานที่ไม่ถูกต้องดังเดิม

การแก้ไขปัญหา

จากการพูดคุยได้ข้อสรุปคือการต่อแหล่งจ่ายภายนอกให้แก่ Servo Slider 2 ตัว แยกกับระบบหลัก โดยที่นี้ใช้เป็น DEB Board แล้วต่อ Common Ground ร่วมกัน ผลที่ได้คือการทำงานกลับมาใช้ได้ตามปกติและตรงตามต้องการ

บทที่ 5

สรุปและข้อเสนอแนะ

5.1 สรุปโครงการ

โครงการ “Pick and Pay” เป็นการพัฒนาระบบร้านค้าไร้คนขายที่ผสมผสานการทำงานของไมโครคอนโทรลเลอร์ (Arduino, ESP32), อุปกรณ์ตรวจจับ (Load Cell, RFID, Ultrasonic, Sensor อื่น ๆ), ระบบควบคุม (Servo Motor, Gate System) และ ฐานข้อมูล เพื่อจำลองขั้นตอนการซื้อสินค้า ตั้งแต่การตรวจจับสินค้า การคำนวณราคา ไปจนถึงการยืนยันการชำระเงิน

ผลการดำเนินโครงการแสดงให้เห็นว่า ระบบสามารถทำงานได้จริงตามวัตถุประสงค์หลัก ได้แก่

- 1) การอำนวยความสะดวกแก่ผู้ประกอบการ: ลดต้นทุนแรงงาน, บริหารจัดการร้านจากระยะไกล และสามารถเปิดบริการได้ 24 ชั่วโมง
- 2) การเพิ่มความสะดวกแก่ลูกค้า: สามารถเลือกซื้อสินค้าได้ด้วยตนเอง ไม่ต้องรอพนักงาน, มีความเป็นส่วนตัว, และสามารถชำระเงินแบบอัตโนมัติ

แม้ว่าต้นแบบยังมีข้อจำกัดด้านความเสถียรของฮาร์ดแวร์และความแม่นยำของเซนเซอร์ แต่โครงการนี้ถือว่าประสบความสำเร็จในการพิสูจน์แนวคิด (Proof of Concept) ว่าระบบร้านค้าไร้คนขายสามารถทำได้จริงตามเป้าประสงค์และมีศักยภาพต่อยอดในอนาคต

5.2 ประเมินผลโครงการในมุมมองผู้จัดทำ

โครงการ Pick and Pay ได้รับการออกแบบและพัฒนาตามหลักการบูรณาการระหว่างฮาร์ดแวร์ ซอฟต์แวร์ และการออกแบบระบบ โดยมีผลการประเมินดังนี้

5.2.1 ด้านการทำงานของระบบ

- 1) การตรวจจับสินค้า: Load Cell สามารถตรวจจับน้ำหนักได้ในระดับที่เพียงพอสำหรับสินค้าประเภทขนมและเครื่องดื่ม แต่พบที่มีความคลาดเคลื่อนเมื่อมีการวางสินค้าหลายชิ้นพร้อมกัน หรือเมื่อสินค้าไม่ถูกวางในตำแหน่งที่กำหนดอย่างแม่นยำ
- 2) การระบุตัวตนผู้ใช้ด้วย RFID: สามารถทำงานได้สำเร็จในการยืนยันตัวตนของผู้ซื้อ อย่างไรก็ตาม ระยะการอ่านที่สั้น (ประมาณ 3–5 ซม.) ทำให้ผู้ใช้งานต้องวางบัตรในตำแหน่งที่เหมาะสม จึงอาจไม่สะดวกต่อผู้ใช้ทั่วไป

- 3) กลไกควบคุมประตูลูกและกล่องสินค้า: Servo Motor สามารถเปิด-ปิดได้ตามคำสั่ง แต่มีปัญหาด้านความเสถียรเมื่อทำงานต่อเนื่องเป็นเวลานาน เนื่องจากแรงดันไฟไม่คงที่
- 4) การบันทึกและประมวลผลข้อมูล: ฐานข้อมูลสามารถเก็บข้อมูลการซื้อขายได้ครบถ้วน แต่ยังมีค่าล่าช้าเล็กน้อยเมื่อต้องบันทึกหลายเหตุการณ์ต่อเนื่อง
- 5) กลไกการผลักและยกสินค้า สามารถทำงานได้ แต่ไม่ตรงตามที่วางไว้ในตอนเริ่มต้นให้สามารถแรงและมีเสถียรภาพมากกว่านี้

5.2.2 ด้านประสิทธิภาพและความถูกต้อง

- 1) ระบบโดยรวมสามารถทำงานได้ตามลำดับขั้นตอนที่ออกแบบ เช่น การตรวจจับสินค้า → คำนวณราคา → ยืนยันการชำระเงิน → เปิดประตูทางออก
- 2) ความถูกต้องของการตรวจจับสินค้าอยู่ในระดับ 80–90% ซึ่งถือว่าเหมาะสมสำหรับต้นแบบ แต่ยังต้องพัฒนาเพื่อให้ใกล้เคียง 100% หากจะนำไปใช้งานจริง
- 3) ความเร็วในการทำงานโดยเฉลี่ยใช้เวลา 10–15 วินาทีต่อการซื้อสินค้า 1 รายการ ซึ่งถือว่ายังช้ากว่าระบบร้านค้าจริงเล็กน้อย

5.2.3 ด้านผู้ใช้งาน

- 1) ผู้ใช้งานสามารถเรียนรู้วิธีการใช้งานได้ในเวลาไม่นาน แต่หากไม่มีคำอธิบายเบื้องต้น อาจเกิดความสับสนในขั้นตอนการยืนยันตัวตนหรือการชำระเงิน
- 2) UX/UI ยังอยู่ในระดับพื้นฐานและต้องอาศัยการปรับปรุงให้ง่ายต่อการเข้าใจ เช่น การใช้ข้อความแจ้งเตือนที่ชัดเจนขึ้น หรือการใช้หน้าจอสัมผัสแทนปุ่มกด

5.2.4 ด้านการเรียนรู้และการทำงานเป็นทีม

- 1) ผู้จัดทำได้ฝึกทักษะทั้งการต่อวงจรอิเล็กทรอนิกส์ การเขียนโค้ดไมโครคอนโทรลเลอร์ (Arduino UNO / ESP32) การออกแบบฐานข้อมูล รวมทั้งการใช้งานเครื่องจักรต่างๆ เช่น เครื่องตัด เครื่องเจาะ เครื่องขัดหยาบ
- 2) ได้ประสบการณ์จริงในการทำงานเป็นทีม การแก้ปัญหาเฉพาะหน้า และการปรับเปลี่ยนแผนงานตามทรัพยากรที่มีอยู่
- 3) ได้ตระหนักถึงข้อจำกัดของงบประมาณและเวลาที่มี ส่งผลให้ต้องเลือกใช้อุปกรณ์ที่คุ้มค่า และสามารถแก้ไขปัญหาเฉพาะหน้าได้อย่างเหมาะสม

5.3 ข้อเสนอแนะ

จากปัญหาและข้อจำกัดที่พบในการดำเนินโครงการ สามารถนำมาสรุปข้อเสนอแนะเพื่อการพัฒนาต่อยอดได้ดังนี้

5.3.1 ข้อเสนอแนะด้านฮาร์ดแวร์

- 1) ควรใช้อุปกรณ์ที่มีคุณภาพสูงขึ้น เช่น Load Cell ความละเอียดสูงเพื่อลดความคลาดเคลื่อน, RFID Reader ที่รองรับการอ่านได้ไกลขึ้น (10–20 ซม.) และ Servo Motor ที่มีแรงบิดมากขึ้นเพื่อรองรับการทำงานต่อเนื่อง
- 2) เพิ่มการจ่ายไฟแยกสำหรับเซนเซอร์และมอเตอร์ เช่น ใช้ Power Supply ภายนอก หรือ วงจรแปลงแรงดันแบบ DC-DC Converter เพื่อลดปัญหาแรงดันตก
- 3) เสริมความทนทานของโครงสร้างทางกล เช่น การทำกล่องสินค้าและประตูด้วยวัสดุที่แข็งแรงขึ้น เพื่อลดความเสียหายเมื่อใช้งานจริง

5.3.2 ข้อเสนอแนะด้านซอฟต์แวร์

- 1) พัฒนาโค้ดให้มีระบบตรวจจับข้อผิดพลาด (Error Handling) เช่น กรณีเซนเซอร์ไม่ตอบสนอง หรือการอ่าน RFID ล้มเหลว
- 2) ปรับปรุงการจัดการฐานข้อมูลให้ทำงานได้เร็วขึ้น และรองรับการใช้งานพร้อมกันหลายรายการ (Concurrency)
- 3) พัฒนาระบบแจ้งเตือนผ่านหน้าจอหรือแอปพลิเคชัน เพื่อให้ผู้ใช้เข้าใจขั้นตอนการซื้อสินค้าอย่างชัดเจน

5.3.3 ข้อเสนอแนะด้านการใช้งานจริง

- 1) เพิ่มระบบรักษาความปลอดภัย เช่น กล้องวงจรปิด AI ที่ตรวจสอบพฤติกรรมการซื้อขาย, ระบบล็อกสองชั้น และเซนเซอร์ตรวจสอบการเปิดประตูผิดปกติ
- 2) ขยายรูปแบบการชำระเงินให้หลากหลาย เช่น QR Code, e-Wallet, NFC Payment หรือ Mobile Banking เพื่ออำนวยความสะดวกแก่ลูกค้าหลากหลายกลุ่ม
- 3) พิจารณาออกแบบระบบให้รองรับลูกค้าหลายคนพร้อมกัน (Multi-user) ซึ่งใกล้เคียงกับร้านค้าจริงมากกว่า

5.3.4 ข้อเสนอแนะด้านการพัฒนาในอนาคต

- 1) ศึกษาการนำ Computer Vision และ AI มาประยุกต์ใช้เพื่อตรวจจับสินค้าแทน RFID เพื่อลดความยุ่งยากในการทำงาน
- 2) พัฒนาแอปพลิเคชันสมาร์ทโฟนเพื่อให้ผู้ใช้งานสามารถตรวจสอบตะกร้าสินค้าแบบเรียลไทม์และชำระเงินผ่านมือถือ
- 3) ขยายระบบฐานข้อมูลไปสู่ Cloud Computing เพื่อรองรับการใช้งานร้านค้าหลายสาขาและการวิเคราะห์ข้อมูลพฤติกรรมการณ์การซื้อของลูกค้า
- 4) ปรับปรุง UX/UI ให้เหมาะสมกับผู้ใช้งานจริง โดยใช้หลัก Human-Centered Design เพื่อให้ระบบเป็นมิตรกับผู้ใช้งานทุกกลุ่ม

ภาคผนวก

Arduino Documentation. (2568, 6 กันยายน). Arduino® UNO R3 Datasheet [ออนไลน์]. เข้าถึงได้จาก : <https://docs.arduino.cc/resources/datasheets/A000066-datasheet.pdf>

Arduino Documentation. (2568, 6 กันยายน). UNO R3 – Arduino Hardware [ออนไลน์]. เข้าถึงได้จาก : <https://docs.arduino.cc/hardware/uno-rev3>

ESPRESSIF SYSTEMS. (2568, 7 กันยายน). ESP32 Series Datasheet [ออนไลน์]. เข้าถึงได้จาก : <https://www.alldatasheet.com/datasheet-pdf/pdf/1148023/ESPRESSIF/ESP32.html>

Hardkernel. (2568, 7 กันยายน). ODROID-C4 Product Specification [ออนไลน์]. เข้าถึงได้จาก : <https://wiki.odroid.com/odroid-c4/specifications>

SparkFun. (2568, 8 กันยายน). HX711 24-Bit ADC Datasheet [ออนไลน์]. เข้าถึงได้จาก : https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf

Adafruit. (2568, 8 กันยายน). HX711 24-bit ADC Module [ออนไลน์]. เข้าถึงได้จาก : <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-hx711-24-bit-adc.pdf>

DFRobot. (2568, 8 กันยายน). HX711 Load Cell Kit – Specification [ออนไลน์]. เข้าถึงได้จาก : https://wiki.dfrobot.com/HX711_Weight_Sensor_Kit_SKU_KIT0176

Morgan, E. J. (2568, 10 กันยายน). HC-SR04 Ultrasonic Sensor Datasheet [ออนไลน์]. เข้าถึงได้จาก : <https://www.alldatasheet.com/html-pdf/1132204/ETC2/HCSR04/110/1/HCSR04.html>

NUBWO. (2568, 11 กันยายน). NWC-560 Webcam Specification / Datasheet [ออนไลน์]. เข้าถึงได้จาก : <https://www.nubwo.co.th/webcamera-nwc560/>

Handson Technology. (2568, 13 กันยายน). SG90 Micro Servo Datasheet [ออนไลน์]. เข้าถึงได้จาก : https://handsontec.com/dataspecs/motor_fan/SG90-Servo.pdf

Tower Pro. (2568, 13 กันยายน). SG90-Analog [ออนไลน์]. เข้าถึงได้จาก : <https://towerpro.com.tw/product/sg90-analog/>

Tower Pro. (2568, 13 กันยายน). MG90S Micro Servo Datasheet [ออนไลน์]. เข้าถึงได้จาก : <https://towerpro.com.tw/product/mg90s-3/>

PARALLAX. (2568, 15 กันยายน). 4x4 Membrane Keypad Specification [ออนไลน์]. เข้าถึงได้จาก : <https://www.parallax.com/product/4x4-matrix-membrane-keypad/>

Texas Instruments. (2568, 15 กันยายน). PCF8574 / PCF8574A Datasheet [ออนไลน์]. เข้าถึงได้จาก : <https://www.ti.com/lit/ds/symlink/pcf8574.pdf>

Handson Technology. (2568, 15 กันยายน). RC522 Datasheet [ออนไลน์]. เข้าถึงได้จาก :

<https://www.handsontec.com/dataspecs/RC522.pdf>

Handson Technology. (2568, 17 กันยายน). 1602 LCD Datasheet [ออนไลน์]. เข้าถึงได้จาก :

https://www.handsontec.com/dataspecs/module/I2C_1602_LCD.pdf

Contact Technology Corp. (2568, 17 กันยายน). Tact Switch Specification Sheet [ออนไลน์].

เข้าถึงได้จาก : https://www.contactswitch.com/en/product_detail.aspx?id=1059

Cooper, T. (2568, 17 กันยายน). LED Datasheet / Specification [ออนไลน์]. เข้าถึงได้จาก :

<https://learn.adafruit.com/all-about-leds/the-led-datasheet>

Aosong Electronics Co. (2568, 20 กันยายน). DHT22 (AM2302) Datasheet [ออนไลน์]. เข้าถึงได้

จาก : <https://www.alldatasheet.com/html-pdf/1132459/ETC2/DHT22/109/1/DHT22.html>